

[Deep Learning Framework]

Multilayer NN

박성호

컴퓨터공학부

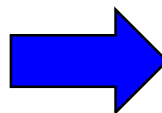
한국공학대학교

학습 목표

1. Multilayer NN 형태

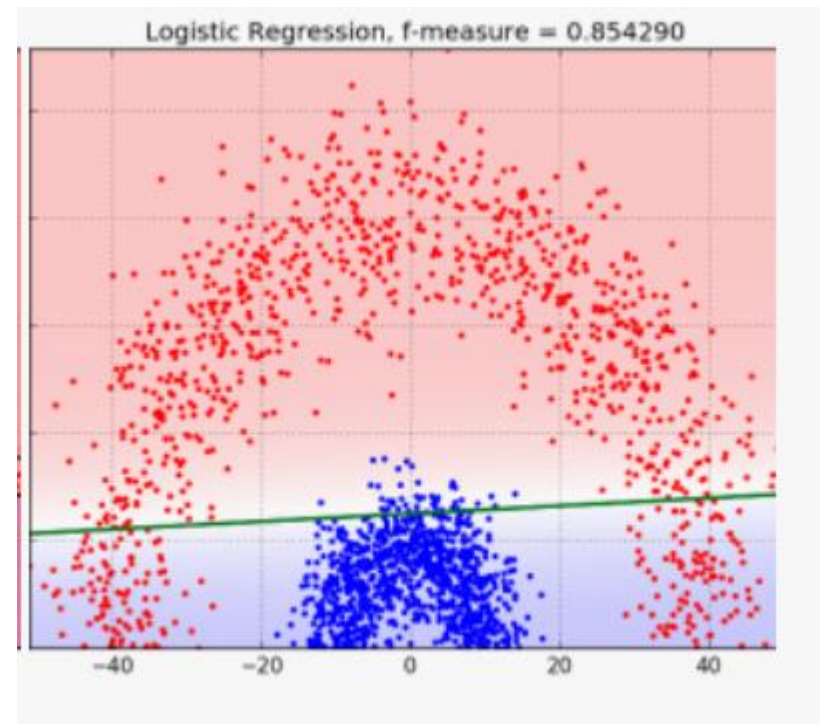
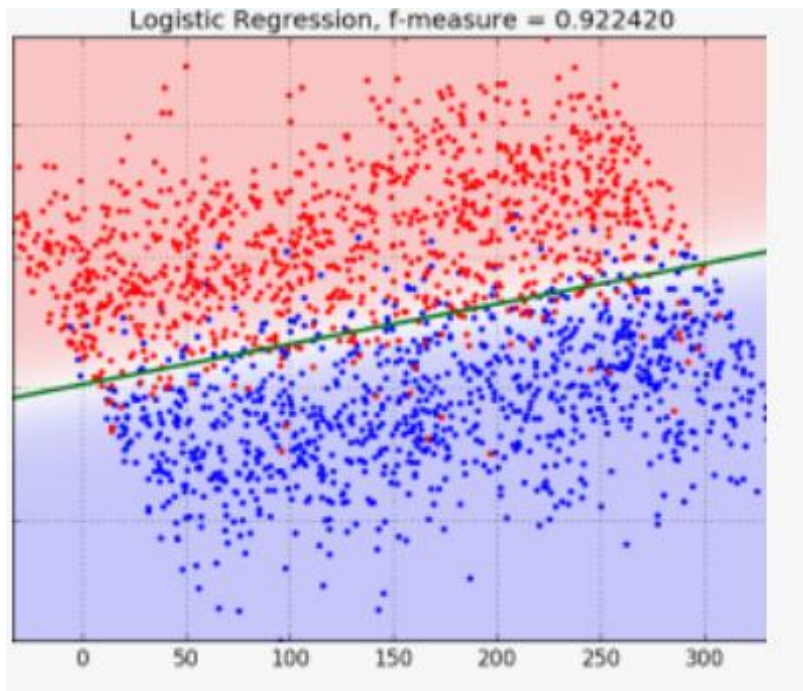
2. 학습

3. Multi-Class

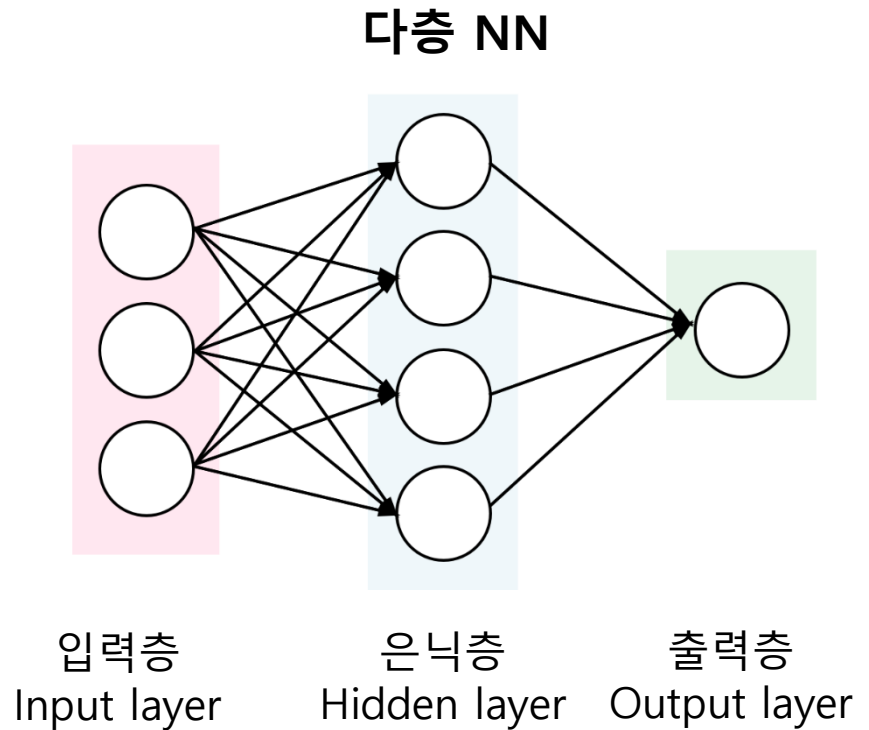
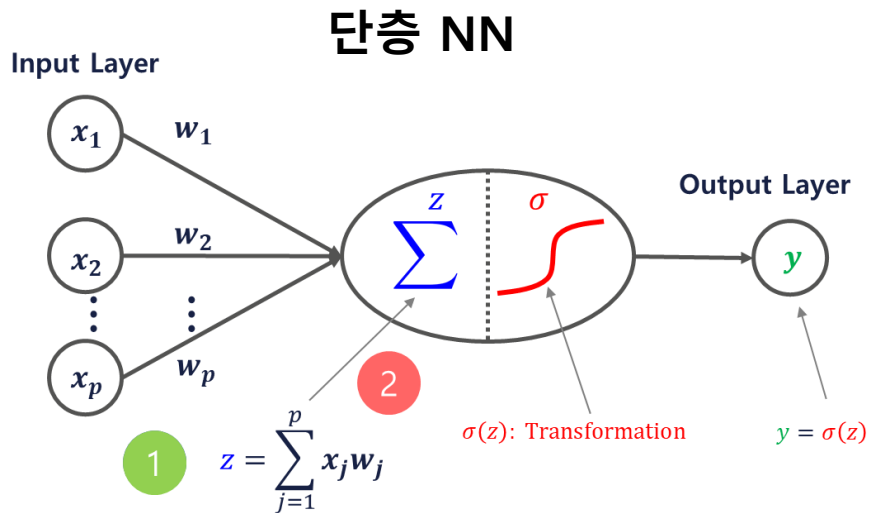


- Loss
- Optimization
- Decision Boundary
-

Perceptron Limitation - Decision Boundaries

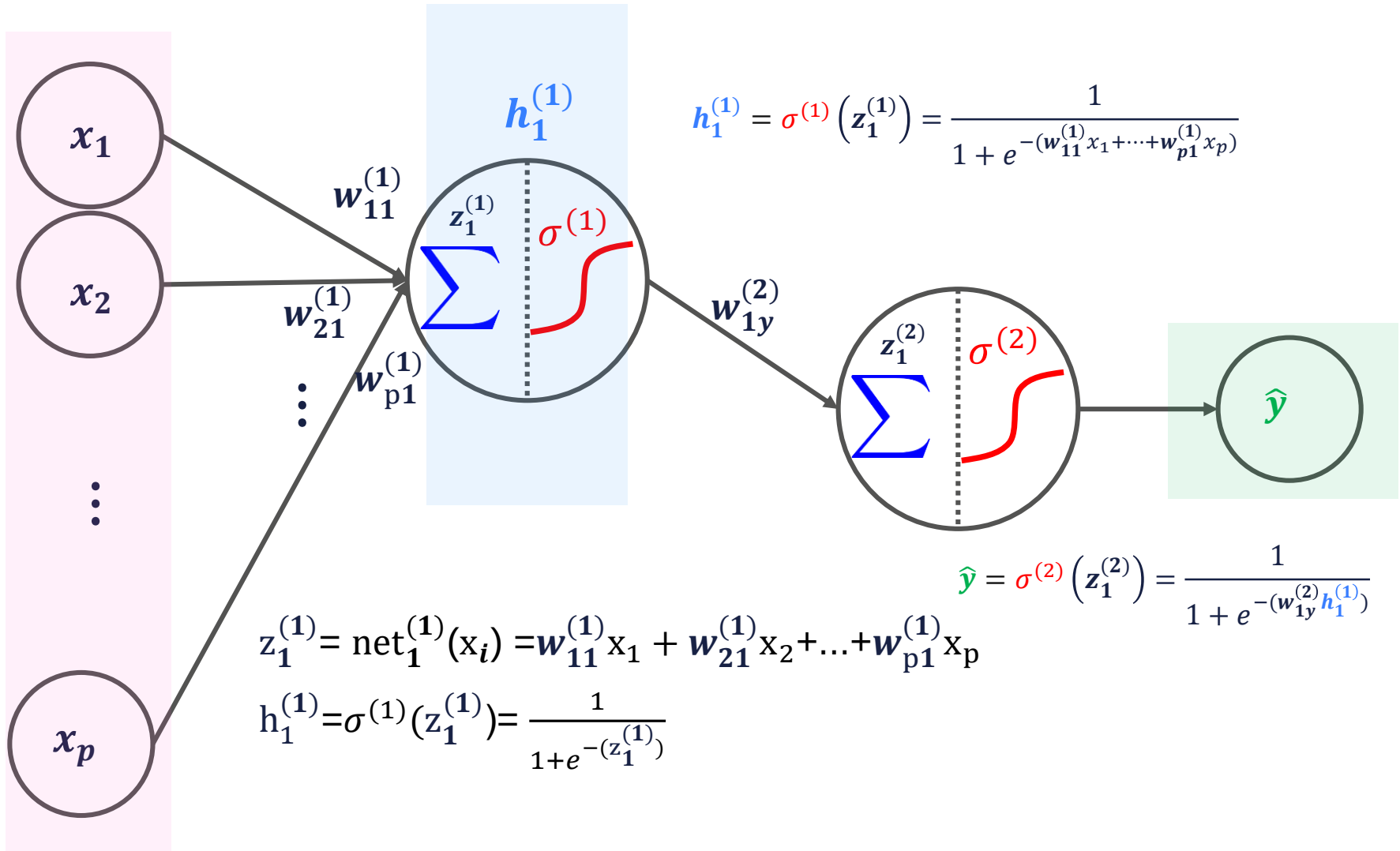


딥러닝

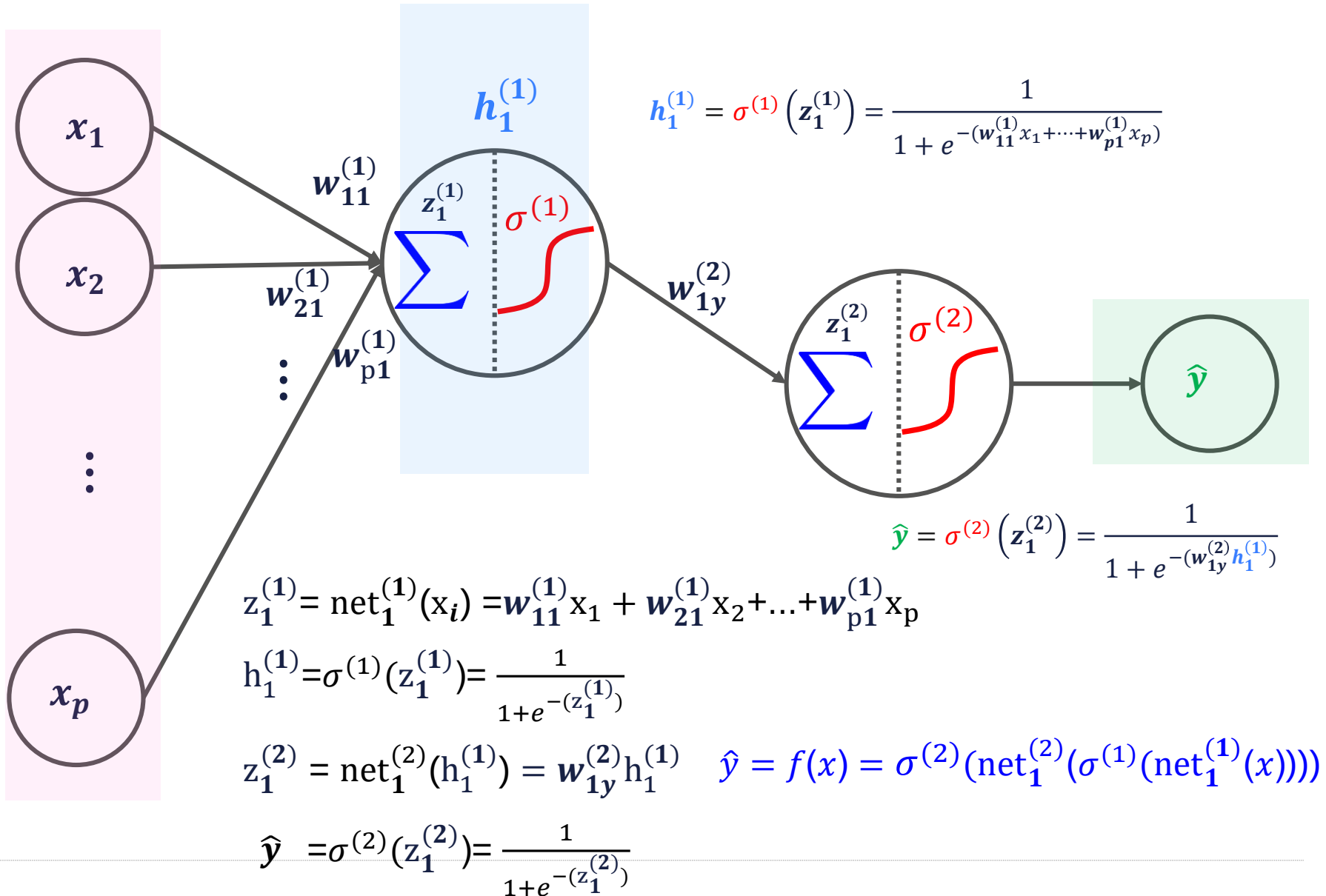


Source: <https://blog.goodaudience.com/artificial-neural-networks-explained-436fcf36e75>

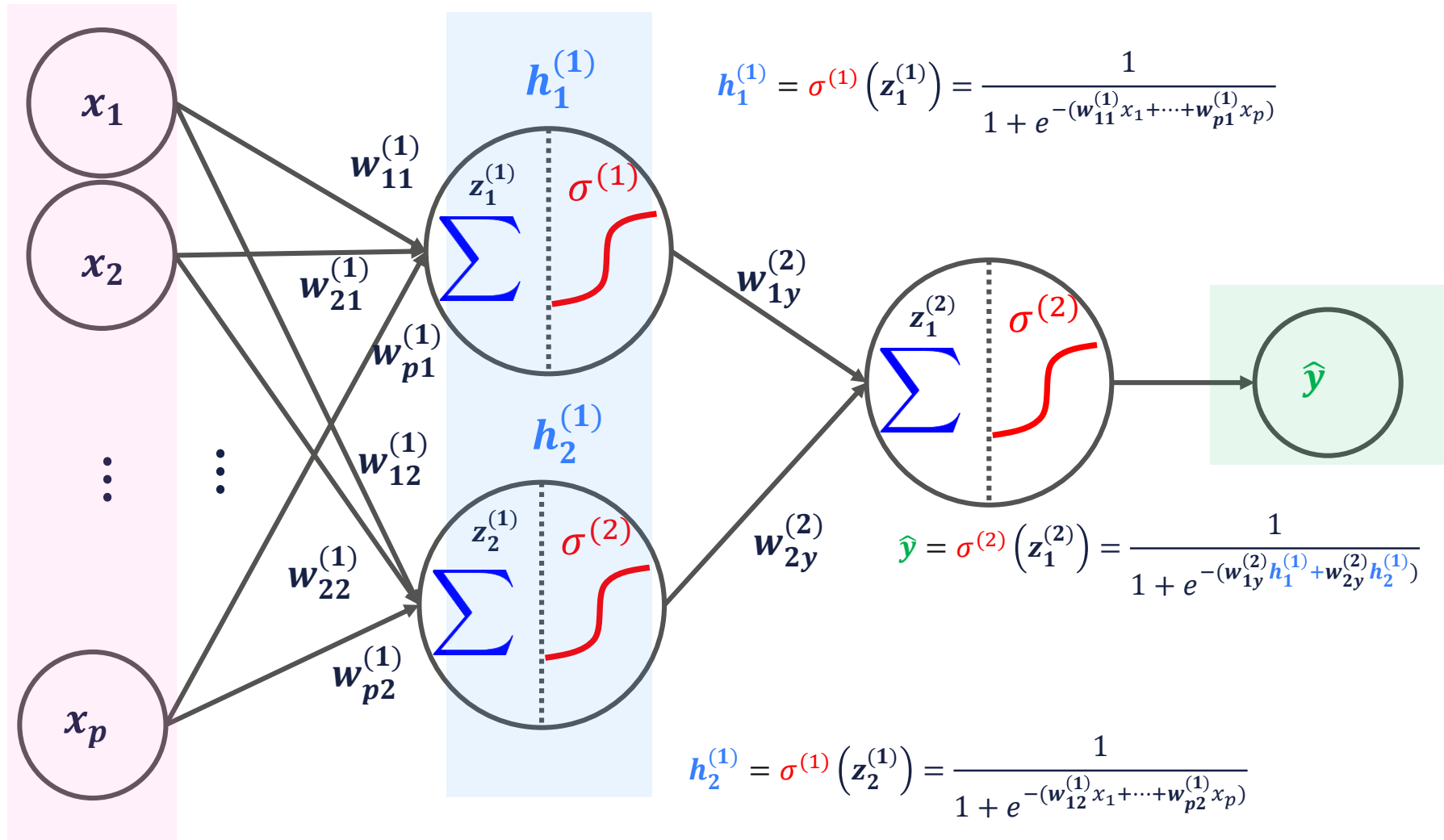
Multi-Layer Perceptron with One Hidden Layer



Multi-Layer Perceptron with One Hidden Layer

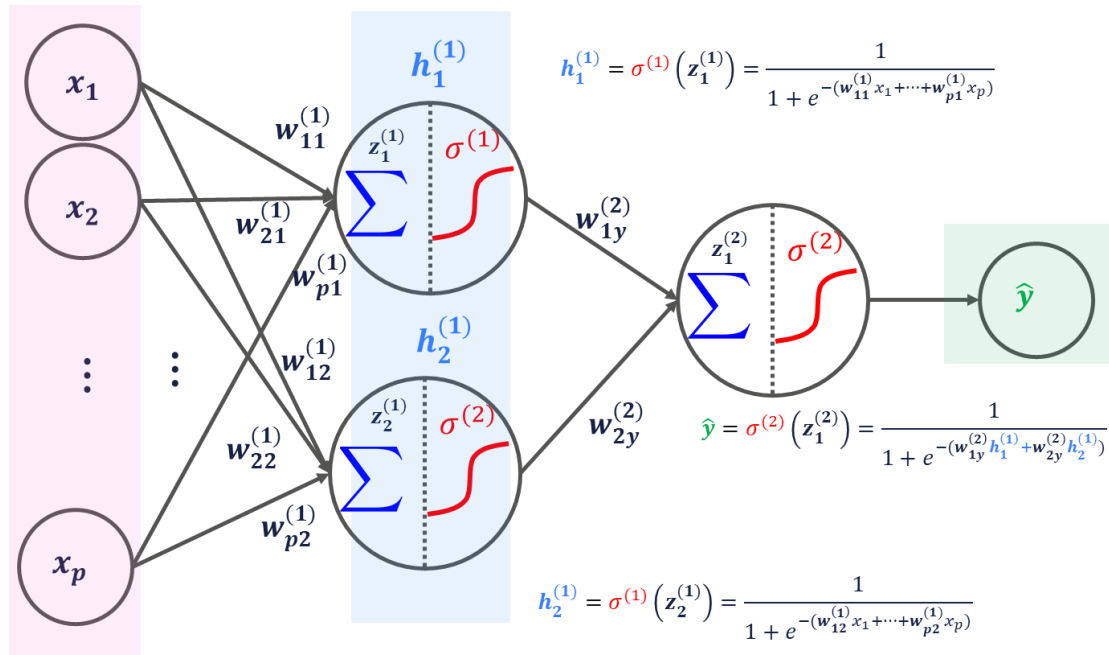


Multi-Layer Perceptron with One Hidden Layer



$$\hat{y}_i = f(x_i) = \sigma^{(2)}(\text{net}^{(2)}(\sigma^{(1)}(\text{net}^{(1)}(x_i))))$$

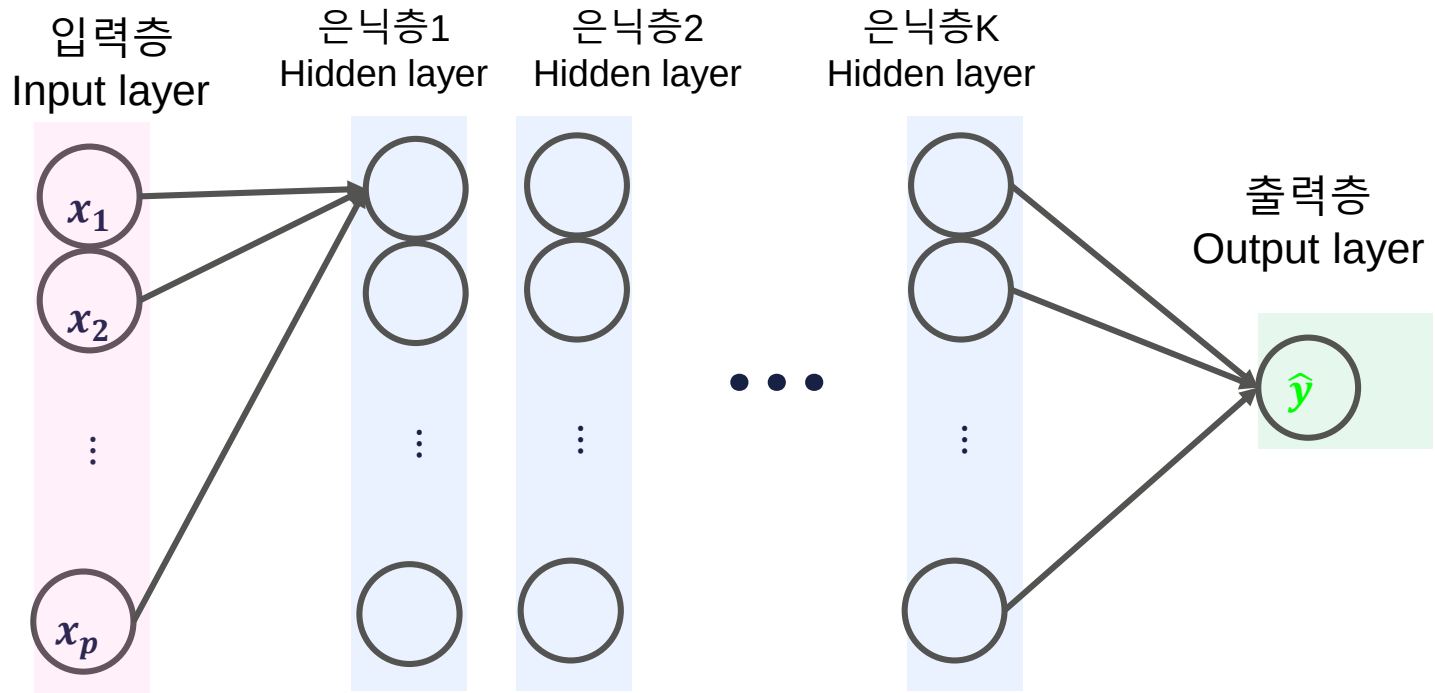
Multi-Layer Perceptron with One Hidden Layer



$$\hat{y}_i = f(x_i) = \sigma^{(2)}(net^{(2)}(\sigma^{(1)}(net^{(1)}(x_i))))$$

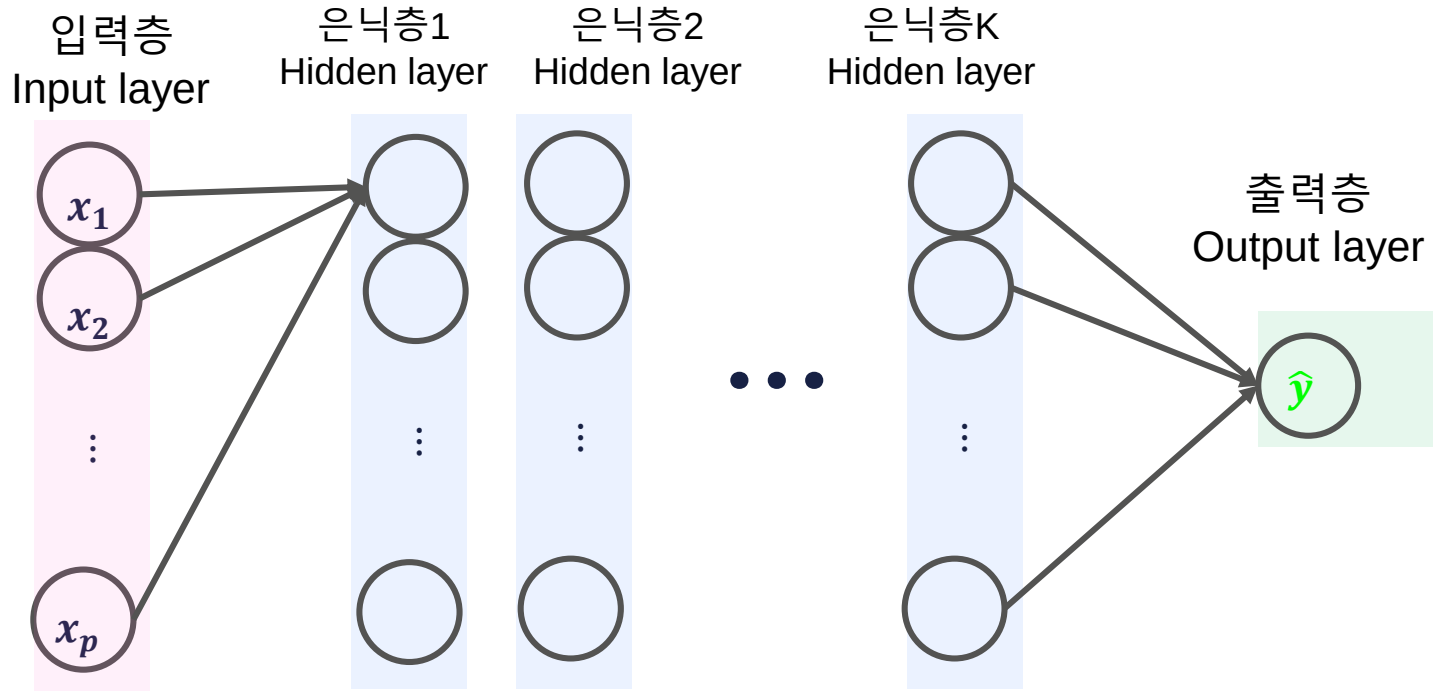
- 다층 퍼셉트론은 합성함수의 형태로 나타낼 수 있다.
- 선형조합 → 비선형 변환 → 선형조합 → 비선형 변환

Multi-Layer Perceptron -> 딥러닝



몇개의 선형과 비선형 함수의 합성곱??
생각해보기!!

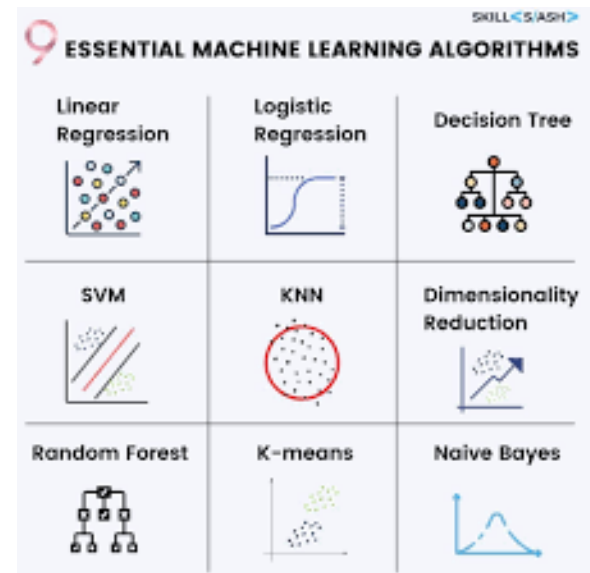
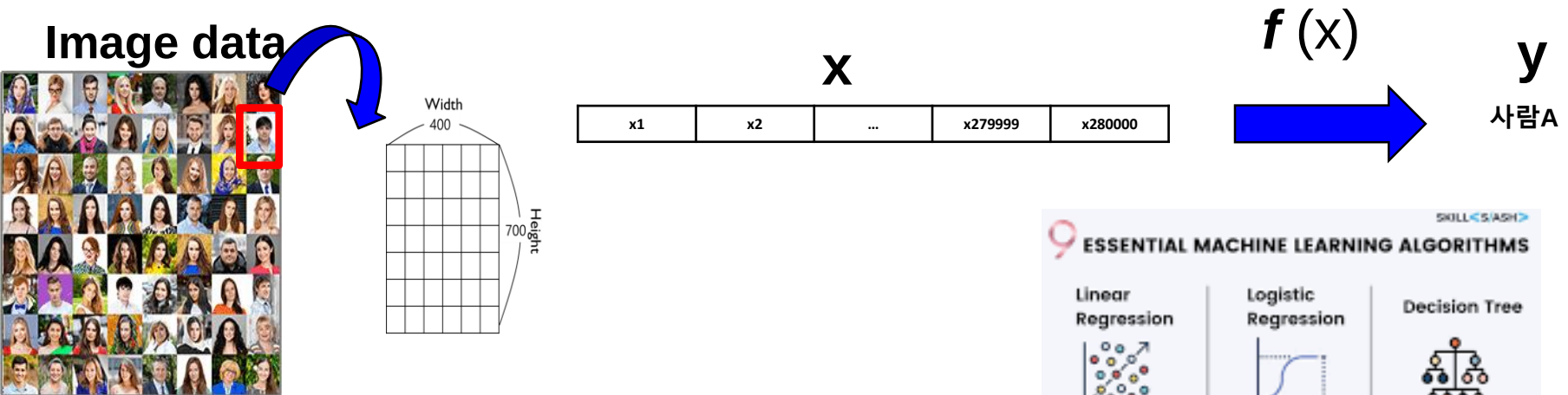
Multi-Layer Perceptron -> 딥러닝



$$\hat{y}_i = f(x_i) = \sigma^{(k+1)}(net^{(k+1)} \dots (\sigma^{(1)}(net^{(1)}(x_i))))$$

Why Deep Learning?

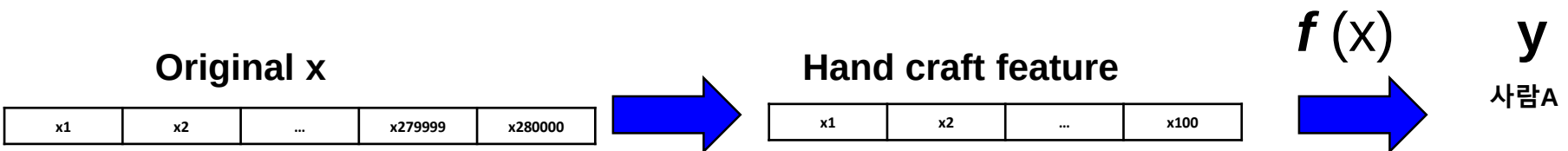
- 왜 여러 개의 레이어를 쌓으려고 하는가?
- 고차원의 데이터(이미지, 음성, 텍스트 등)에 대해 속성 (feature)을 정의하지 않고 원 데이터를 그대로 사용해도 알아서 속성을 판별할 수 있음



Why Deep Learning?

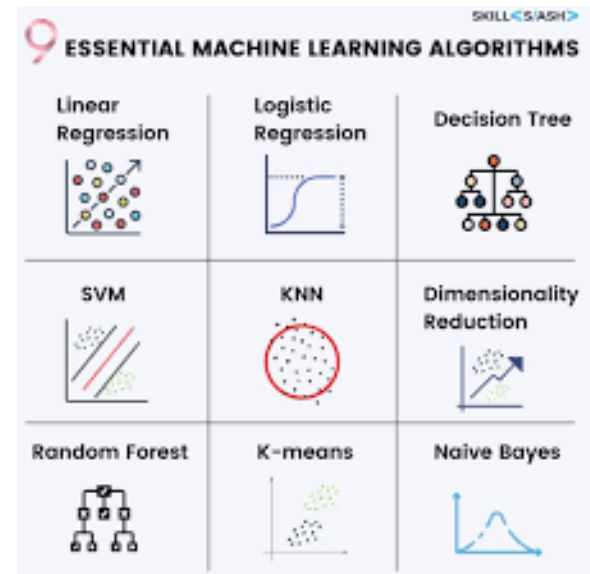
- 왜 여러 개의 레이어를 쌓으려고 하는가?
- 고차원의 데이터(이미지, 음성, 텍스트 등)에 대해 속성 (feature)을 정의하지 않고 원 데이터를 그대로 사용해도 알아서 속성을 판별할 수 있음

Feature engineering



What are features?

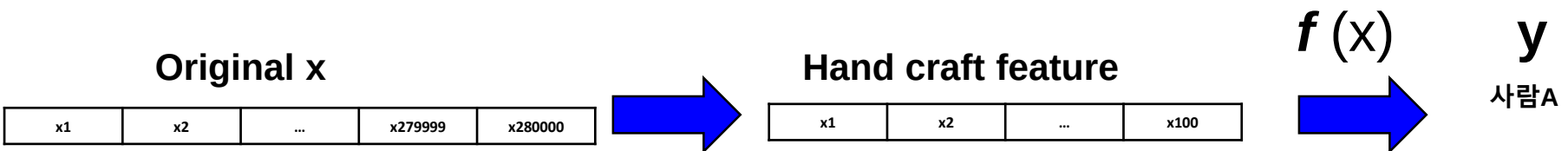
Features are parts or patterns of an object in an image that help to identify it. For example — a square has 4 corners and 4 edges, they can be called features of the square, and they help us humans identify it's a square. Features include properties like corners, edges, regions of interest points, ridges, etc.



Why Deep Learning?

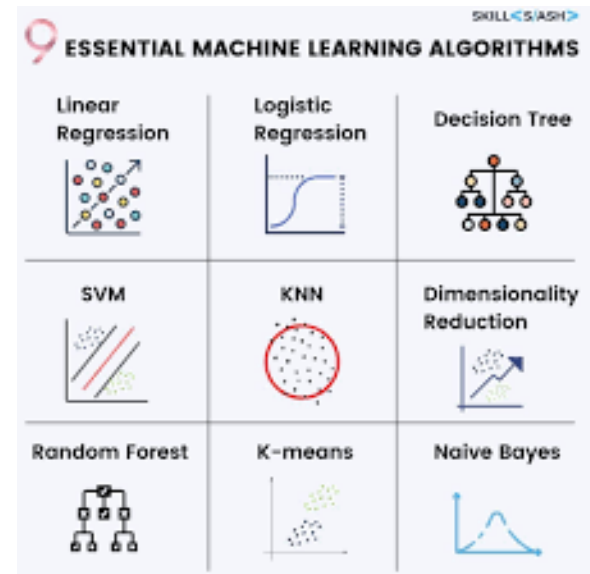
- 왜 여러 개의 레이어를 쌓으려고 하는가?
- 고차원의 데이터(이미지, 음성, 텍스트 등)에 대해 속성 (feature)을 정의하지 않고 원 데이터를 그대로 사용해도 알아서 속성을 판별할 수 있음

Feature engineering



Feature engineering is the process of transforming raw data into features that better represent the underlying problem to the predictive models, resulting in improved model accuracy on unseen data.

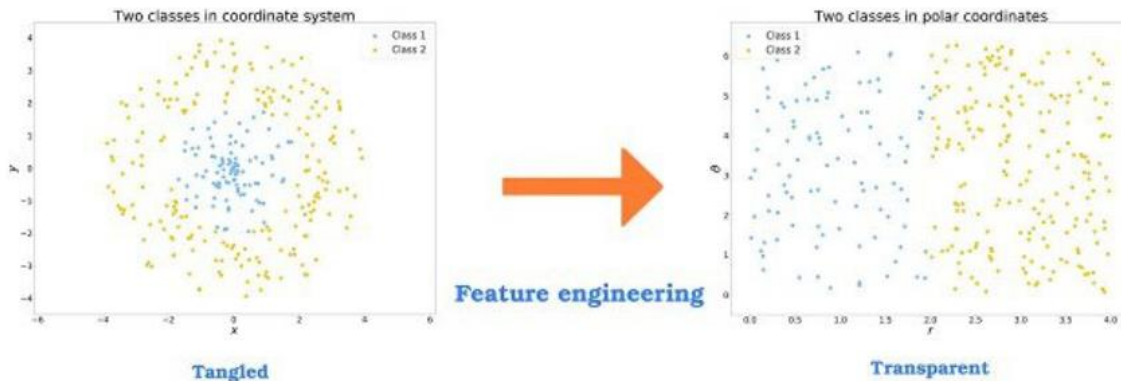
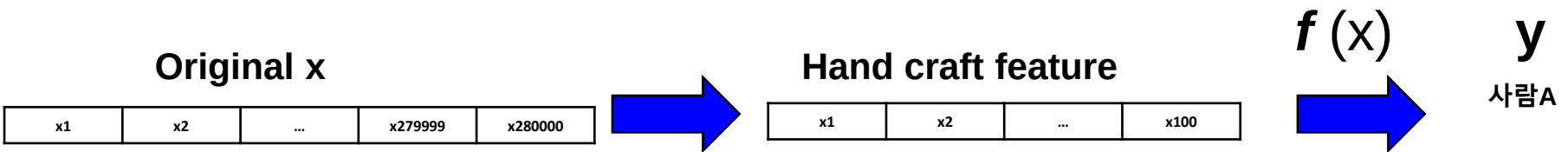
feature engineering asks: what is the best representation of the sample data to learn a solution to your problem?



Why Deep Learning?

- 왜 여러 개의 레이어를 쌓으려고 하는가?
- 고차원의 데이터(이미지, 음성, 텍스트 등)에 대해 속성 (feature)을 정의하지 않고 원 데이터를 그대로 사용해도 알아서 속성을 판별할 수 있음

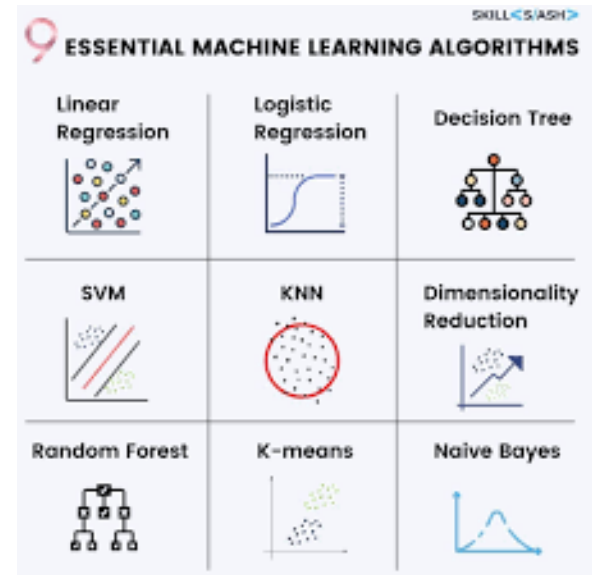
Feature engineering



From the human perspective, it is easy to understand that we need to consider the points in some limited radius from the some region.
Tree algorithm would require a lot of such splits.

$$r = \sqrt{x^2 + y^2} \quad \theta = \arctan \frac{y}{x}$$

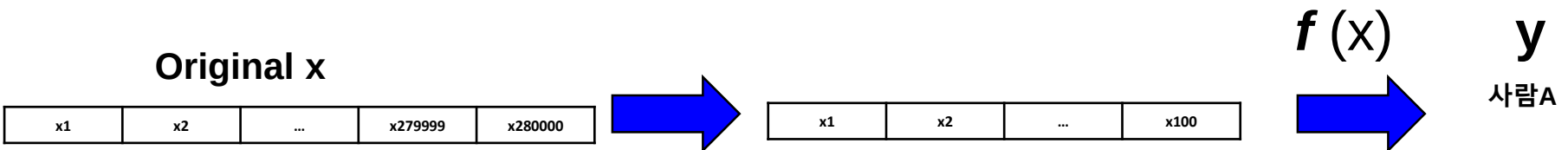
cartesian coordinates



Why Deep Learning?

- 왜 여러 개의 레이어를 쌓으려고 하는가?
- 고차원의 데이터(이미지, 음성, 텍스트 등)에 대해 속성 (feature)을 정의하지 않고 원 데이터를 그대로 사용해도 알아서 속성을 판별할 수 있음

Feature engineering

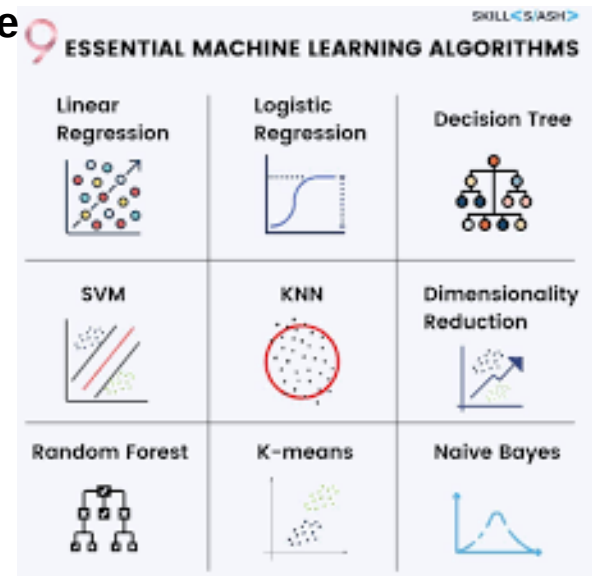


the area of the house and total price.

Sq Ft.	Amount
2400	9 Million
3200	15 Million
2500	10 Million
2100	1.5 Million
2500	8.9 Million

New feature

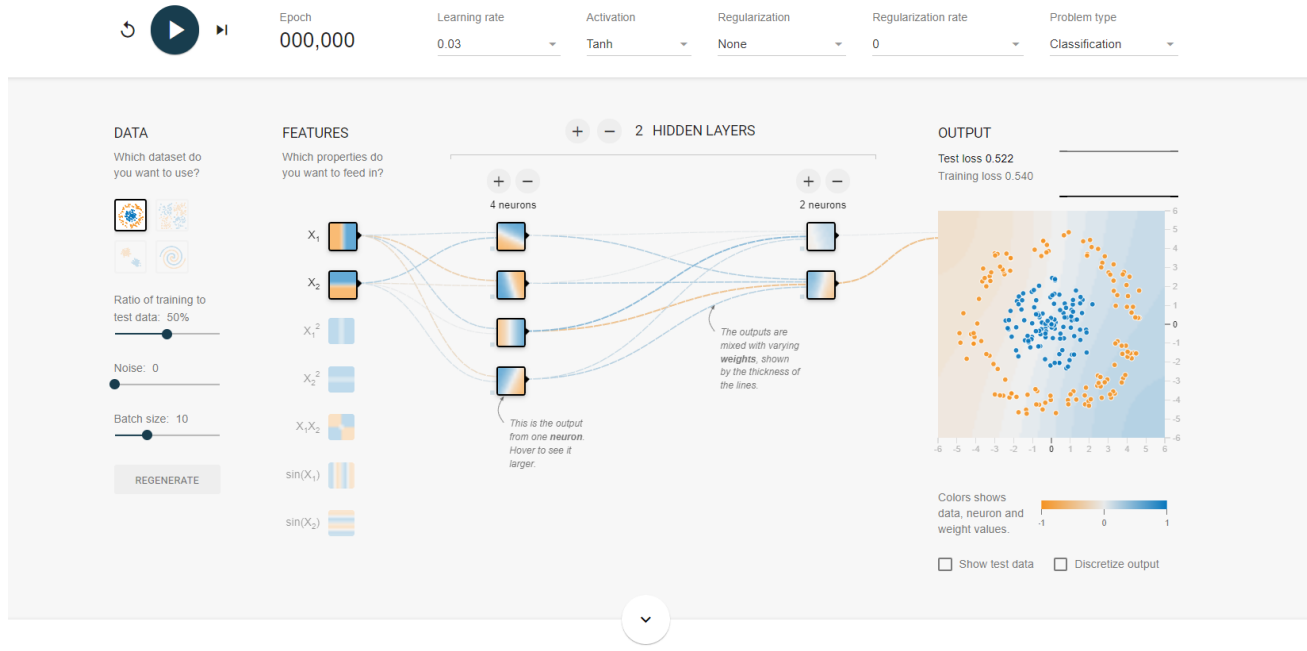
Sq Ft.	Amount	Cost Per Sq Ft
2400	9 Million	4150
3200	15 Million	4944
2500	10 Million	3950
2100	1.5 Million	510
2500	8.9 Million	3600



- This new feature will help us understand a lot about our data.
- use **Domain Knowledge** to contact a property

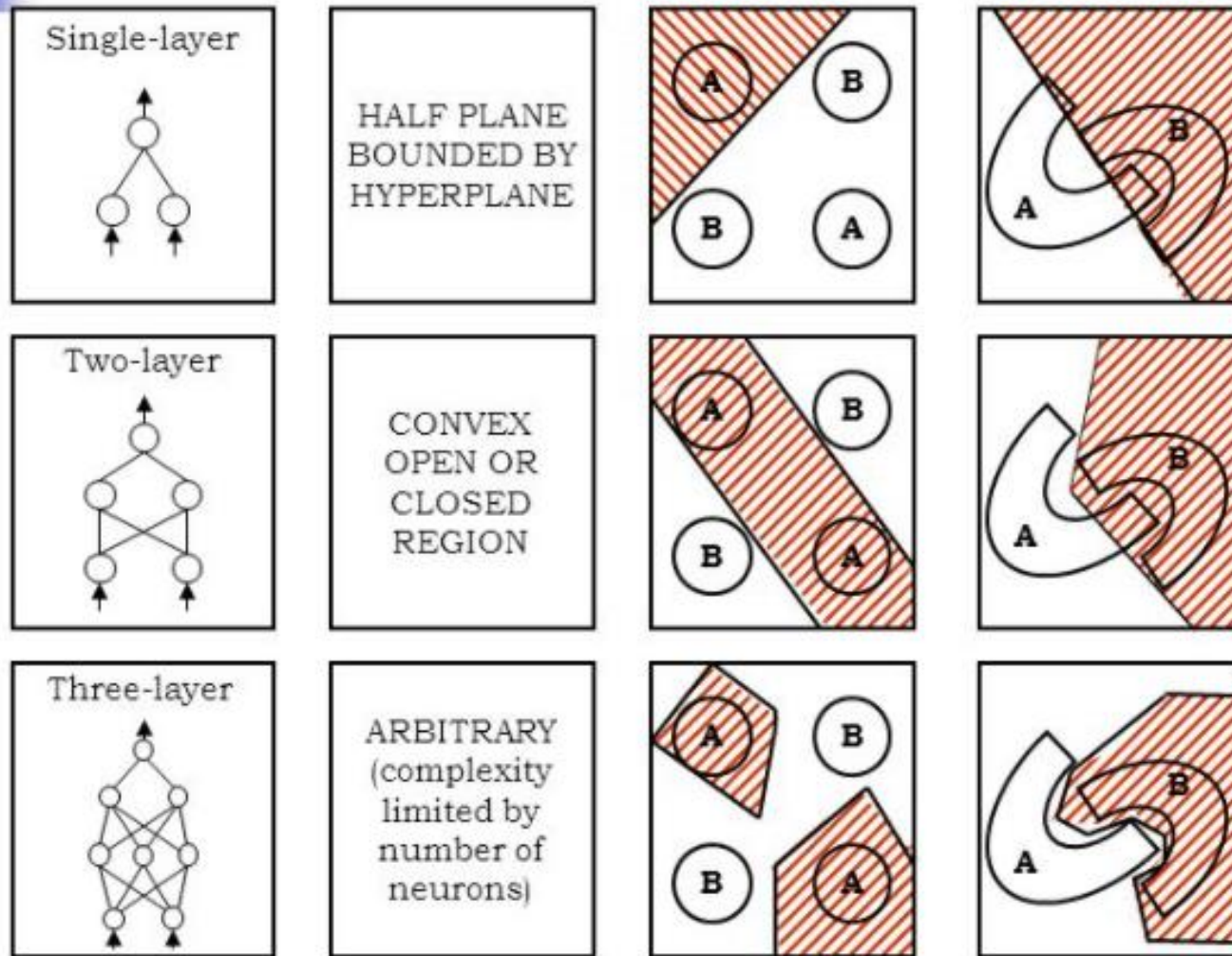
Why Deep Learning?

Tinker With a **Neural Network** Right Here in Your Browser.
Don't Worry, You Can't Break It. We Promise.

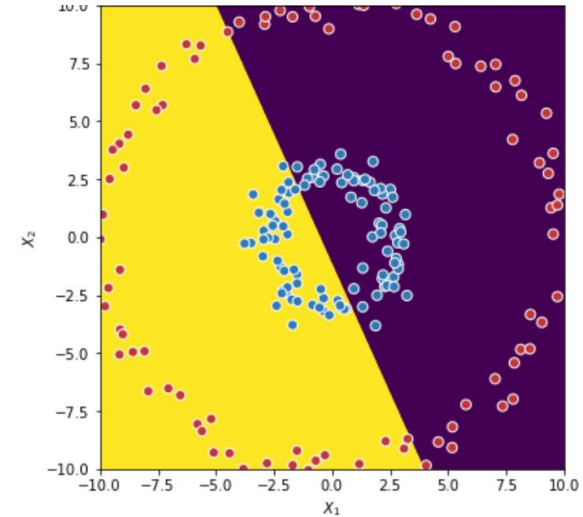
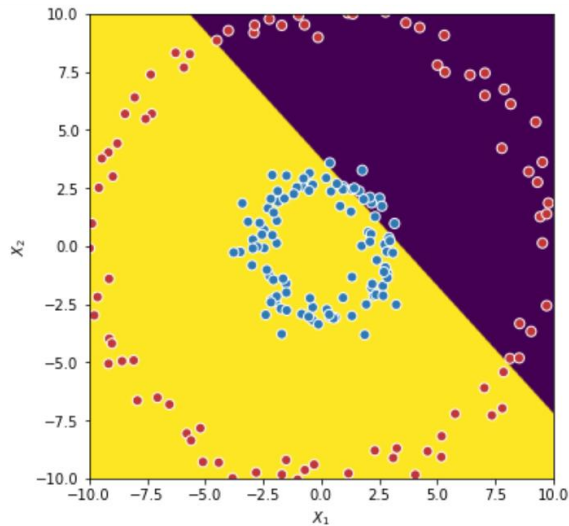


<https://playground.tensorflow.org/>

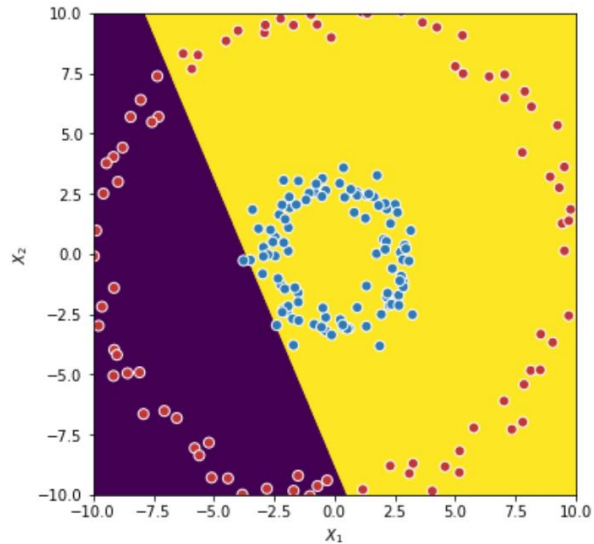
Multi-Layer Perceptron -> 딥러닝



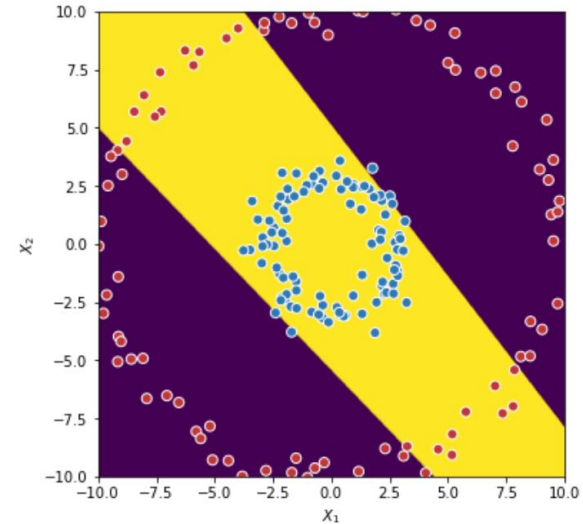
Multi-Layer Perceptron -> 딥러닝



Classifier(hidden_layer_sizes=(1),max_iter=500000,activation='logistic',learning_rate_init=0.01)



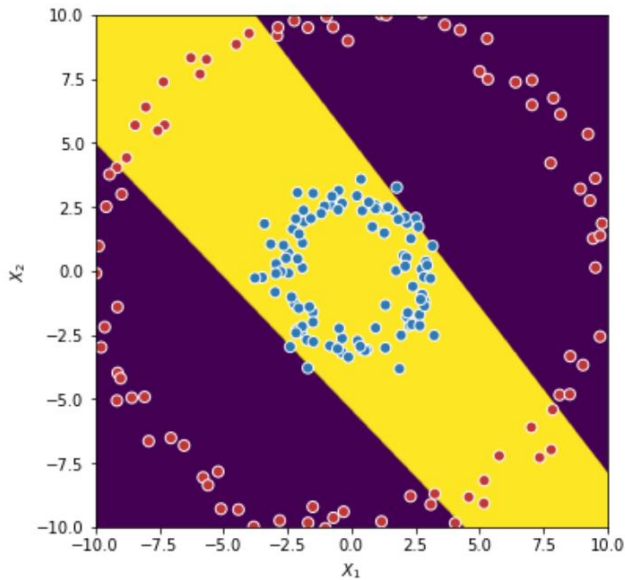
Classifier(hidden_layer_sizes=(1),max_iter=500000,activation='identity',learning_rate_init=0.01)



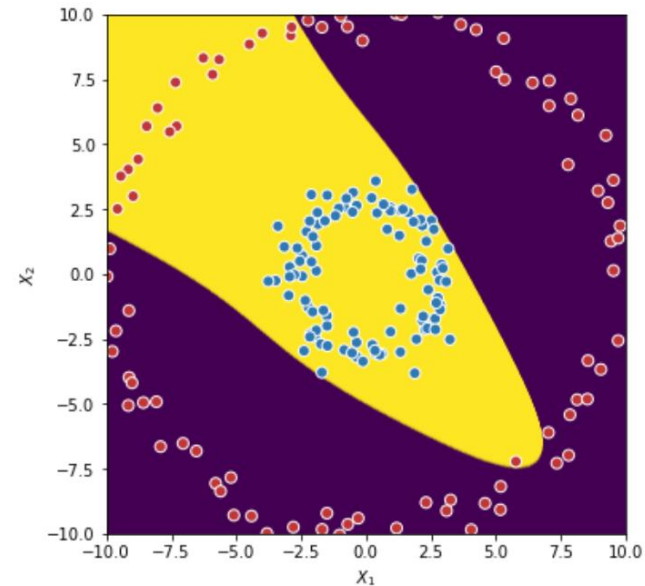
Classifier(hidden_layer_sizes=(1),max_iter=500000,activation='relu',learning_rate_init=0.01)

Non Linear problem with 1 layer and 2 neurons

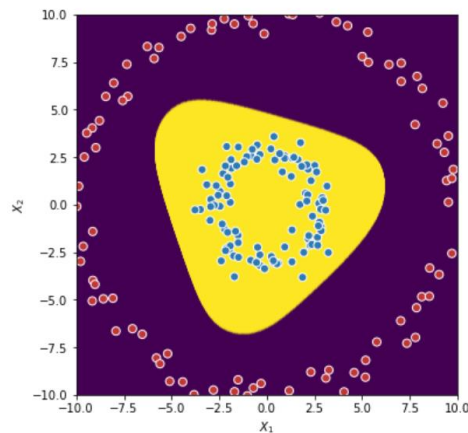
Multi-Layer Perceptron -> 딥러닝



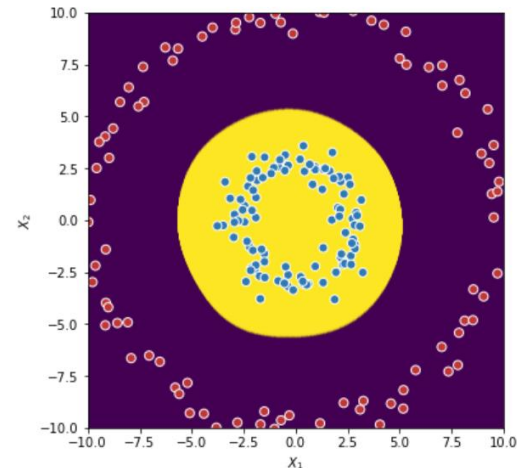
Non Linear problem with 1 layer and 2 neurons



Decision plane with 3 neurons : lower error than previous case.

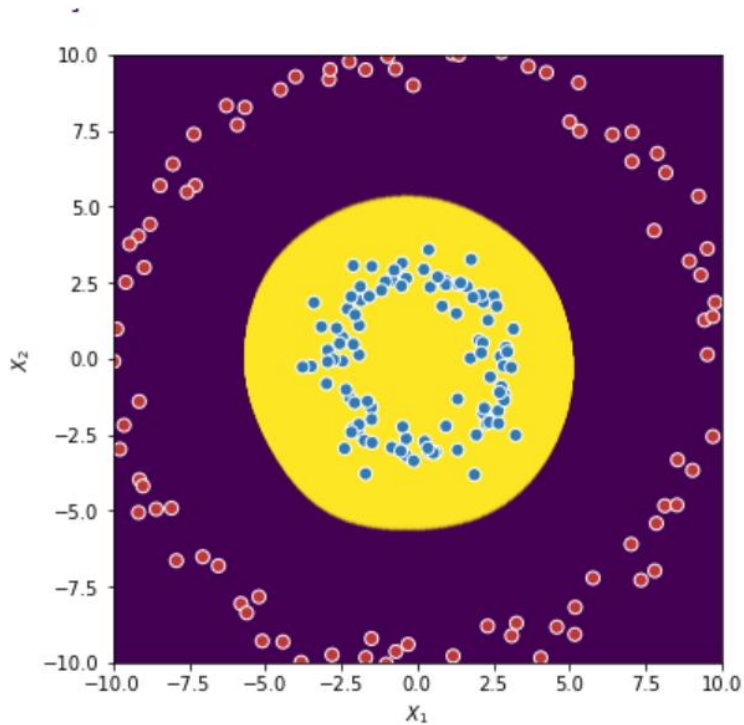


Decision plane with 6 neurons. Perfect classification with 100 % accuracy. Model now has learnt to combine 6 decisions to form a final decision. Points in yellow region are classified as blue and rest are classified as red

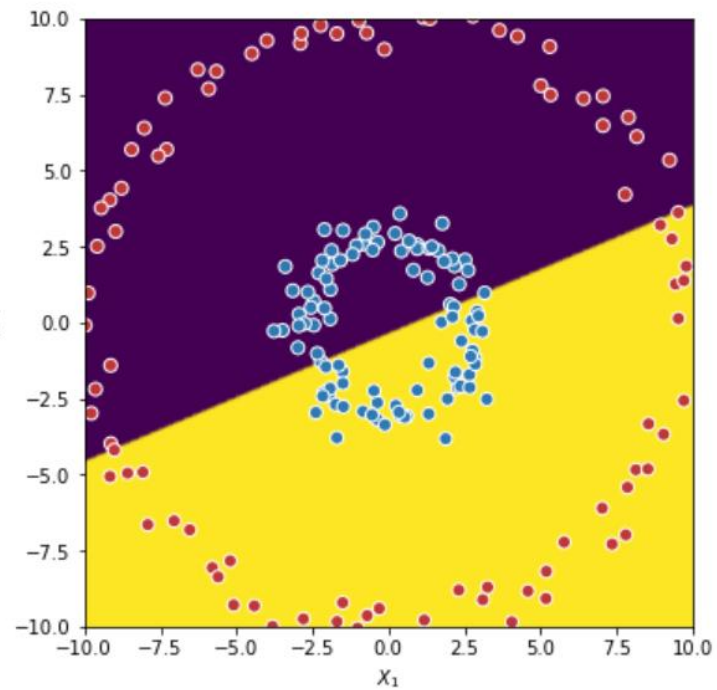


Decision plane with 30 neurons

Multi-Layer Perceptron -> 딥러닝

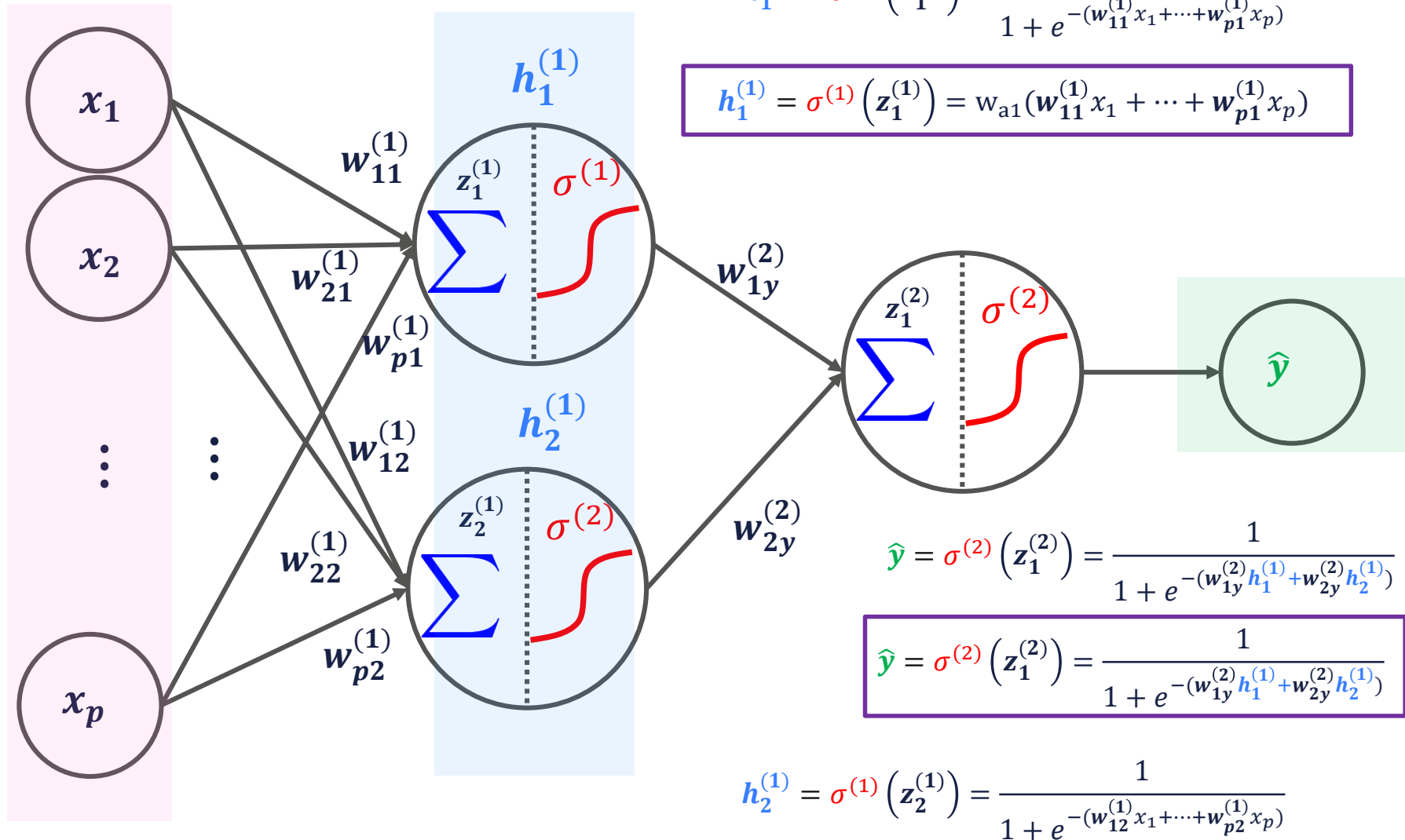


Decision plane with 30 neurons



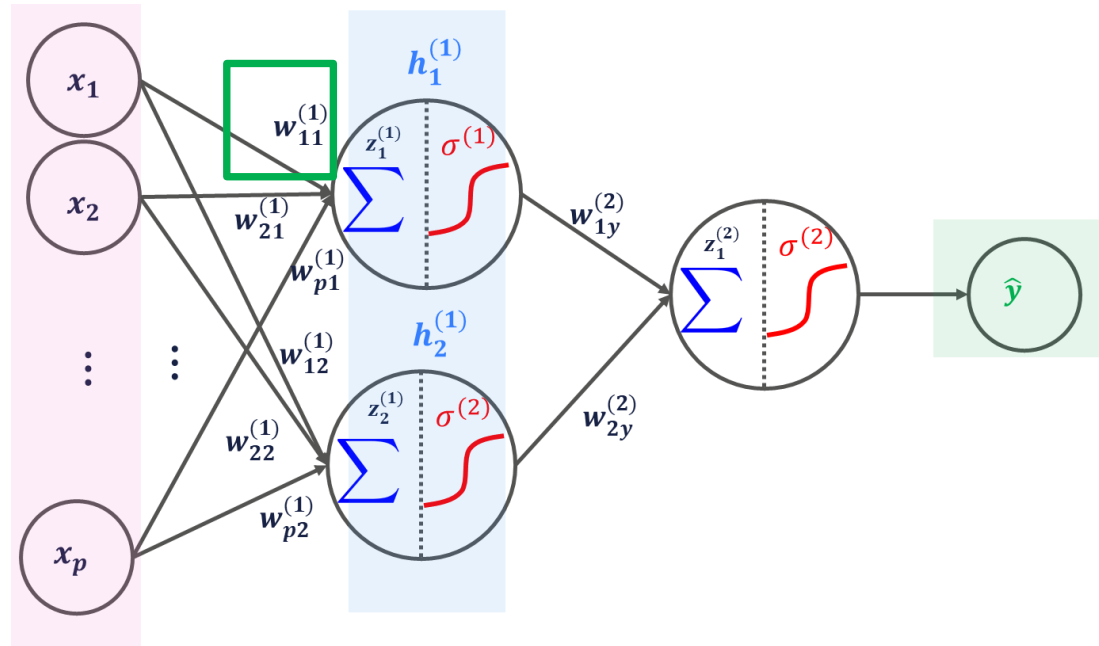
30 neurons with linear activation function

Multi-Layer Perceptron with One Hidden Layer



$$h_2^{(1)} = \sigma^{(1)}(z_2^{(1)}) = w_{a1}(w_{12}^{(1)}x_1 + \dots + w_{p2}^{(1)}x_p)$$

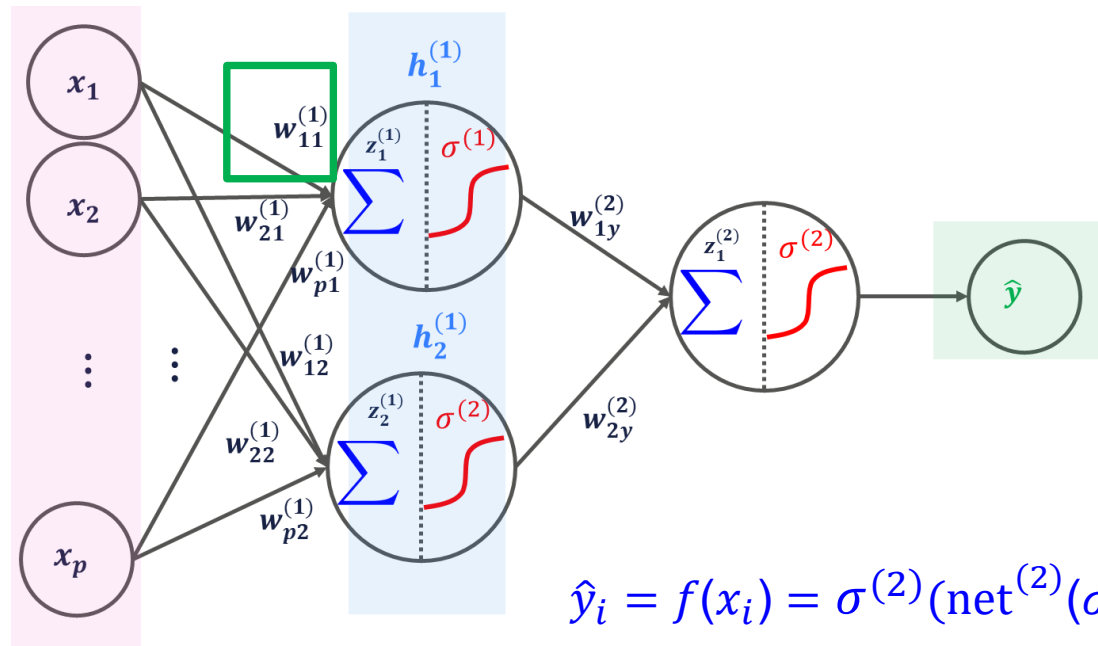
Multi-layer Perceptron Learning



$$\frac{dl_i}{dw_{11}^{(1)}} = ?$$

생각해보기!!

Multi-layer Perceptron Learning



$$\hat{y}_i = f(x_i) = \sigma^{(2)}(\text{net}^{(2)}(\sigma^{(1)}(\text{net}^{(1)}(x_i))))$$

$$\frac{dl_i}{dw_{11}^{(1)}} = ?$$

$$\frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dh_{i1}^{(1)}} \frac{dh_{i1}^{(1)}}{dz_{i1}^{(1)}} \frac{dz_{i1}^{(1)}}{dw_{11}^{(1)}}$$

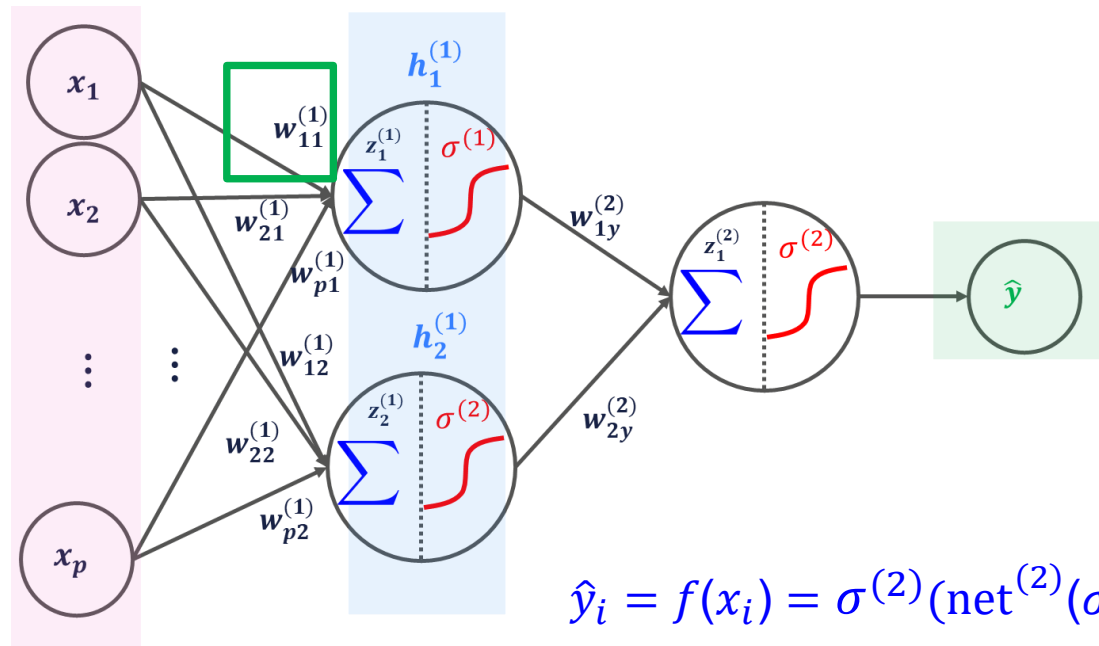
$$z_{i1}^{(1)} = \text{net}_1^{(1)}(x_i) = w_{11}^{(1)} x_{i1} + w_{21}^{(1)} x_{i2} + \dots + w_{p1}^{(1)} x_{ip}$$

$$h_{i1}^{(1)} = \sigma^{(1)}(z_{i1}^{(1)}) = \frac{1}{1 + e^{-(z_{i1}^{(1)})}}$$

$$z_{i1}^{(2)} = \text{net}_1^{(2)}(h_{i1}^{(1)}) = w_{1y}^{(2)} h_{i1}^{(1)}$$

$$\hat{y}_i = \sigma^{(2)}(z_{i1}^{(2)}) = \frac{1}{1 + e^{-(z_{i1}^{(2)})}}$$

Multi-layer Perceptron Learning

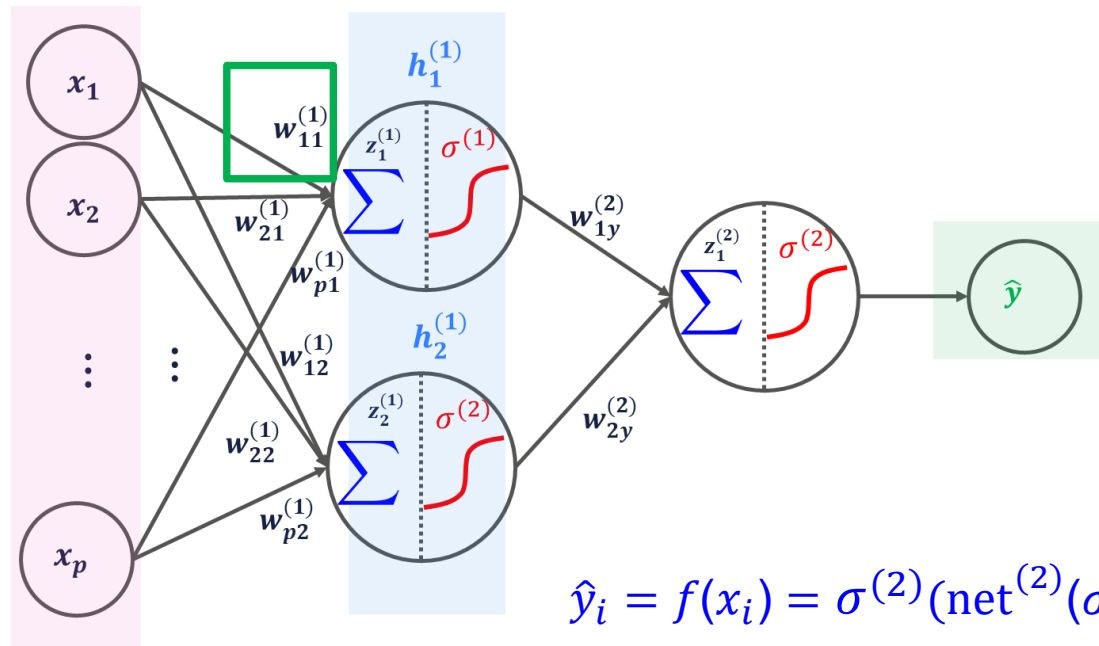


$$\hat{y}_i = f(x_i) = \sigma^{(2)}(\text{net}^{(2)}(\sigma^{(1)}(\text{net}^{(1)}(x_i))))$$

$$\frac{dl_i}{dw_{1y}^{(2)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{1y}^{(2)}}$$

$$\frac{dl_i}{dw_{11}^{(1)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dh_{i1}^{(1)}} \frac{dh_{i1}^{(1)}}{dz_{i1}^{(1)}} \frac{dz_{i1}^{(1)}}{dw_{11}^{(1)}}$$

Multi-layer Perceptron Learning

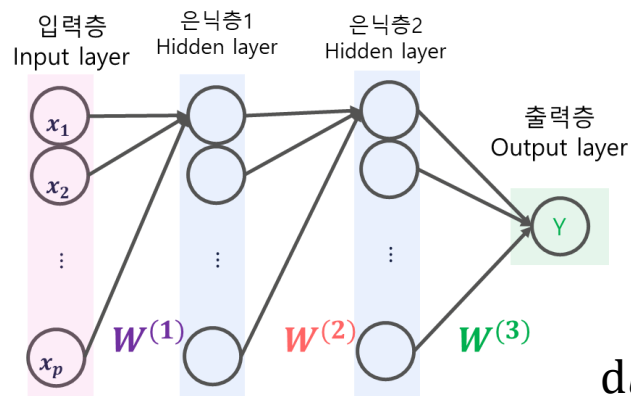


$$\hat{y}_i = f(x_i) = \sigma^{(2)}(\text{net}^{(2)}(\sigma^{(1)}(\text{net}^{(1)}(x_i))))$$

$$\frac{dl_i}{dw_{1y}^{(2)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{1y}^{(2)}}$$

$$\frac{dl_i}{dw_{11}^{(1)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dh_{i1}^{(1)}} \frac{dh_{i1}^{(1)}}{dz_{i1}^{(1)}} \frac{dz_{i1}^{(1)}}{dw_{11}^{(1)}}$$

Multi-layer Perceptron Learning



$$\hat{y}_i = f(x_i) = \sigma^{(3)}(\text{net}^{(3)}(\sigma^{(2)}(\text{net}^{(2)}(\sigma^{(1)}(\text{net}^{(1)}(x_i))))))$$

경사하강법

$$\frac{dl_i}{dw_{1y}^{(3)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dw_{1y}^{(3)}}$$

$$\frac{dl_i}{dw_{11}^{(2)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dh_{i1}^{(2)}} \frac{dh_{i1}^{(2)}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{11}^{(2)}}$$

$$\frac{dl_i}{dw_{11}^{(1)}} = \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dh_{i1}^{(2)}} \frac{dh_{i1}^{(2)}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dh_{i1}^{(1)}} \frac{dh_{i1}^{(1)}}{dz_{i1}^{(1)}} \frac{dz_{i1}^{(1)}}{dw_{11}^{(1)}}$$

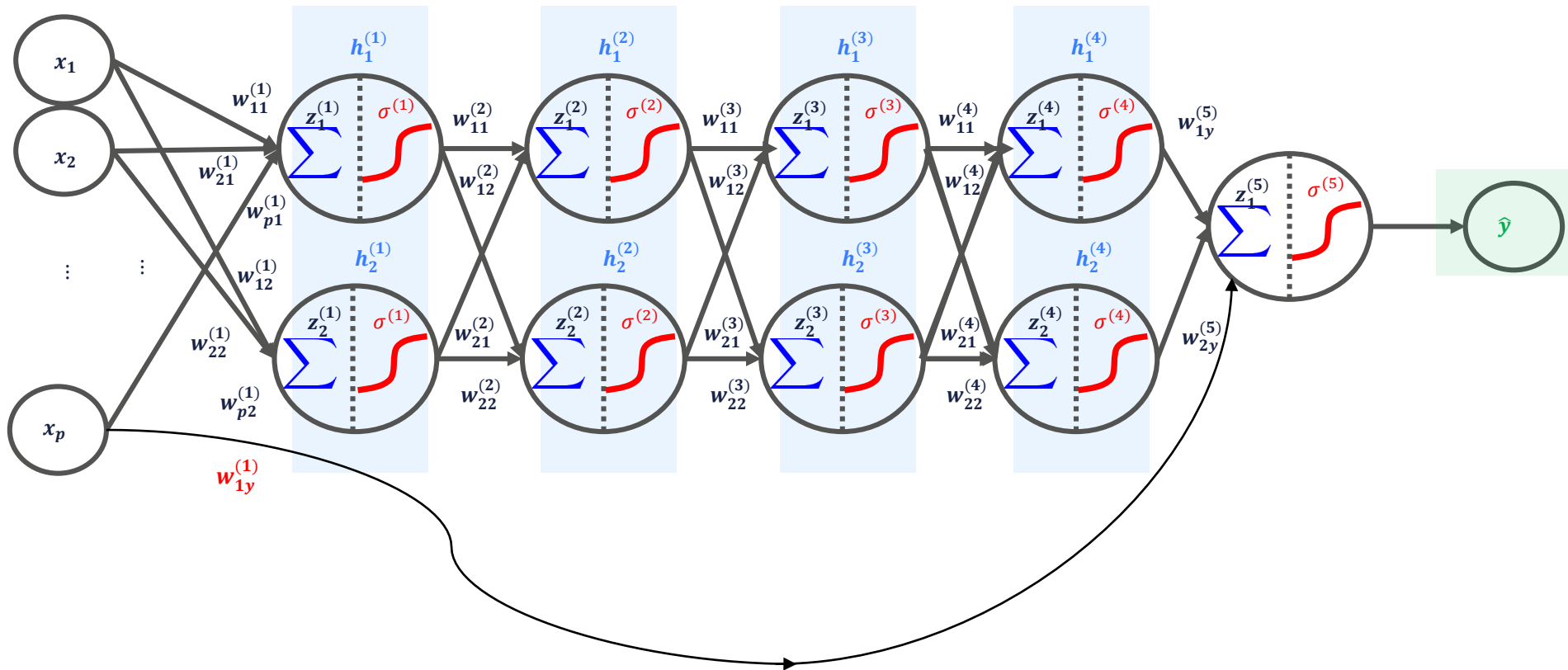
- Gradient의 규칙성
- 반복적으로 나타나는 부분은 프로그래밍 시 효율적으로 처리해야 함
- Theano/tensorflow/torch등의 라이브러리는 gradient를 간편하게 계산해주는 기능을 포함 26

Deep Learning Frameworks

Keras vs Tensorflow vs Pytorch

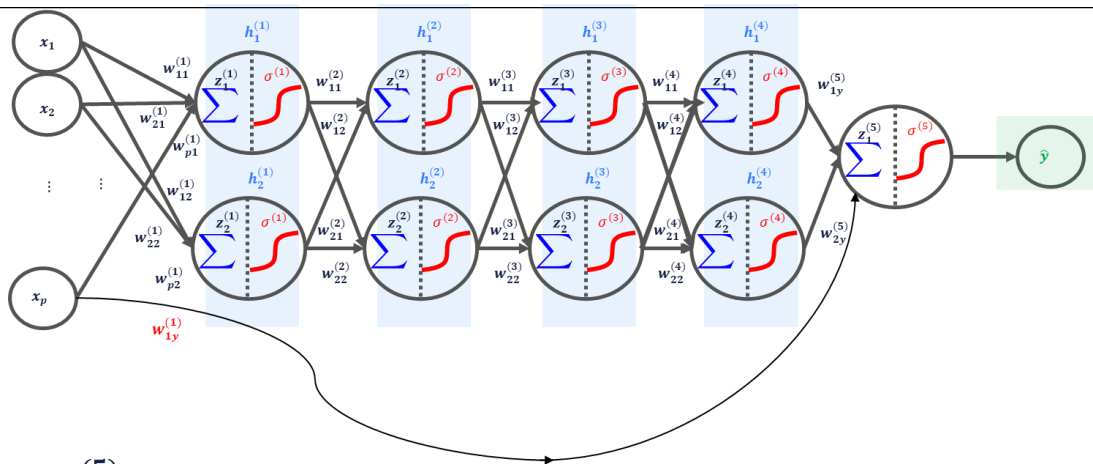
	Keras	PyTorch	TensorFlow
API Level	High	Low	High and Low
Architecture	Simple, concise, readable	Complex, less readable	Not easy to use
Datasets	Smaller datasets	Large datasets, high performance	Large datasets, high performance
Debugging	Simple network, so debugging is not often needed	Good debugging capabilities	Difficult to conduct debugging
Does It Have Trained Models?	Yes	Yes	Yes
Popularity	Most popular	Third most popular	Second most popular
Speed	Slow, low performance	Fast, high-performance	Fast, high-performance
Written In	Python	Lua	C++, CUDA, Python

Multi-layer Perceptron Learning



$$\hat{y}_i = f(x_i) = \sigma^{(4+1)}(\text{net}^{(4+1)} \dots (\sigma^{(1)}(\text{net}^{(1)}(x_i))))$$

Multi-layer Perceptron Learning



출력층 연결 - $w_{1y}^{(5)}$

$$\frac{dl_i}{dw_{1y}^{(5)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dw_{1y}^{(5)}}$$

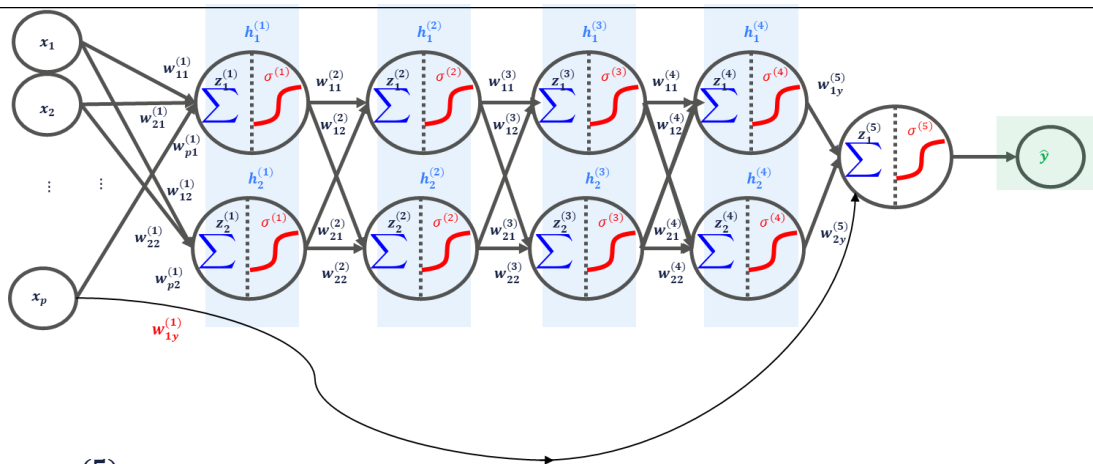
중간 은닉층 연결 - $w_{11}^{(4)}$

$$\frac{dl_i}{dw_{11}^{(4)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dh_{i1}^{(4)}} \frac{dh_{i1}^{(4)}}{dz_{i1}^{(4)}} \frac{dz_{i1}^{(4)}}{dw_{11}^{(4)}}$$

중간 은닉층 연결 - $w_{11}^{(3)}$

$$\frac{dl_i}{dw_{11}^{(3)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dh_{i1}^{(4)}} \frac{dh_{i1}^{(4)}}{dz_{i1}^{(4)}} \frac{dz_{i1}^{(4)}}{dw_{11}^{(3)}} \frac{dw_{11}^{(3)}}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dh_{i1}^{(3)}} \frac{dh_{i1}^{(3)}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{11}^{(3)}}$$

Multi-layer Perceptron Learning



출력층 연결 - $w_{1y}^{(5)}$

$$\frac{dl_i}{dw_{1y}^{(5)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dw_{1y}^{(5)}}$$

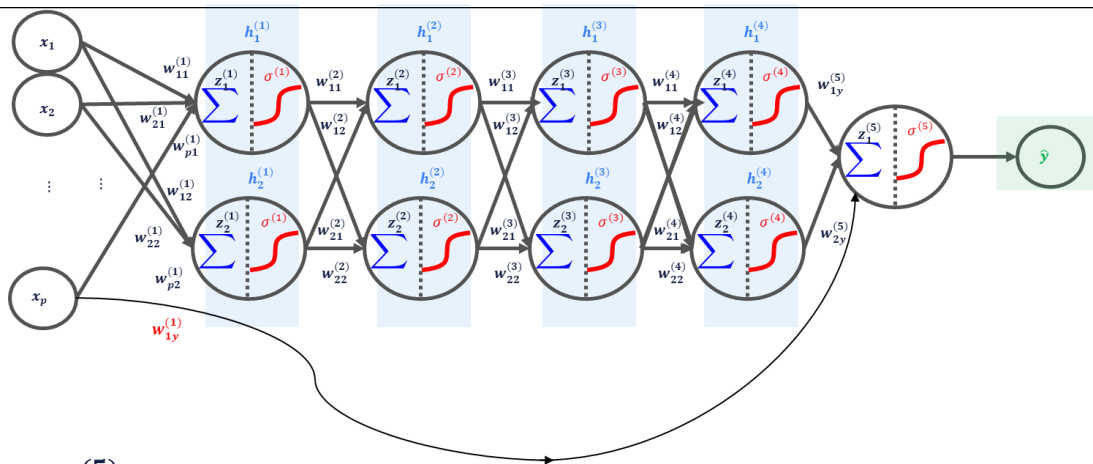
중간 은닉층 연결- $w_{11}^{(4)}$

$$\frac{dl_i}{dw_{11}^{(4)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dh_{i1}^{(4)}} \frac{dh_{i1}^{(4)}}{dz_{i1}^{(4)}} \frac{dz_{i1}^{(4)}}{dw_{11}^{(4)}}$$

중간 은닉층 연결- $w_{11}^{(3)}$

$$\frac{dl_i}{dw_{11}^{(3)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dh_{i1}^{(4)}} \frac{dh_{i1}^{(4)}}{dz_{i1}^{(4)}} \frac{dz_{i1}^{(4)}}{dw_{11}^{(3)}} \frac{dw_{11}^{(3)}}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dh_{i1}^{(3)}} \frac{dh_{i1}^{(3)}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{11}^{(3)}}$$

Multi-layer Perceptron Learning



출력층 연결 - $w_{1y}^{(5)}$

$$\frac{dl_i}{dw_{1y}^{(5)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dw_{1y}^{(5)}}$$

중간 은닉층 연결- $w_{11}^{(4)}$

$$\frac{dl_i}{dw_{11}^{(4)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dh_{i1}^{(4)}} \frac{dh_{i1}^{(4)}}{dz_{i1}^{(4)}} \frac{dz_{i1}^{(4)}}{dw_{11}^{(4)}}$$

중간 은닉층 연결- $w_{11}^{(3)}$

$$\frac{dl_i}{dw_{11}^{(3)}} = \sum_{i=1}^n \frac{dl_i}{d\hat{y}_i} \frac{d\hat{y}_i}{dz_{i1}^{(5)}} \frac{dz_{i1}^{(5)}}{dh_{i1}^{(4)}} \frac{dh_{i1}^{(4)}}{dz_{i1}^{(4)}} \frac{dz_{i1}^{(4)}}{dw_{11}^{(3)}} \frac{dw_{11}^{(3)}}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dh_{i1}^{(3)}} \frac{dh_{i1}^{(3)}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{11}^{(3)}}$$

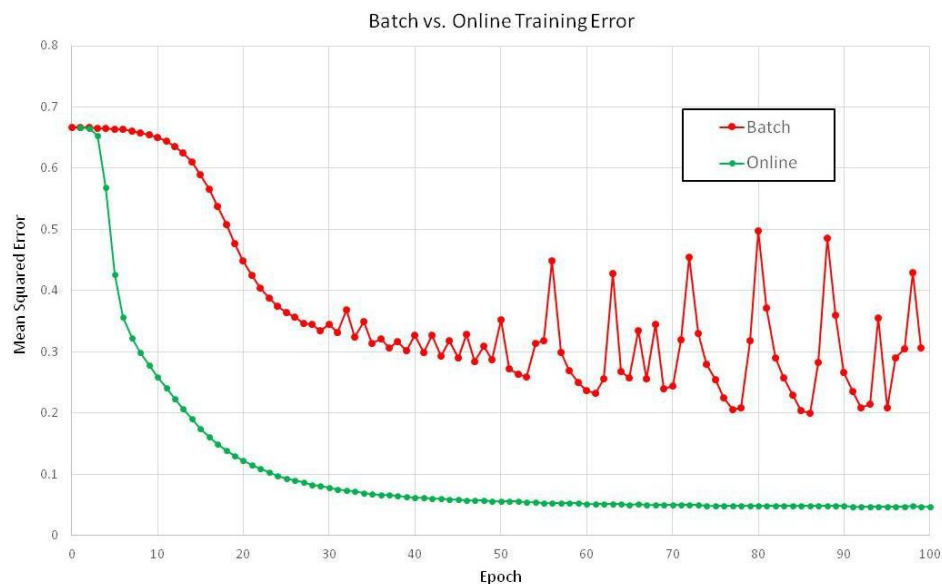
중간 은닉층 연결- $w_{11}^{(2)}$

$$\frac{dl_i}{dw_{11}^{(2)}} = \sum_{i=1}^n \frac{dl_i}{dz_{i1}^{(3)}} \frac{dz_{i1}^{(3)}}{dh_{i1}^{(2)}} \frac{dh_{i1}^{(2)}}{dz_{i1}^{(2)}} \frac{dz_{i1}^{(2)}}{dw_{11}^{(2)}} \frac{dw_{11}^{(2)}}{dz_{i1}^{(1)}} \frac{dz_{i1}^{(1)}}{dh_{i1}^{(1)}} \frac{dh_{i1}^{(1)}}{dz_{i1}^{(0)}} \frac{dz_{i1}^{(0)}}{dw_{11}^{(2)}}$$

Batch Size, Epoch

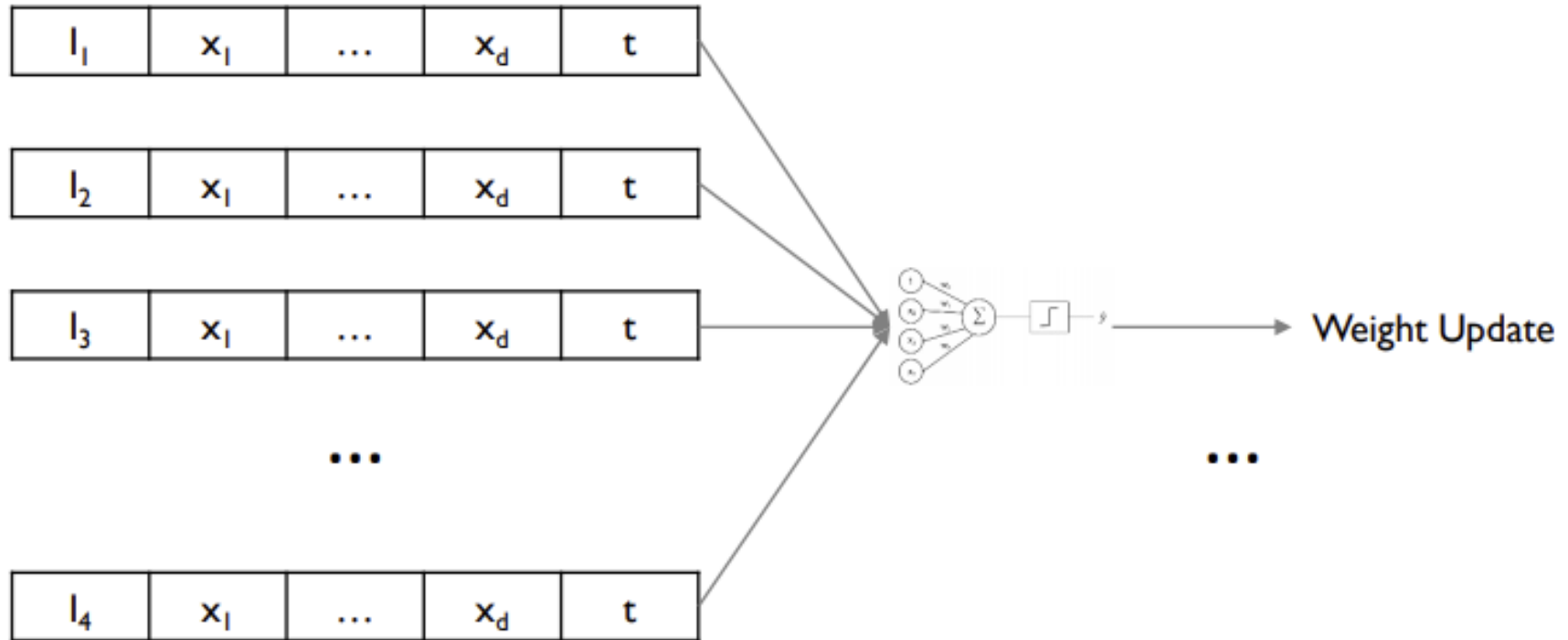
- ❖ Batch size: 1회 업데이트 시 사용할 관측치의 수
 - 클수록 빠르지만 성능 저하
 - 작을수록 느리지만 성능 향상
- ❖ Epochs: 전체 데이터에 대해 학습을 반복하는 횟수

x_1	x_2	x_3	y



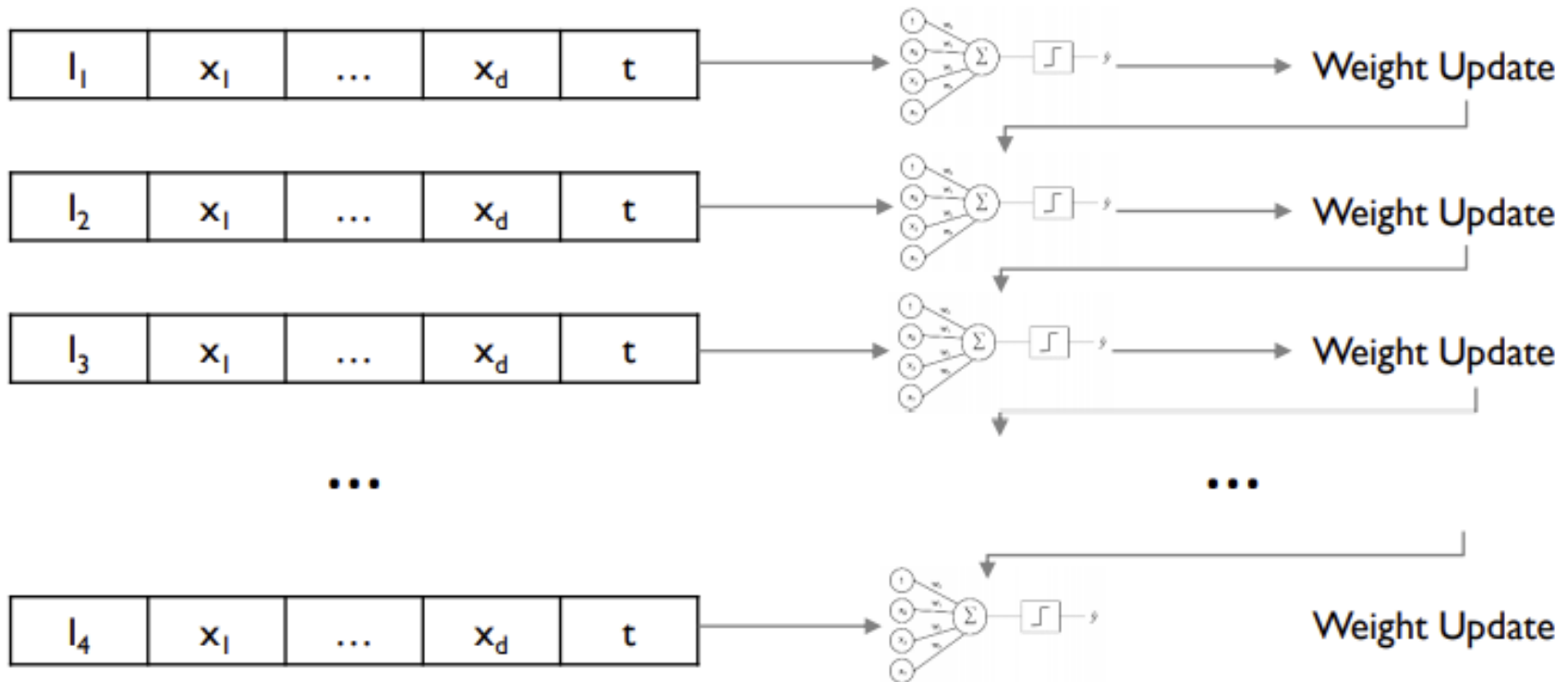
MLP Learning

- 배치 경사하강법 Batch Gradient Descent (BGD)
- 모든 훈련 데이터를 이용해서 필요한 계산을 다하고, 업데이트 시키는 방법



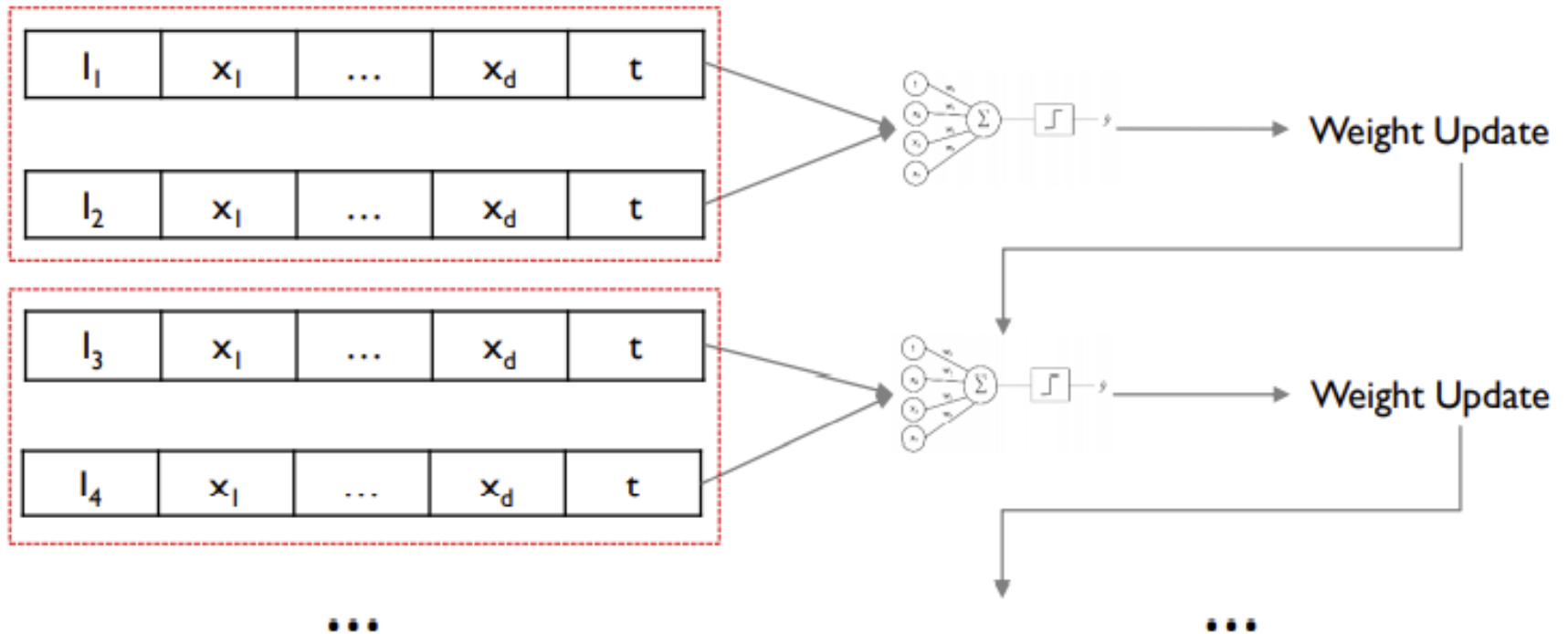
MLP Learning

- 확률적 경사하강법 Stochastic Gradient Descent (SGD)
- 개별 훈련 데이터를 이용해서 필요한 계산을 다하고, 업데이트 시키는 방법



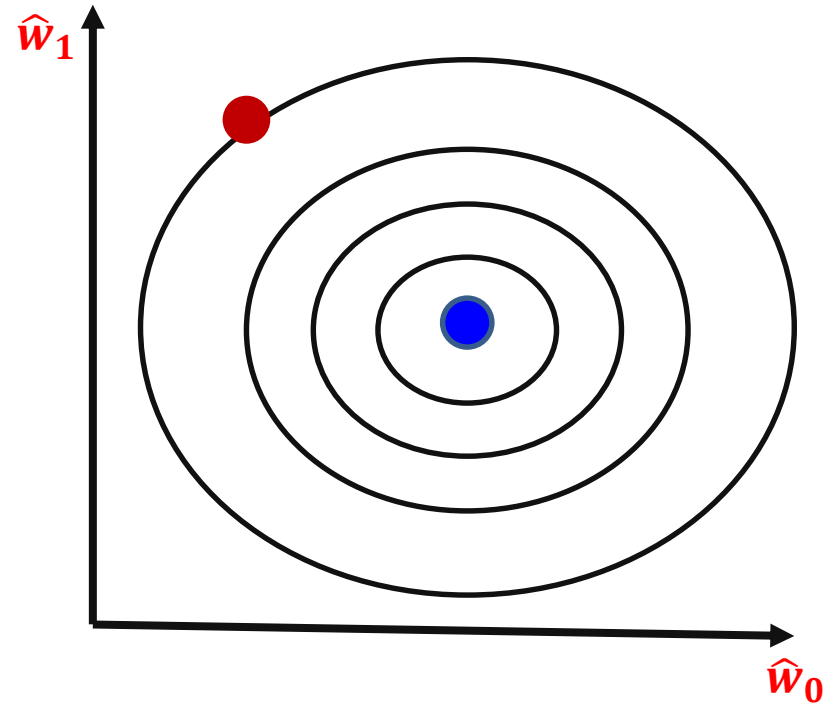
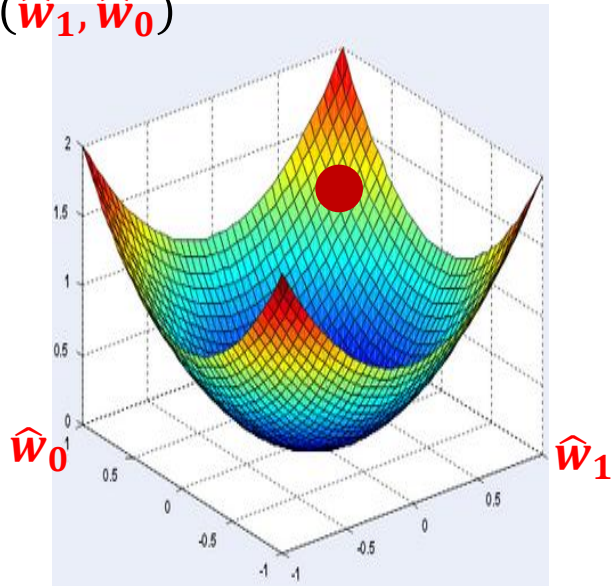
MLP Learning

- 미니배치 경사하강법 Mini-Batch Stochastic Gradient Descent (MBGD)
- 배치크기 훈련 데이터를 이용해서 필요한 계산을 다하고, 업데이트 시키는 방법



BGD vs SGD vs MBGD

$Loss(\hat{w}_1, \hat{w}_0)$



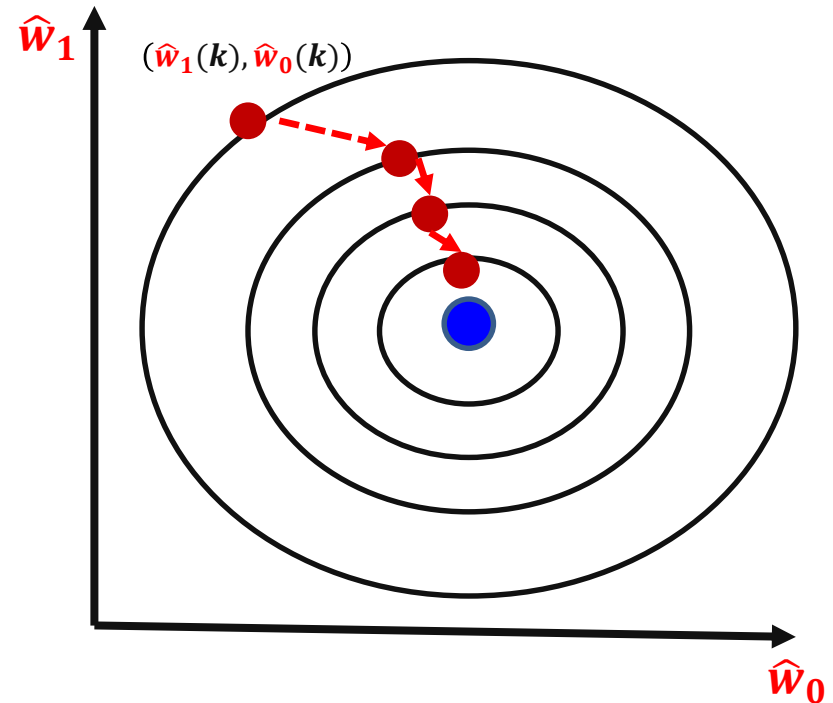
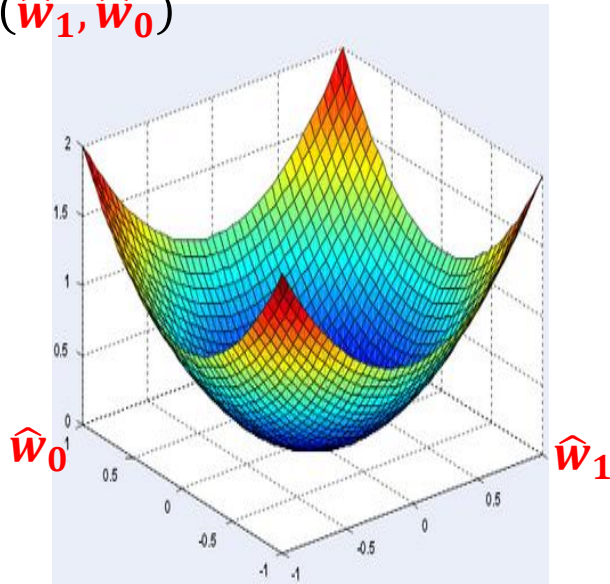
BGD vs SGD vs MBGD

$$\begin{aligned}\hat{\mathbf{w}}_1(k+1) &:= \hat{\mathbf{w}}_1(k) - \alpha \cdot \sum_{i=1}^n \frac{\partial \text{Loss}^{(x^i, y^i)}(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_1} \\ \hat{\mathbf{w}}_0(k+1) &:= \hat{\mathbf{w}}_0(k) - \alpha \cdot \sum_{i=1}^n \frac{\partial \text{Loss}^{(x^i, y^i)}(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_0}\end{aligned}$$

BGD

기울기=방향

$\text{Loss}(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)$



BGD vs SGD vs MBGD

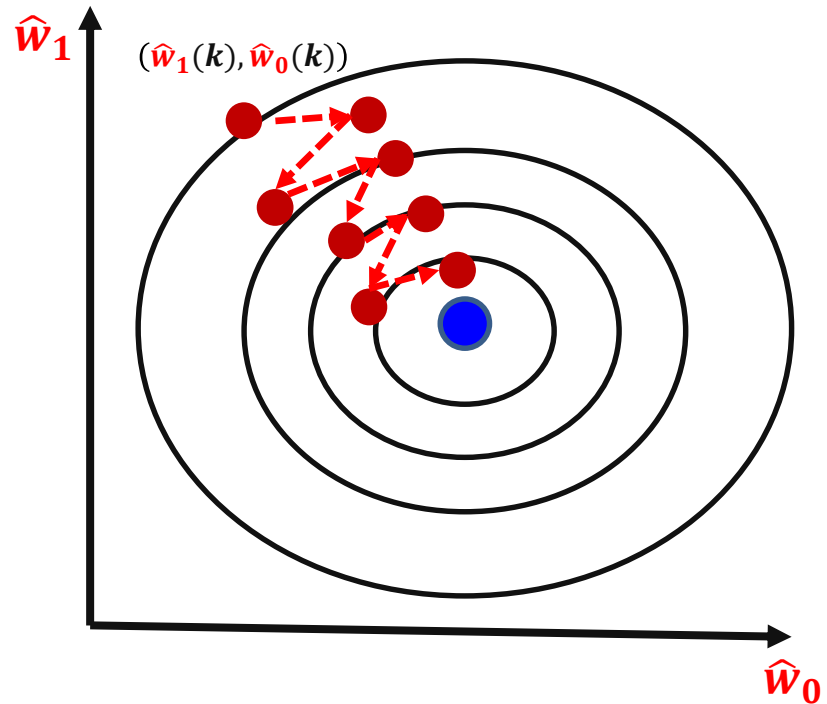
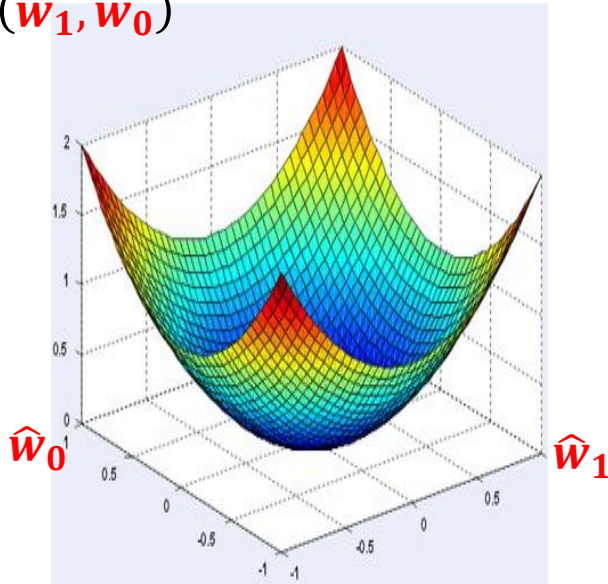
$$\hat{\mathbf{w}}_1(k+1) := \hat{\mathbf{w}}_1(k) - \alpha \cdot \frac{\partial \text{Loss}(x^i, y^i)(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_1}$$

$$\hat{\mathbf{w}}_0(k+1) := \hat{\mathbf{w}}_0(k) - \alpha \cdot \frac{\partial \text{Loss}(x^i, y^i)(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_0}$$

SGD

기울기=방향

$\text{Loss}(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)$



BGD vs SGD vs MBGD

$$\hat{\mathbf{w}}_1(k+1) := \hat{\mathbf{w}}_1(k) - \alpha \cdot \sum_{i=1}^{batch < n} \frac{\partial \text{Loss}(x^i, y^i)(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_1}$$

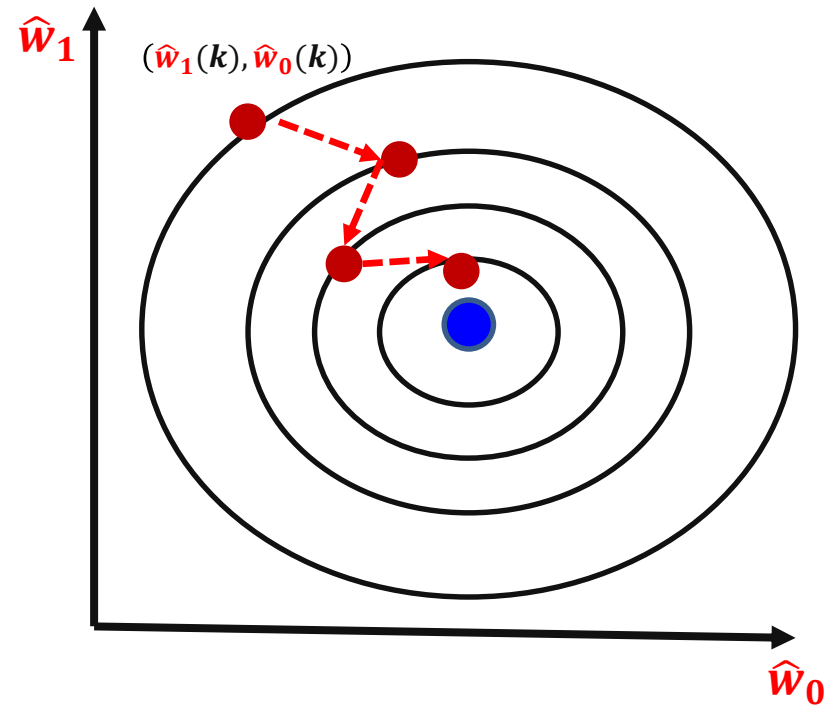
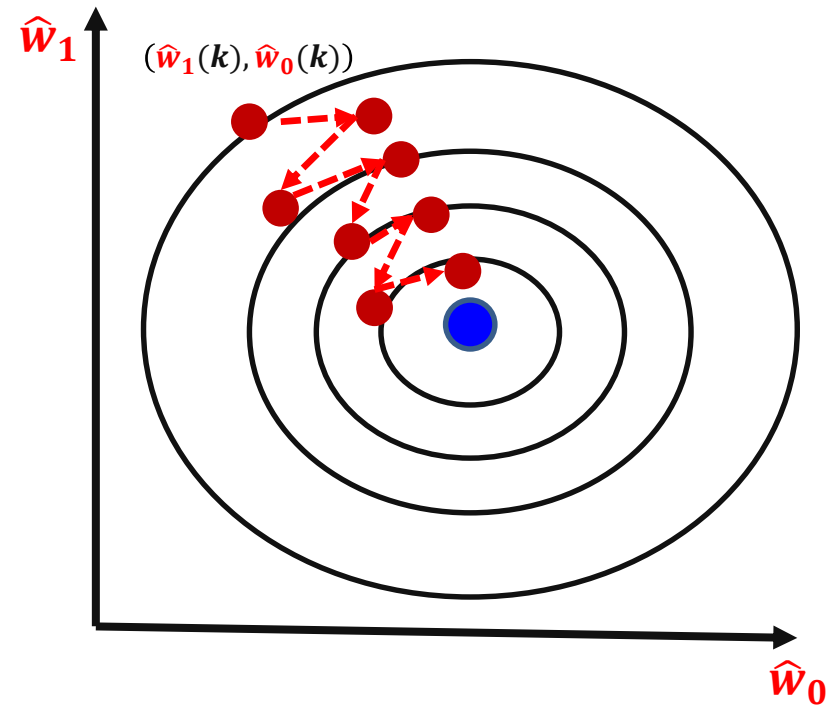
$$\hat{\mathbf{w}}_0(k+1) := \hat{\mathbf{w}}_0(k) - \alpha \cdot \sum_{i=1}^{batch < n} \frac{\partial \text{Loss}(x^i, y^i)(\hat{\mathbf{w}}_1, \hat{\mathbf{w}}_0)}{\partial \hat{\mathbf{w}}_0}$$

MBGD

기울기=방향

SGD

MBGD



Learning Rate

SGD – MBGD - BGD

$$\hat{\mathbf{w}}_1(k+1) := \hat{\mathbf{w}}_1(k) - \alpha \cdot \text{Objective}^{(1)}(\hat{\mathbf{w}}(k))$$

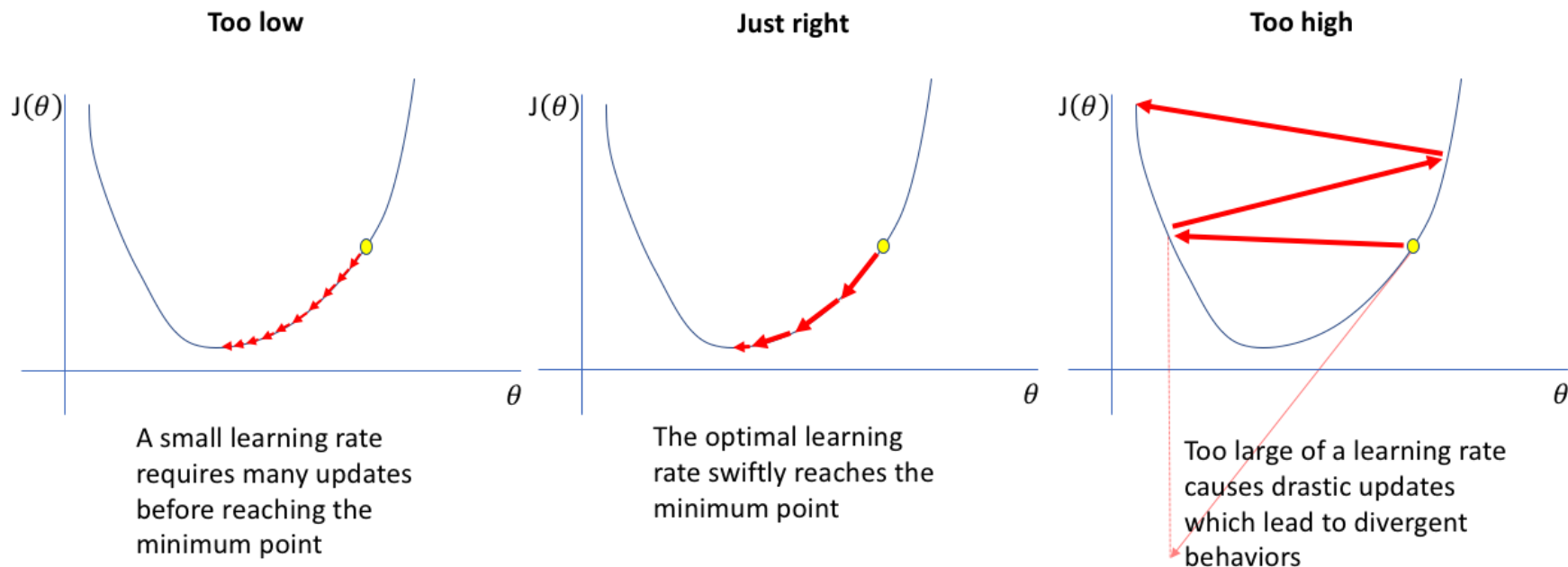
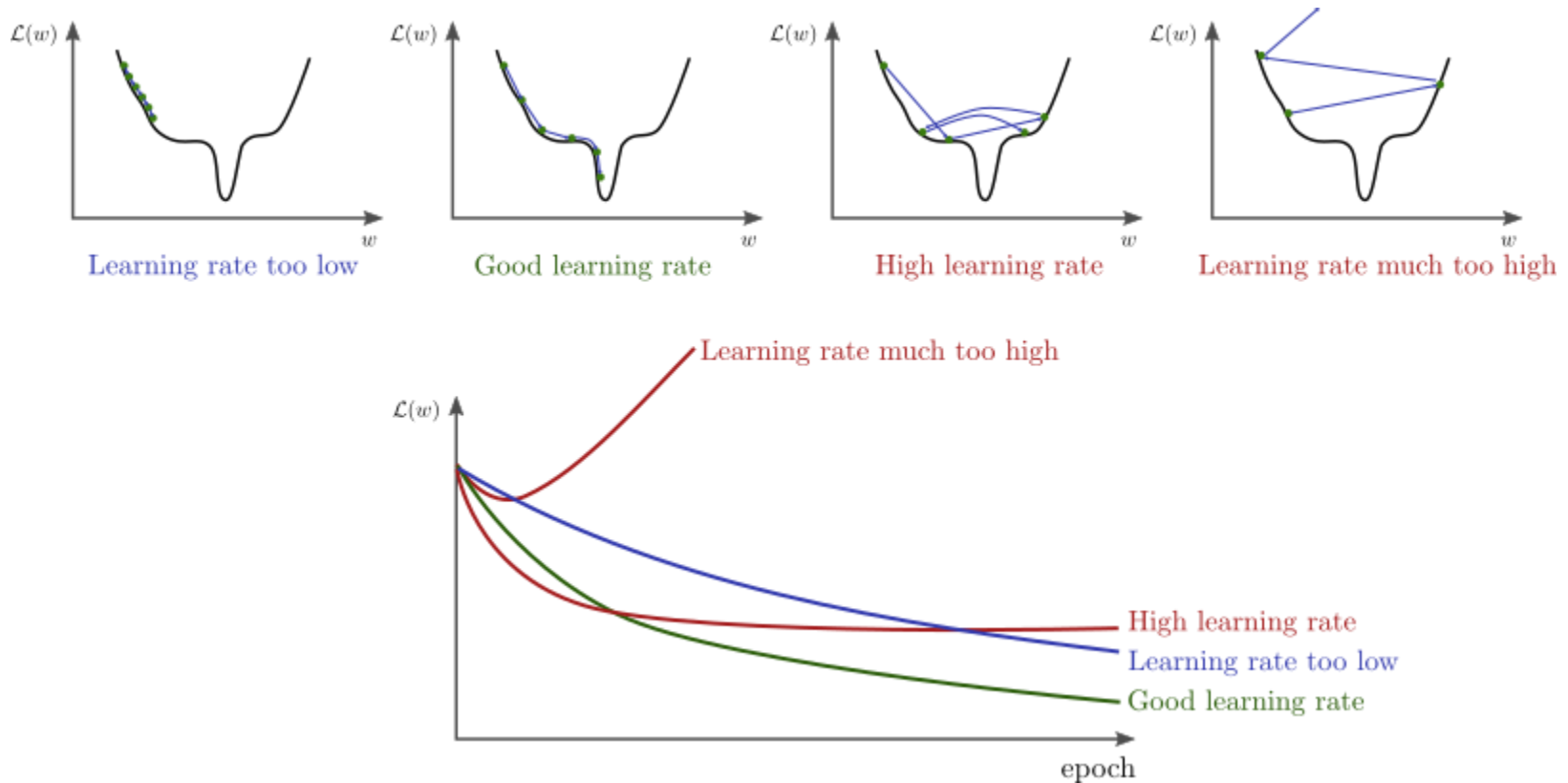


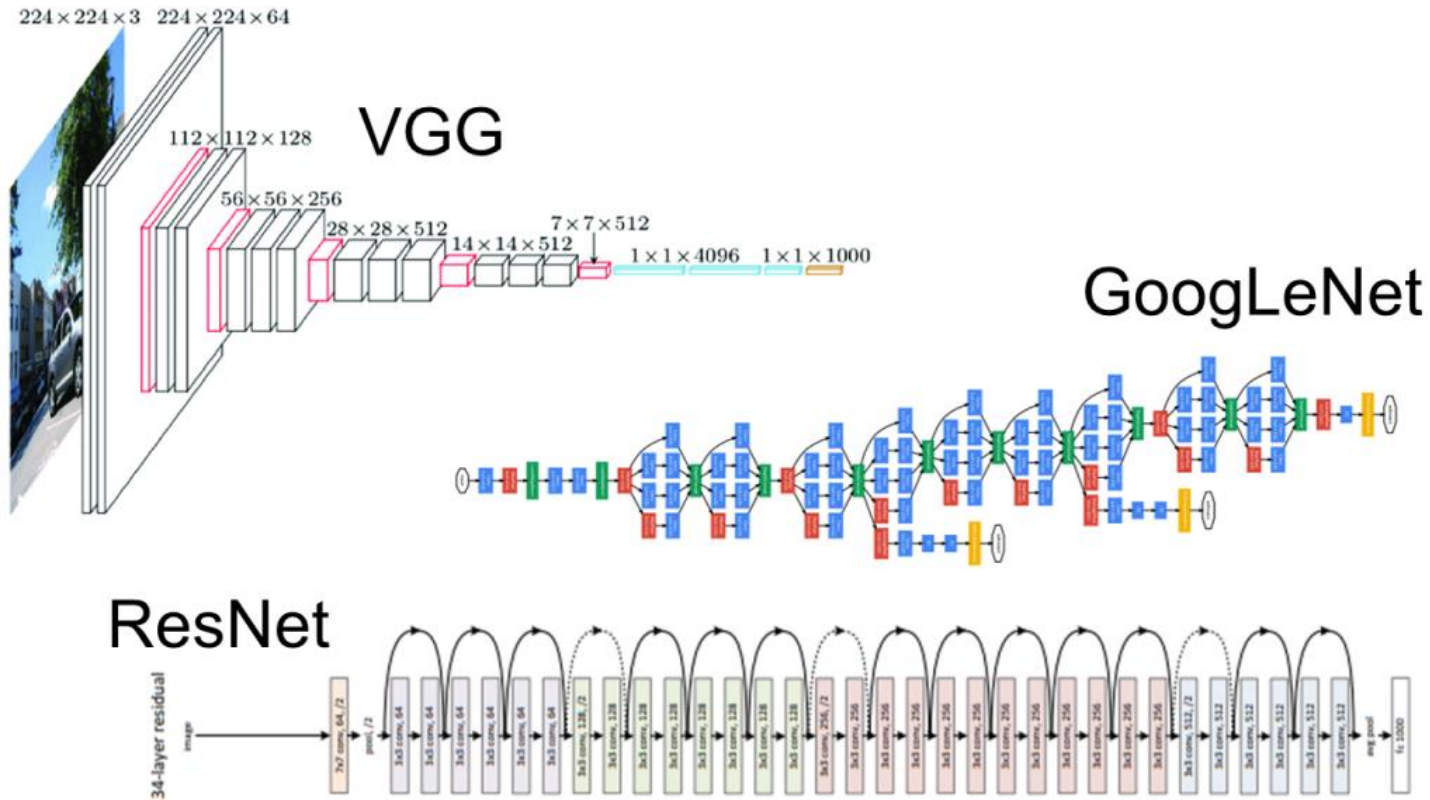
그림: <https://www.jeremyjordan.me/content/images/2018/02/Screen-Shot-2018-02-24-at-11.47.09-AM.png>

이슈- Learning Rate



이미지: <http://www.bdhammel.com/assets/learning-rate/lr-types.png>

Deep Learning Issues

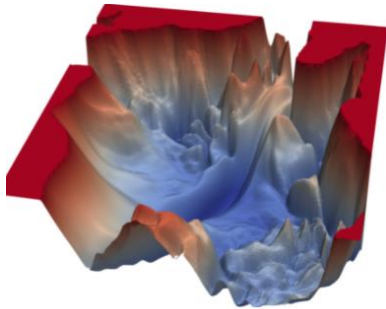


이미지: <https://glassboxmedicine.files.wordpress.com/2020/12/vgg-resnet-googlenet-1.png?w=768>

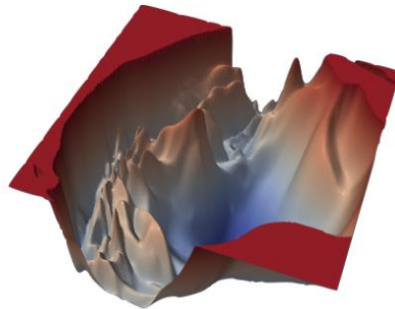
ResNet 50, 152: # of layers = 50, 152

Model Parameter Space

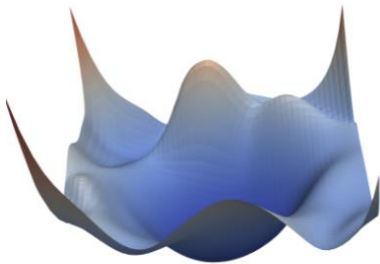
VGG-56



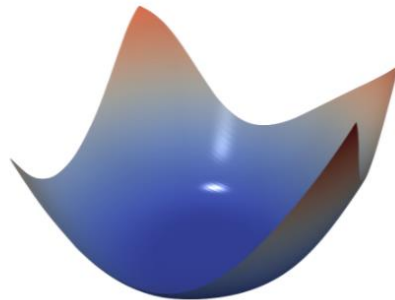
VGG-110



Resnet-56



Densenet-121



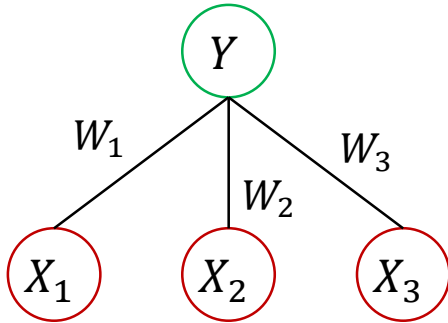
- There exist two crucial issues for training multi-layer neural networks until 2006:
 - ① Less available data and huge volume of parameters lead to model overfitting problem.
 - Large scale of labeled data
 - Hardware gpu
 - **Better optimizer**
 - ② Deep architecture leads to gradient diffusion problem.
 - **New architectures**
 - **New activation functions**

summary

- 가장 간단한 신경망 모델은 logistic regression / linear regression과 같다

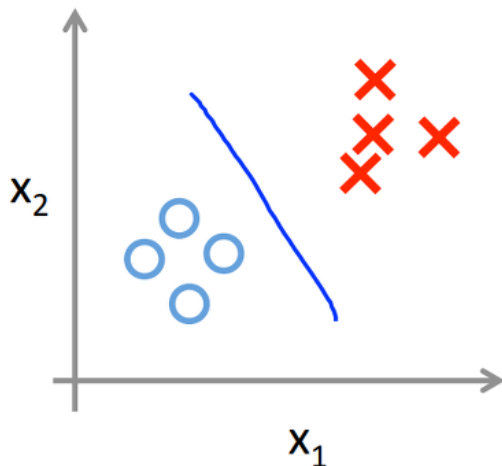
For classification,

$$\hat{Y} = \text{Activation}(W_1X_1 + W_2X_2 + W_3X_3 + b)$$



$$\hat{y} = \hat{f}(\mathbf{x}) = \frac{1}{1 + e^{-(W_1X_1 + W_2X_2 + W_3X_3 + b)}}$$

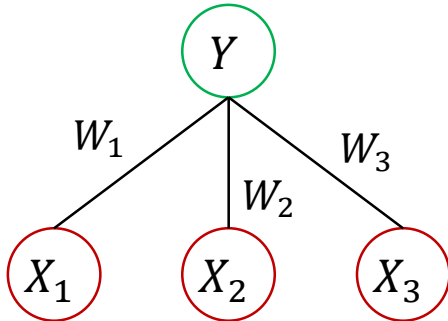
Binary classification:



- 개 vs 고양이
- 양품 vs 불량
- 암 vs 정상
-

summary

- 가장 간단한 신경망 모델은 logistic regression / linear regression과 같다



For classification,

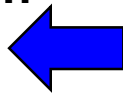
$$\hat{Y} = \text{Activation}(W_1X_1 + W_2X_2 + W_3X_3 + b)$$

For regression,

$$\hat{Y} = W_1X_1 + W_2X_2 + W_3X_3 + b$$

- Define Loss function

- Max, Min problem



Square loss , MSE

$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m (y_i - \hat{y}_i)^2$$

Binary Cross Entropy

$$\text{minimize}_{\hat{w}_0, \hat{w}_1} \sum_{j=1}^m -\{y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)\}$$

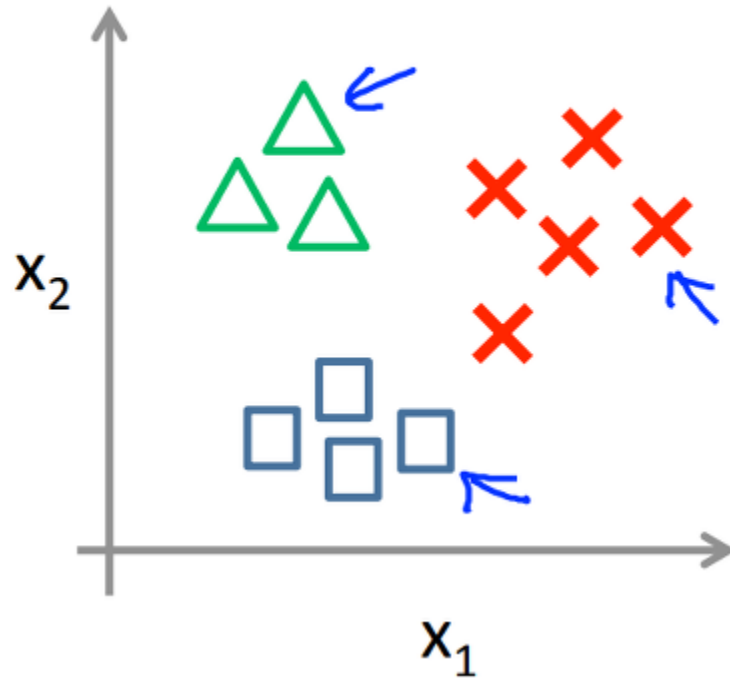
- Analytical solutions
- Closed Form Solution

- Numerical Solution->Gradient Descent

- ✓ Chain Rule
- ✓ 합성함수 (Function composition)
- ✓ 미분가능

Multi-class

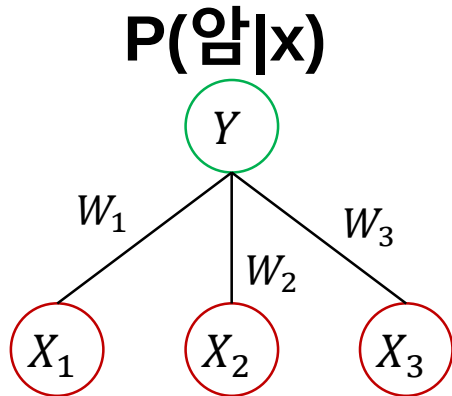
Multi-class classification:



- 개 vs 고양이 vs 새
- 양품 vs 불량1 vs 불량2
- 암1 vs 암2 vs 정상
-

Multi-class

- 가장 간단한 신경망 모델은 logistic regression / linear regression과 같다



For classification,

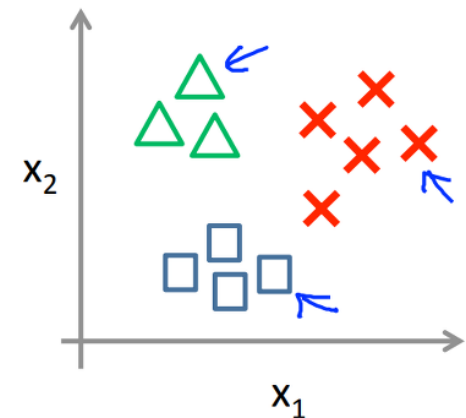
$$\hat{Y} = \text{Activation}(W_1X_1 + W_2X_2 + W_3X_3 + b)$$

$$\hat{y} = \hat{f}(\mathbf{x}) = \frac{1}{1 + e^{-(W_1X_1 + W_2X_2 + W_3X_3 + b)}}$$



- 개 vs 고양이 vs 새
- 양품 vs 불량1 vs 불량2
- 암1 vs 암2 vs 정상
-

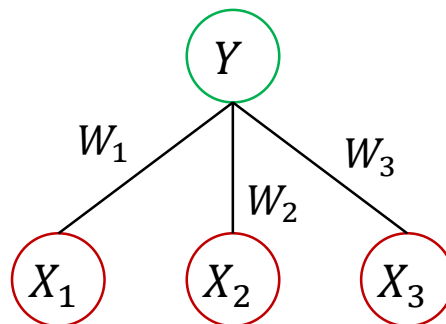
Multi-class classification:



Multi-class

$P(\text{암}|x)$

나이 x	y
28	정상
32	암1
46	암2
52	암3



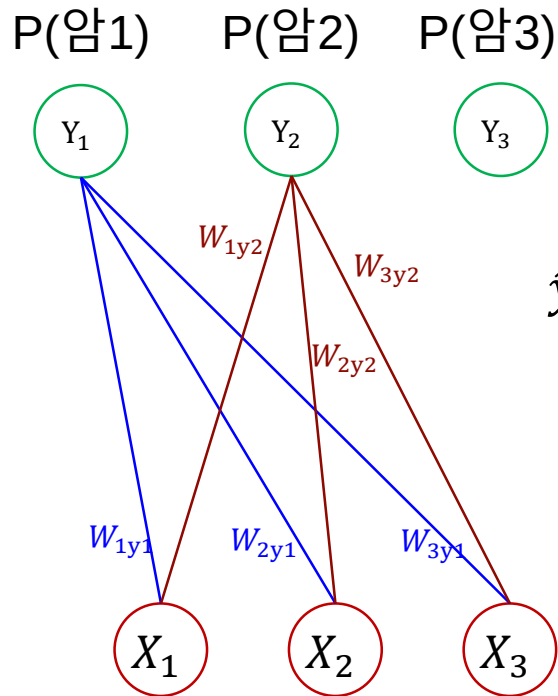
For classification,

$$\hat{Y} = \text{Activation}(W_1X_1 + W_2X_2 + W_3X_3 + b)$$

$$\hat{y} = \hat{f}(\mathbf{x}) = \frac{1}{1 + e^{-(W_1X_1 + W_2X_2 + W_3X_3 + b)}}$$

Multi-class

나이 x	y
28	정상
32	암1
46	암2
52	암3



$$\hat{y} = \hat{f}(\mathbf{x}) = \{\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \hat{f}_3(\mathbf{x})\}$$

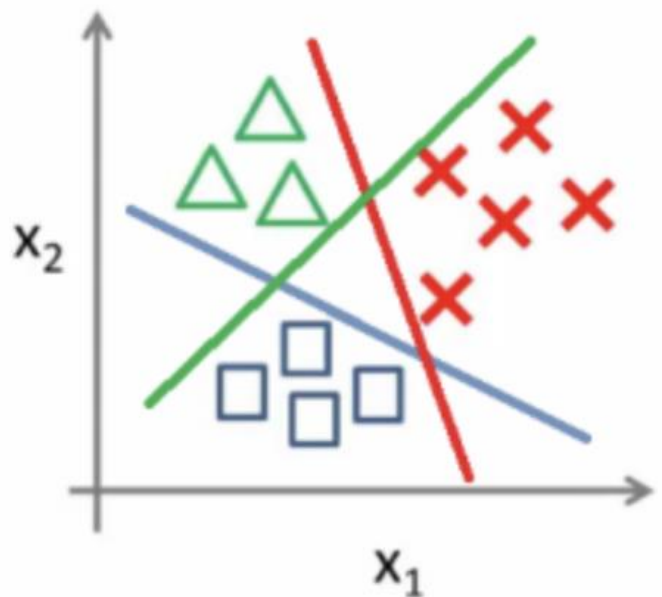
$$\hat{f}_1(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)}}$$

$$\hat{f}_2(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)}}$$

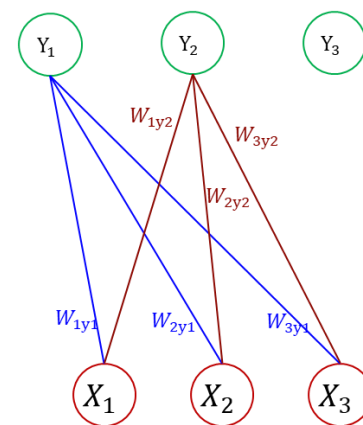
$$\hat{f}_3(\mathbf{x}) =$$

Multi-class

Multi-class classification:



P(암1) P(암2) P(암3)



$$\hat{y} = \hat{f}(\mathbf{x}) = \{\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \hat{f}_3(\mathbf{x})\}$$

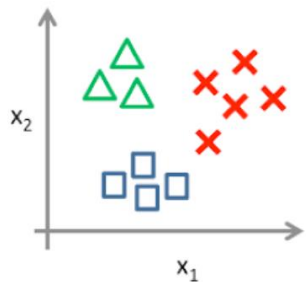
$$\hat{f}_1(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)}}$$

$$\hat{f}_2(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)}}$$

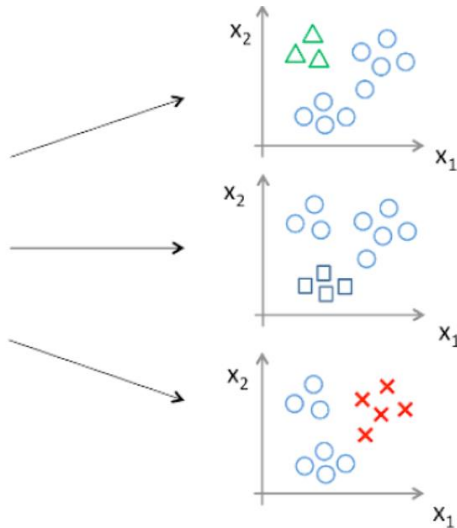
$$\hat{y} = \operatorname{argmax}_{c \in \{c_1, c_2, \dots\}} \hat{f}_c(\mathbf{x})$$

Multi-class

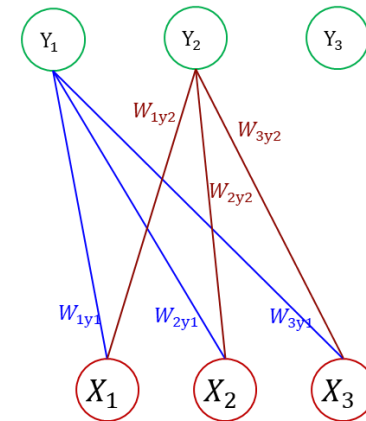
One-vs-all (one-vs-rest):



Class 1: Green
Class 2: Blue
Class 3: Red



P(암1) P(암2) P(암3)



$$\hat{y} = \hat{f}(\mathbf{x}) = \{\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \hat{f}_3(\mathbf{x})\}$$

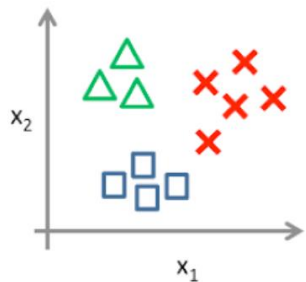
$$\hat{f}_1(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)}}$$

$$\hat{f}_2(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)}}$$

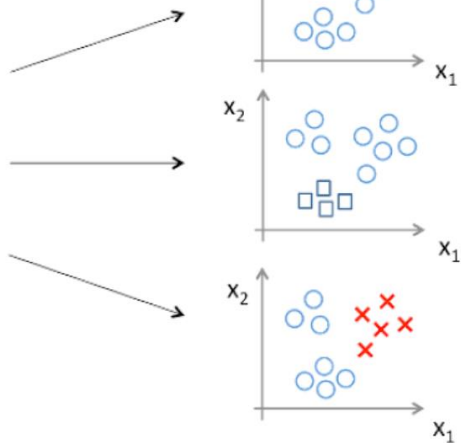
$$\hat{y} = \operatorname{argmax}_{c \in \{c_1, c_2, \dots\}} \hat{f}_c(\mathbf{x})$$

Multi-class

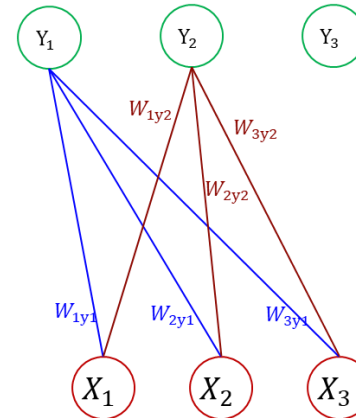
One-vs-all (one-vs-rest):



Class 1: Green
Class 2: Blue
Class 3: Red



P(암1) P(암2) P(암3)



$$\hat{y} = \hat{f}(\mathbf{x}) = \{\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \hat{f}_3(\mathbf{x})\}$$

$$\hat{f}_1(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)}}$$

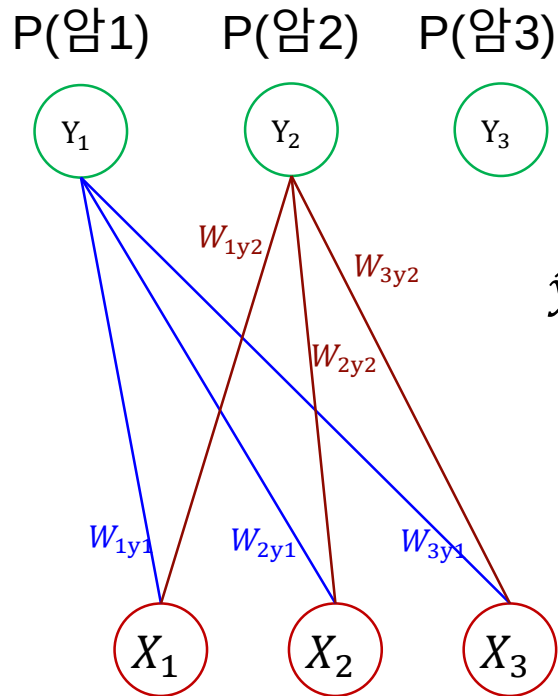
$$\hat{f}_2(\mathbf{x}) = \frac{1}{1 + e^{-(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)}}$$

Logistic
Function 한계

$$\hat{y} = \operatorname{argmax}_{c \in \{c_1, c_2, \dots\}} \hat{f}_c(\mathbf{x})$$

Multi-class

나이 x	y
28	정상
32	암1
46	암2
52	암3



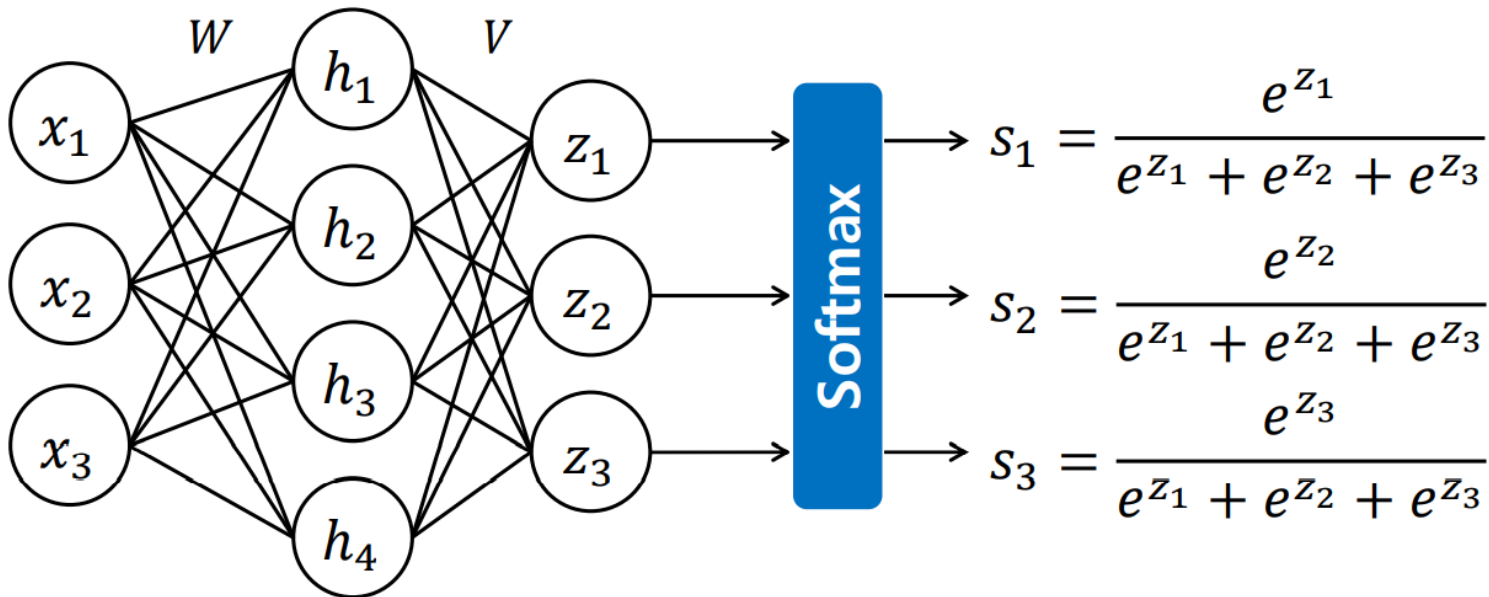
$$\hat{y} = \hat{f}(\mathbf{x}) = \{\hat{f}_1(\mathbf{x}), \hat{f}_2(\mathbf{x}), \hat{f}_3(\mathbf{x})\}$$

$$\hat{f}_1(\mathbf{x}) = \frac{e^{(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)}}{e^{(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)} + e^{(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)} + \dots}$$

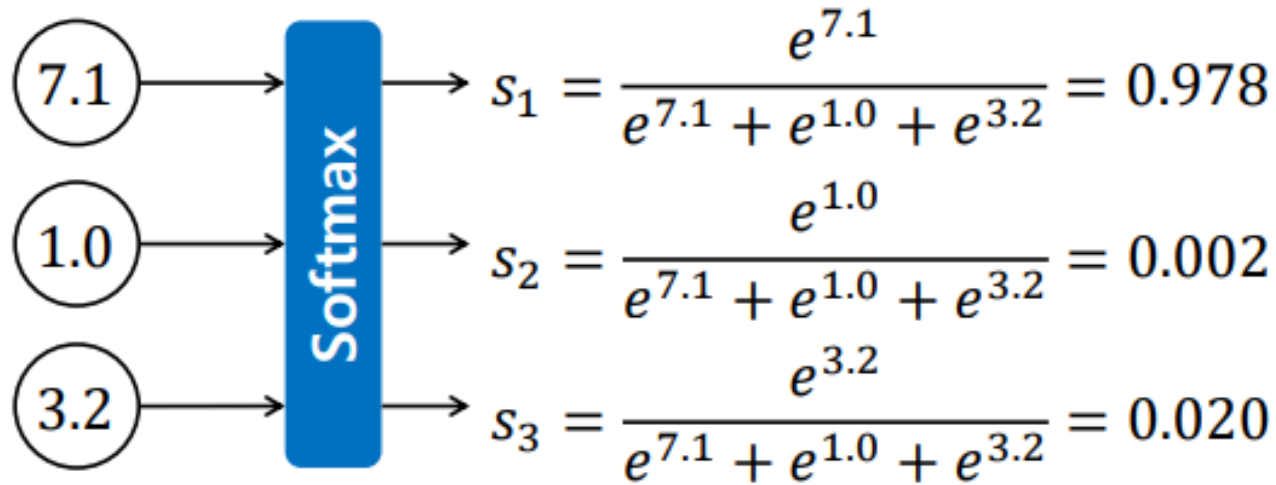
$$\hat{f}_2(\mathbf{x}) = \frac{e^{(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)}}{e^{(W_{1y1}X_1 + W_{2y1}X_2 + W_{3y1}X_3 + b_1)} + e^{(W_{1y2}X_1 + W_{2y2}X_2 + W_{3y2}X_3 + b_2)} + \dots}$$

다중 클래스 - Softmax

- ❖ 지금까지는 output layer에서 sigmoid function을 사용하여 이진 분류 수행
- ❖ Output layer의 크기를 범주 개수만큼 설정하고 softmax를 사용하면 다중 범주 분류 (multiclass classification) 모델로 손쉽게 변형 가능



다중 클래스 - Softmax



$$p(y = i|x) = s_i = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad i = 1, 2, \dots, C.$$

다중 클래스 – Categorical Cross Entropy

$$\text{binary cross entropy} = -y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)$$

$$= \sum_{c=1}^2 y_i^c \log \hat{y}_i^c$$

p와 **q** 분포간의 비유사성

$$\text{binary cross entropy} = -p(y = 1) \log q(y = 1) - p(y = 0) \log q(y = 0)$$

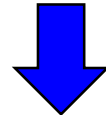
x_1	x_2	y
0	0	정
1	0	암
0	1	암
1	1	암

x_1	x_2	$y, P(y=\text{암})$
0	0	0
1	0	1
0	1	1
1	1	1

$\hat{y}$, $q(y=\text{암})$
0.001
0.9
0.8
0.6

다중 클래스 – Categorical Cross Entropy

$$\text{binary cross entropy} = -y_i \log \hat{y}_i - (1 - y_i) \log(1 - \hat{y}_i)$$


$$= \sum_{c=1}^2 y_i^c \log \hat{y}_i^c$$

$$\text{multiclass cross entropy} = \sum_{c=1}^k y_i^c \log \hat{y}_i^c$$

x_1	x_2	y
0	0	정
1	0	암1
0	1	암2
1	1	암3

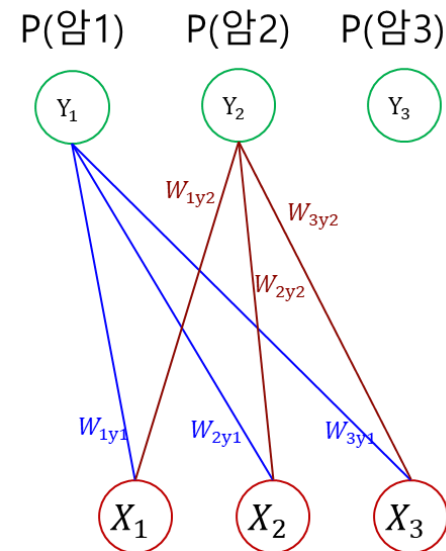
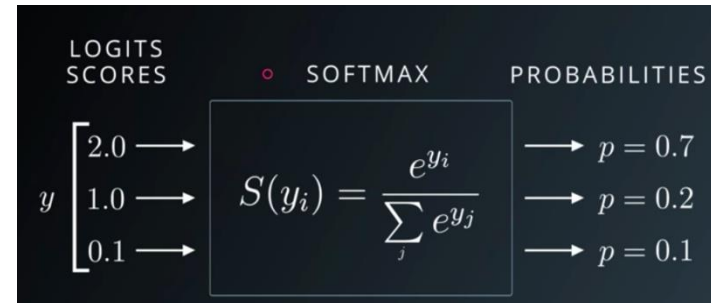
$y_A,$ $p(y=\text{정})$	$y_B,$ $p(y=\text{암1})$	$y_C,$ $p(y=\text{암2})$	$y_D,$ $p(y=\text{암3})$
1	0	0	0
0	1	0	0
0	0	1	0
0	0	0	1

$\hat{y}_A,$ $q(y=\text{정})$	$\hat{y}_B,$ $q(y=\text{암1})$	$\hat{y}_C,$ $q(y=\text{암2})$	$\hat{y}_D,$ $q(y=\text{암3})$
0.7	0.1	0.1	0.1
0.2	0.5	0.2	0.1
0.3	0.2	0.4	0.1
0.1	0.1	0.05	0.75

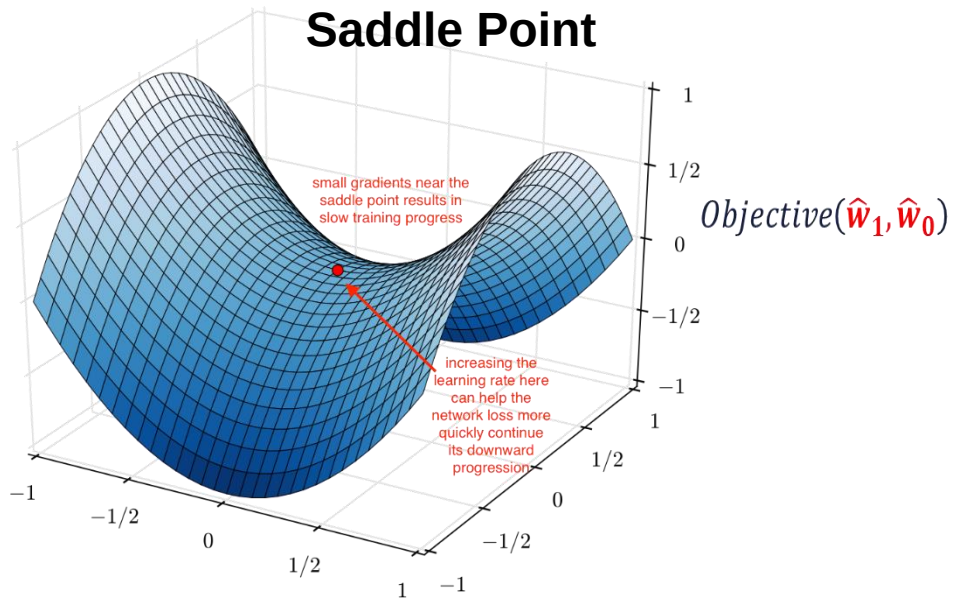
Gradient Descent (경사하강법)

$$\mathbf{w}(k+1) := \mathbf{w}(k) - \alpha \cdot \text{Loss}^{(1)}(\mathbf{w}(k))$$

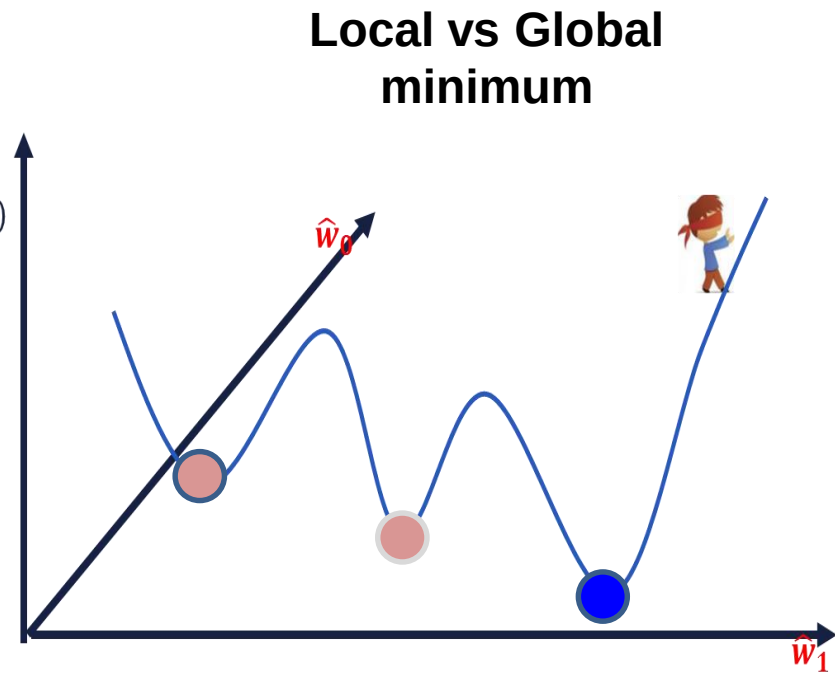
1. 가중치 (학습 파라미터) 임의로 생성
2. 데이터의 **Cross entropy Loss**에 대해 Gradient (미분)계산
 - Chain Rule <- 합성함수
3. 반복
 - 언제까지? 미분값 0
 - 적절한 횟수 만큼 = epoch



이슈-Saddle Point

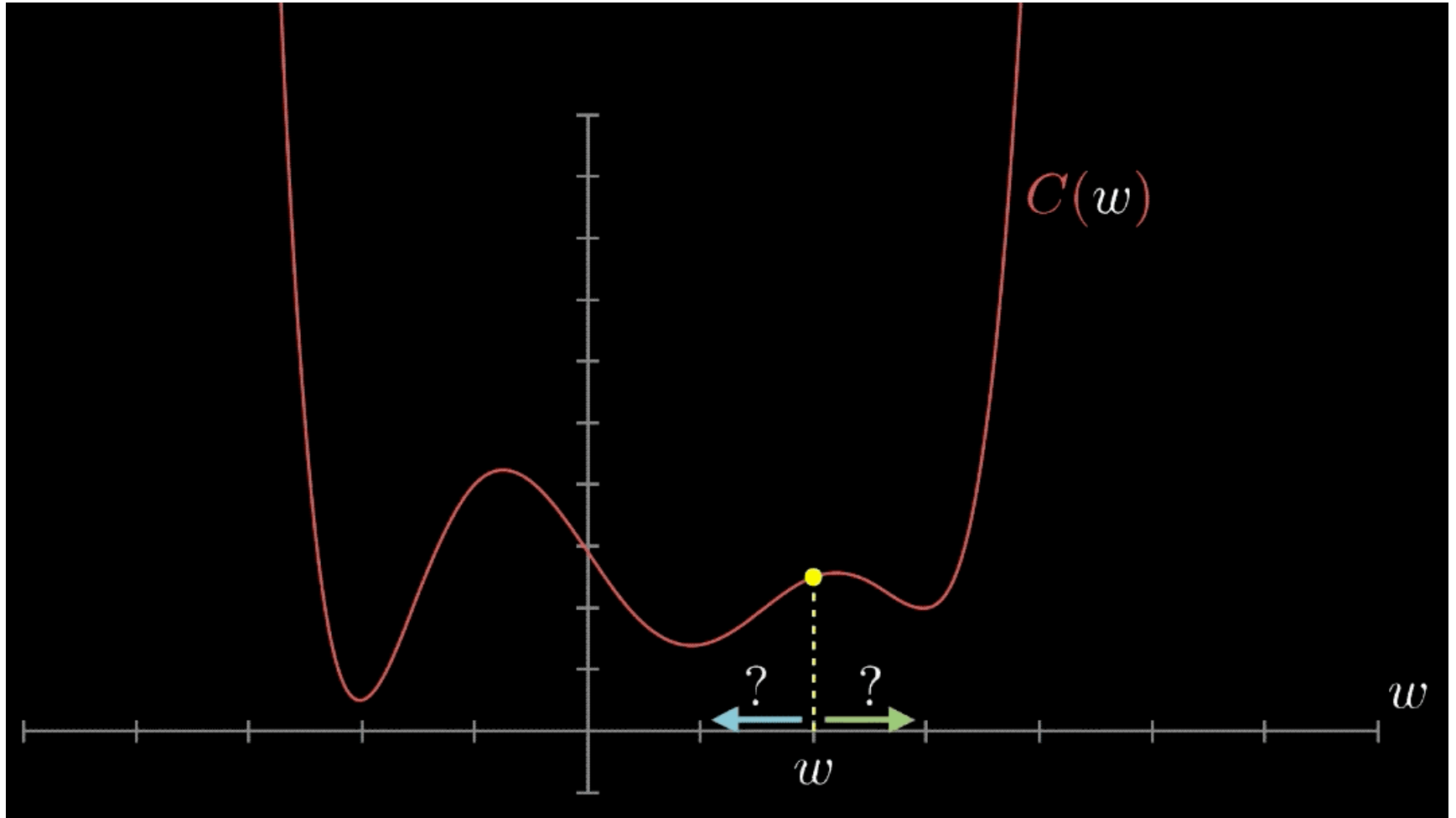


https://www.jeremyjordan.me/content/images/2018/02/Saddle_point.svg.png



기본 Optimizer (SGD) 이슈 실제 예제

$$\hat{w}(k+1) := \hat{w}(k) - \alpha \cdot \frac{\partial \text{Cost}(\hat{w})}{\partial \hat{w}} \left(\sum_{i=1}^M \frac{\partial \text{Loss}(x^i, y^i)(\hat{w})}{\partial \hat{w}} \right)$$



기본 Optimizer (SGD) 장단

Pros

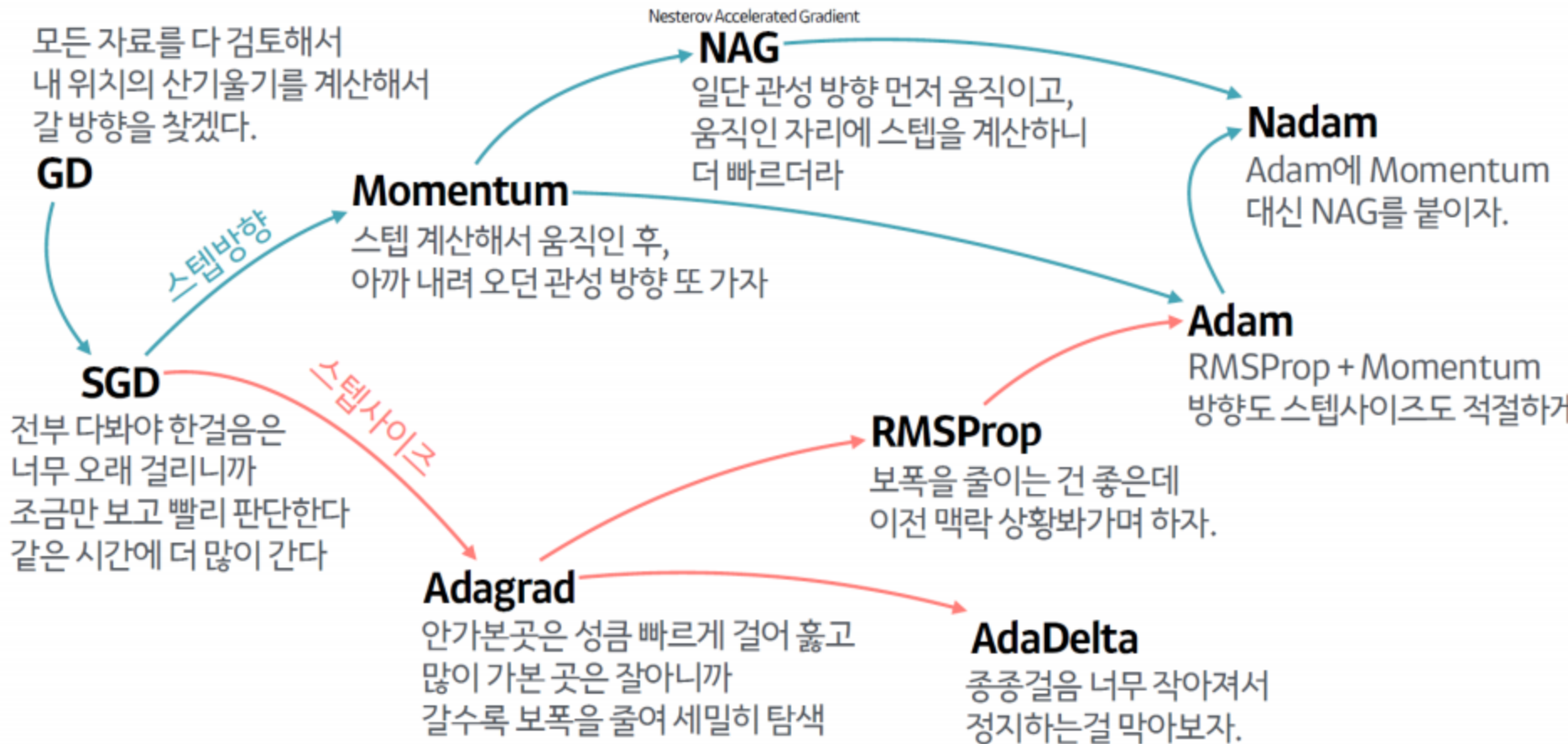
- Relatively fast compared to the older gradient descent approaches
- SGD is comparatively easy to learn for beginners, since it is not as math heavy as the newer approaches to optimizers

Cons

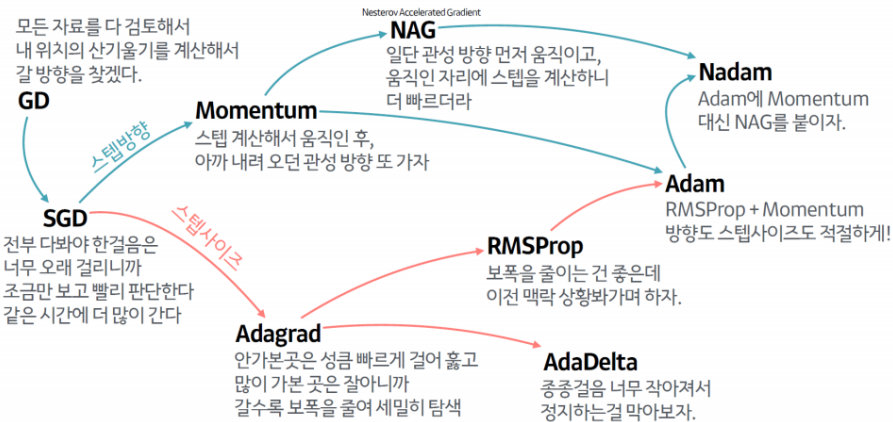
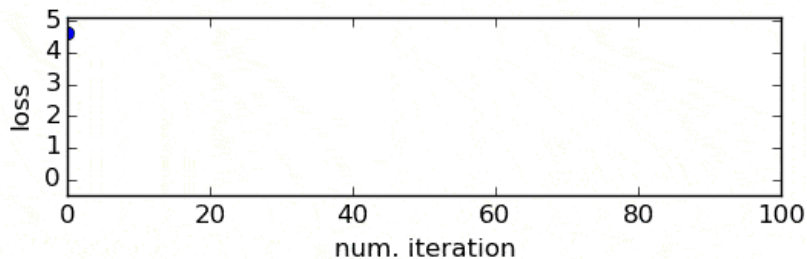
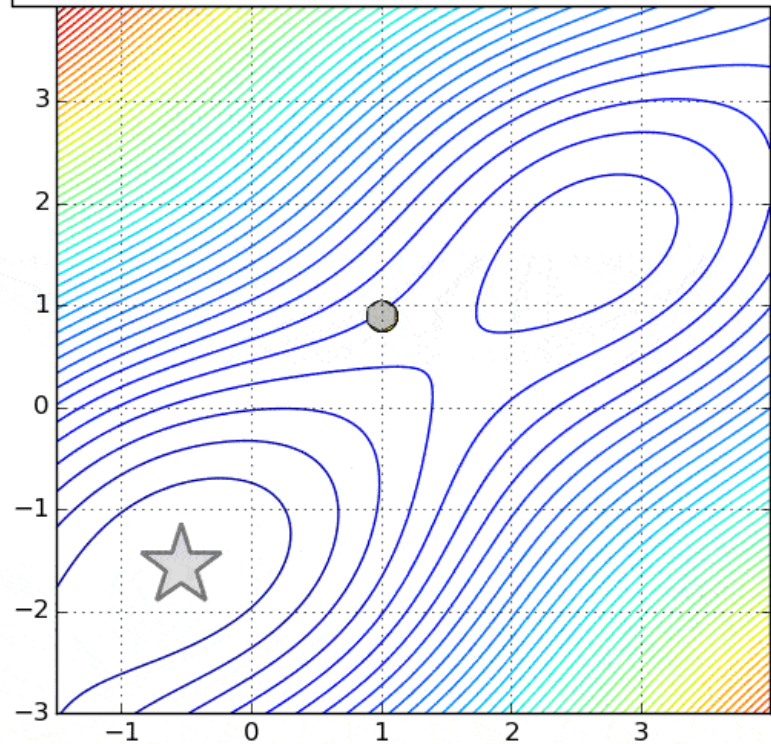
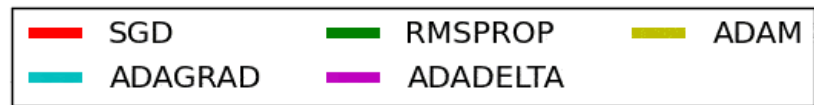
- Converges slower than newer algorithms
 - Has more problems with being stuck in a local minimum than newer approaches
 - Newer approaches outperform SGD in terms of optimizing the cost function
-

Optimizers

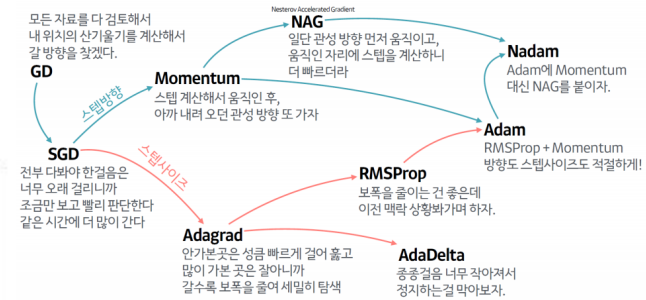
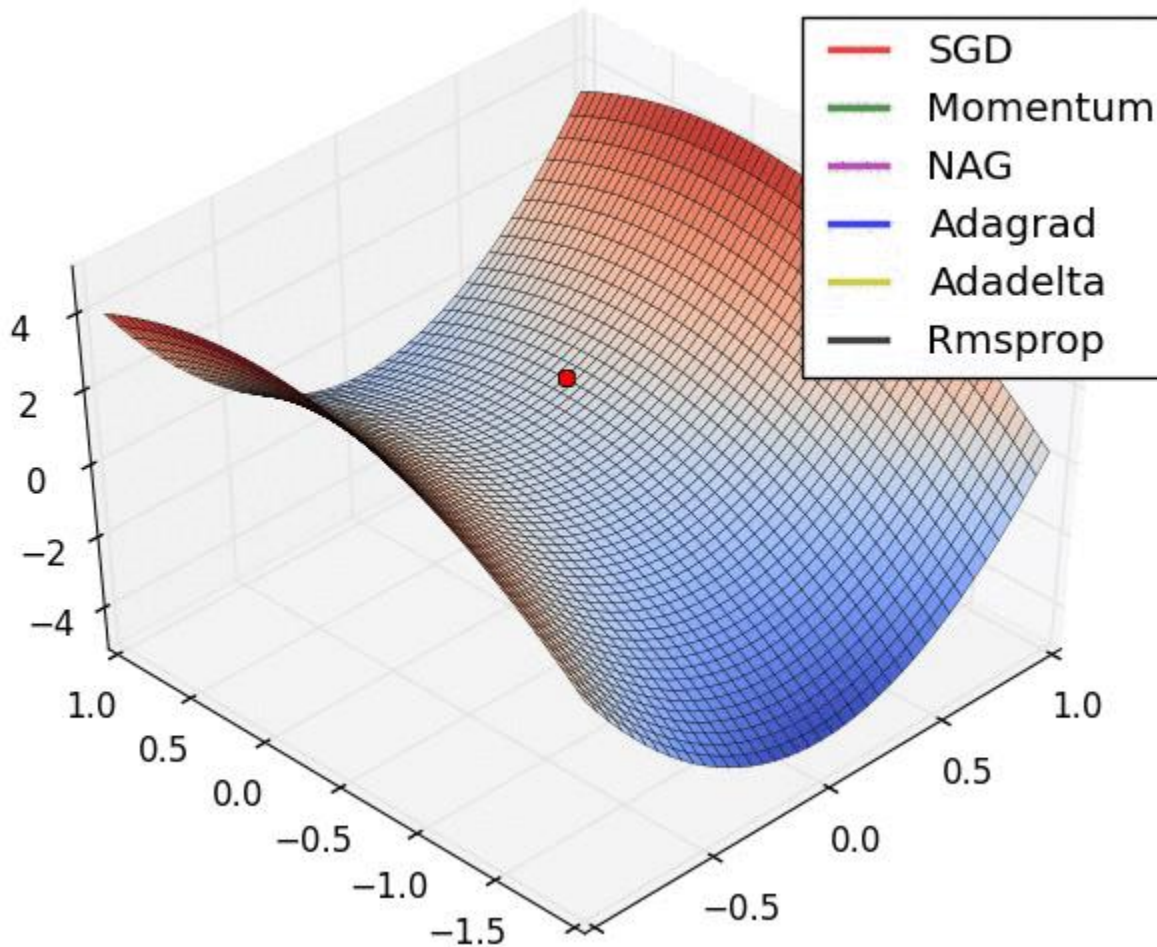
Pick the right optimizer with the right parameters



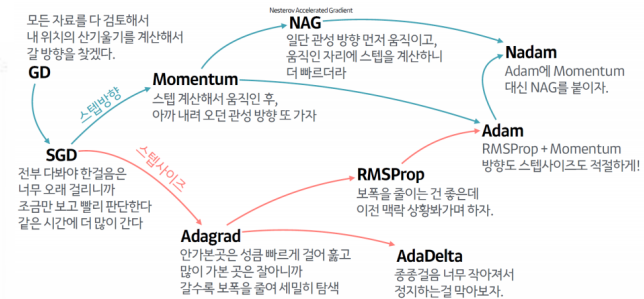
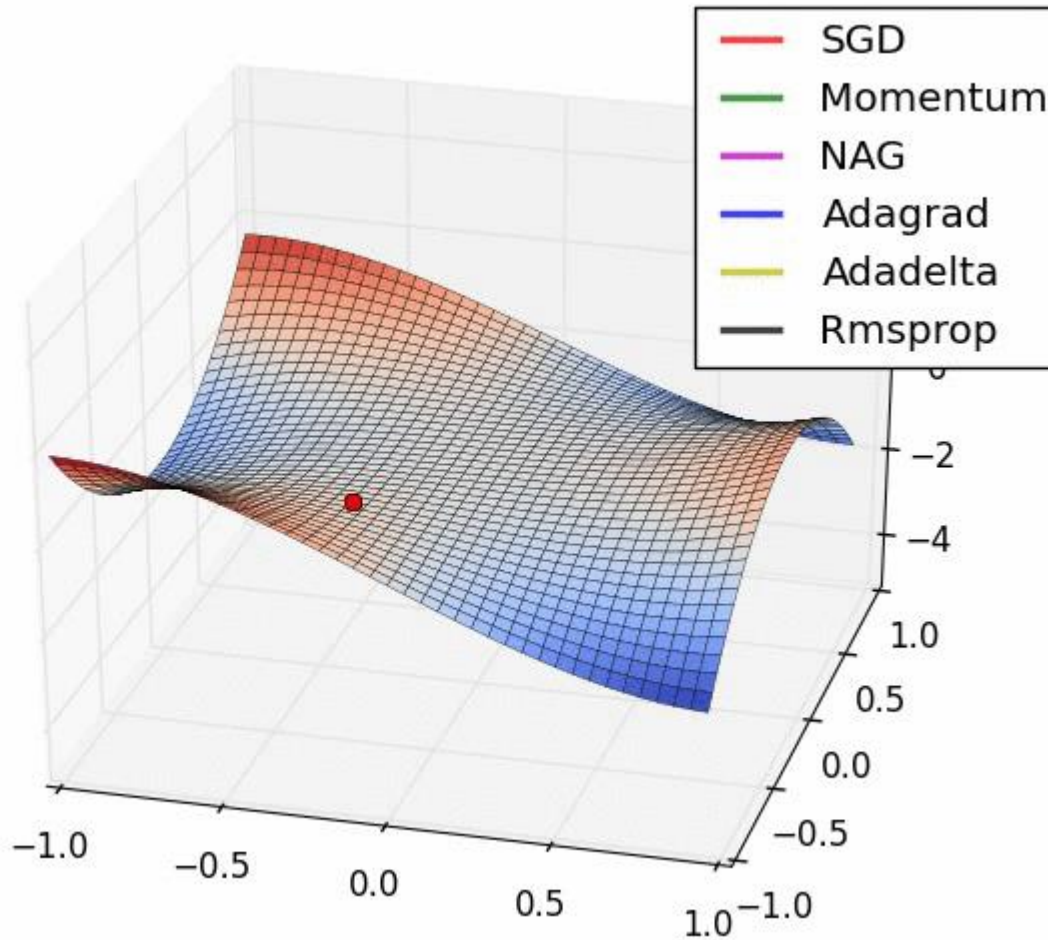
Optimizers



Optimizers

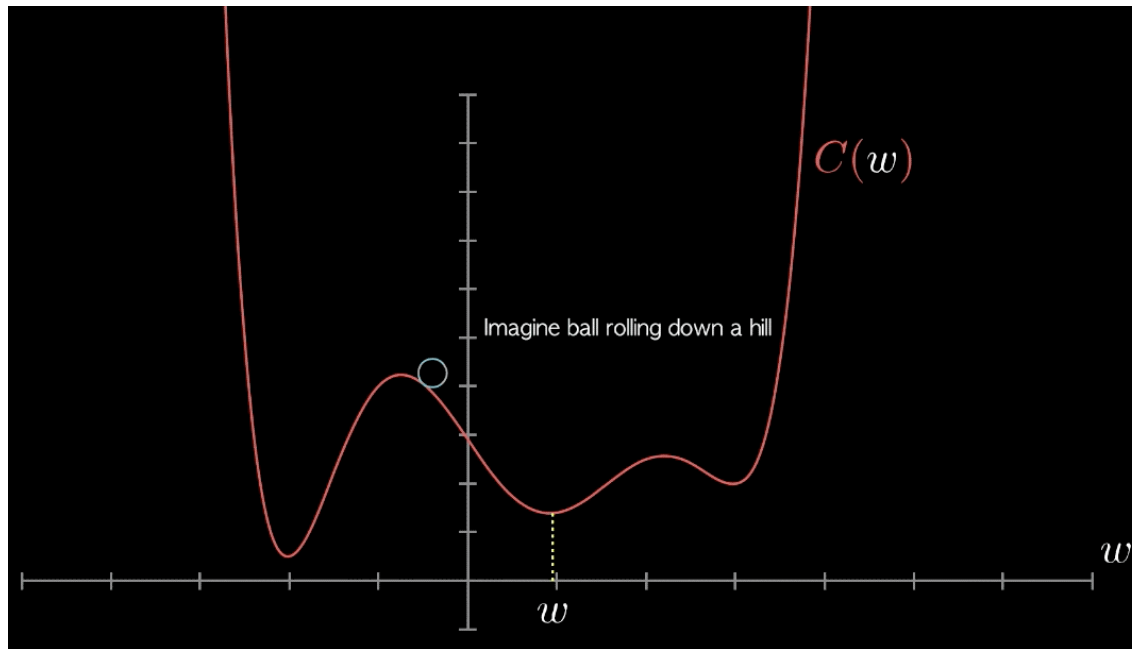


Optimizers

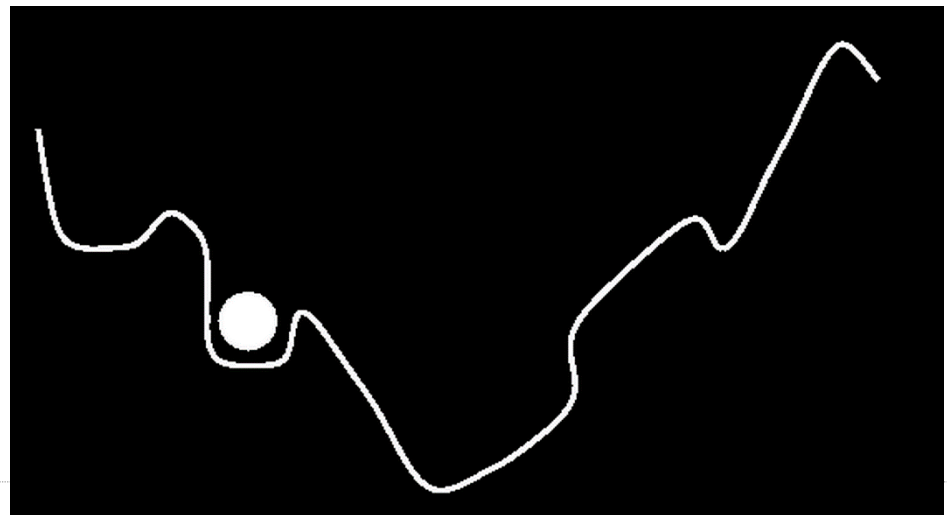


출처 : <http://imgur.com/a/Hqolp>

Optimizers – Momentum



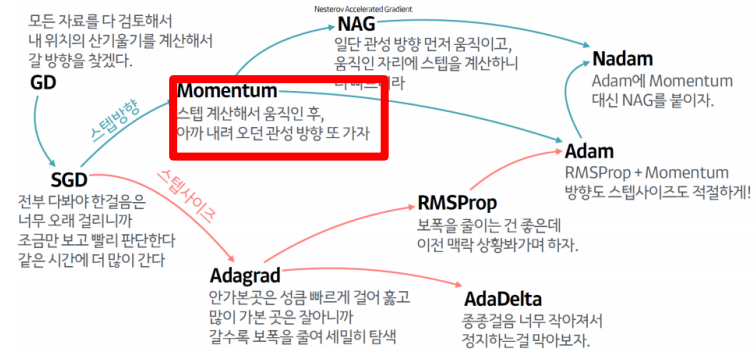
stuck at a local minimum



Optimizers – Momentum

모멘텀(momentum)은 원래, 물리학에서 사용되는 용어인데, 물리학(**physics**)에서는 물체가 한 방향으로 지속적으로 변동하려는 경향을 가리키는 말로 동력, 추진력, 여세, 타성의 뜻으로도 이해될 수 있습니다.

<https://m.blog.naver.com/amoayhwh/221945117527>



$$\hat{\mathbf{w}}(t+1) := \hat{\mathbf{w}}(t) - \alpha \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}(t))}{\partial \hat{\mathbf{w}}(t)}$$

$$\hat{\mathbf{w}}(t+1) := \hat{\mathbf{w}}(t) - m_t \longrightarrow \text{관성속도 (velocity)}$$

Optimizers – Momentum

$$\hat{\mathbf{w}}(t+1) := \hat{\mathbf{w}}(t) - \alpha \cdot \mathbf{m}_t \longrightarrow \text{관성속도 (velocity)}$$

$$\mathbf{m}_t = \left(\gamma \cdot \mathbf{m}_{t-1} + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(t))}{\partial \hat{\mathbf{w}}(t)} \right)$$

Momentum



Local Minima

Saddle point

Gradient가 0이라 하더라도,

v값이 더해지면서 멈추지 않고 옆으로 이동할 수 있다!

$$\hat{\mathbf{w}}(t+1) := \hat{\mathbf{w}}(t) - \alpha \cdot \left(\gamma \cdot \mathbf{m}_{t-1} + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(t))}{\partial \hat{\mathbf{w}}(t)} \right)$$

$$\mathbf{m}_0 = \left(\gamma \cdot \mathbf{m}_{0-1} + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(0))}{\partial \hat{\mathbf{w}}(0)} \right) = \left(\frac{\partial \text{Cost}(\hat{\mathbf{w}}(0))}{\partial \hat{\mathbf{w}}(0)} \right)$$

$$\mathbf{m}_1 = \left(\gamma \cdot \mathbf{m}_0 + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(1))}{\partial \hat{\mathbf{w}}(1)} \right) = \left(\gamma \cdot \left(\frac{\partial \text{Cost}(\hat{\mathbf{w}}(0))}{\partial \hat{\mathbf{w}}(0)} \right) + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(1))}{\partial \hat{\mathbf{w}}(1)} \right)$$

$$\mathbf{m}_2 = \left(\gamma \cdot \mathbf{m}_1 + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(2))}{\partial \hat{\mathbf{w}}(2)} \right) = \left(\gamma^2 \cdot \left(\frac{\partial \text{Cost}(\hat{\mathbf{w}}(0))}{\partial \hat{\mathbf{w}}(0)} \right) + \gamma \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}(1))}{\partial \hat{\mathbf{w}}(1)} + \frac{\partial \text{Cost}(\hat{\mathbf{w}}(2))}{\partial \hat{\mathbf{w}}(2)} \right)$$

Optimizers – Momentum

$$\hat{\mathbf{w}}(t+1) := \hat{\mathbf{w}}(t) - \mathbf{v}_t \longrightarrow \text{관성속도 (velocity)}$$

$$\hat{\mathbf{w}}(t+1) := \hat{\mathbf{w}}(t) - \gamma \cdot \mathbf{v}_{t-1} + \alpha \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}(t))}{\partial \hat{\mathbf{w}}(t)}$$



Local Minima

Saddle point

Gradient가 0이라 하더라도,
v값이 더해지면서 멈추지 않고 옆으로 이동할 수 있다!

Pros

- Faster convergence than traditional SGD

Cons

- As the ball accelerates down the hill, how do we know that we don't miss the local minima?

Optimizers – Momentum

SGD

```
CLASS torch.optim.SGD(params, lr=<required parameter>, momentum=0, dampening=0,  
    weight_decay=0, nesterov=False, *, maximize=False, foreach=None,  
    differentiable=False) [SOURCE]
```

Implements stochastic gradient descent (optionally with momentum).

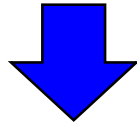
input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
 μ (momentum), τ (dampening), *nesterov*, *maximize*

```
for  $t = 1$  to ... do  
     $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$   
    if  $\lambda \neq 0$   
         $g_t \leftarrow g_t + \lambda \theta_{t-1}$   
    if  $\mu \neq 0$   
        if  $t > 1$   
             $\mathbf{b}_t \leftarrow \mu \mathbf{b}_{t-1} + (1 - \tau) g_t$   
        else  
             $\mathbf{b}_t \leftarrow g_t$   
        if nesterov  
             $g_t \leftarrow g_t + \mu \mathbf{b}_t$   
        else  
             $g_t \leftarrow \mathbf{b}_t$   
    if maximize  
         $\theta_t \leftarrow \theta_{t-1} + \gamma g_t$   
    else  
         $\theta_t \leftarrow \theta_{t-1} - \gamma g_t$   
  
return  $\theta_t$ 
```

Optimizers – AdaGrad

Adaptive Gradients (AdaGrad) provides us with a simple approach, for changing the learning rate over time.

$$\hat{\mathbf{w}}_j(t+1) := \hat{\mathbf{w}}_j(t) - \alpha \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}_j(t))}{\partial \hat{\mathbf{w}}_j(t)}$$



$$\hat{\mathbf{w}}_j(t+1) := \hat{\mathbf{w}}_j(t) - \frac{\alpha}{\sqrt{\sum_{m=1}^t \left(\frac{\partial \text{Cost}(\hat{\mathbf{w}}_j(m))}{\partial \hat{\mathbf{w}}_j(m)} \right)^2} + \epsilon} \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}_j(t))}{\partial \hat{\mathbf{w}}_j(t)}$$

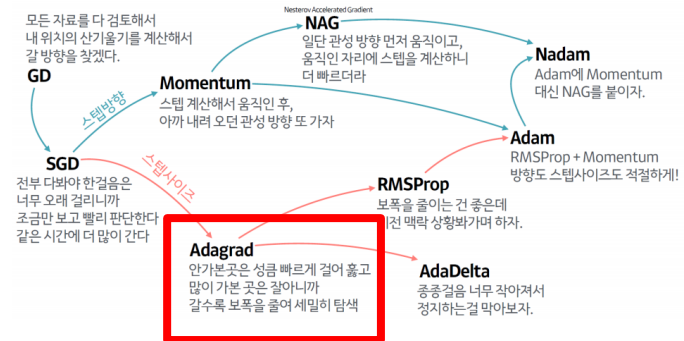
Pros

learning rate is decreasing over time – hence the term *adaptive* learning rate.

AdaGrad: Parameters Gets Different Learning Rates

Cons

learning rate is decreasing over time **Even though not optimal**



Optimizers – AdaGrad

ADAGRAD

```
CLASS torch.optim.Adagrad(params, lr=0.01, lr_decay=0, weight_decay=0,  
    initial_accumulator_value=0, eps=1e-10, foreach=None, *, maximize=False) [SOURCE]
```

Implements Adagrad algorithm.

input : γ (lr), θ_0 (params), $f(\theta)$ (objective), λ (weight decay),
 τ (initial accumulator value), η (lr decay)

initialize : $state_sum_0 \leftarrow 0$

for $t = 1$ **to** ... **do**

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$

$\tilde{\gamma} \leftarrow \gamma / (1 + (t - 1)\eta)$

if $\lambda \neq 0$

$g_t \leftarrow g_t + \lambda \theta_{t-1}$

$state_sum_t \leftarrow state_sum_{t-1} + g_t^2$

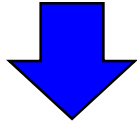
$\theta_t \leftarrow \theta_{t-1} - \tilde{\gamma} \frac{g_t}{\sqrt{state_sum_t + \epsilon}}$

return θ_t

Optimizers – RMSprop (Root Mean Squared Propagation)

It is very close to Adagrad, except for it does not provide the sum of the gradients, but instead an exponentially decaying average.

$$\hat{\mathbf{w}}_j(t+1) := \hat{\mathbf{w}}_j(t) - \alpha \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}_j(t))}{\partial \hat{\mathbf{w}}_j(t)}$$



$$\hat{\mathbf{w}}_j(t+1) := \hat{\mathbf{w}}_j(t) - \left(\frac{\alpha}{\sqrt{v_t} + \epsilon} \right) \cdot \frac{\partial \text{Cost}(\hat{\mathbf{w}}(t))}{\partial \hat{\mathbf{w}}(t)}$$

가중 평균 이동정보 $v_t = \gamma v_{t-1} + (1 - \gamma) g_t^2, g_t^2 = \left(\frac{\partial \text{Cost}(\hat{\mathbf{w}}_j(t))}{\partial \hat{\mathbf{w}}_j(t)} \right)^2$

$$v_0 = \gamma v_{0-1} + (1 - \gamma) g_0^2 = (1 - \gamma) g_0^2$$

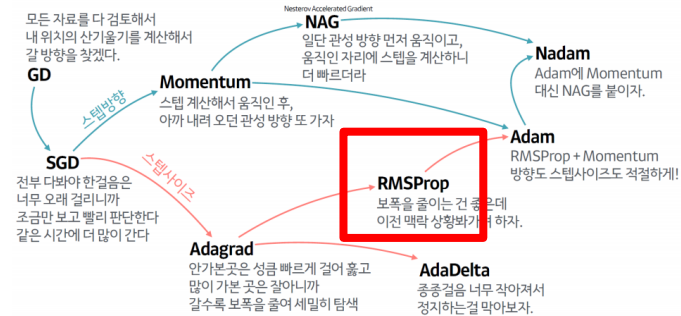
$$v_1 = \gamma v_0 + (1 - \gamma) g_1^2 = \gamma(1 - \gamma) g_0^2 + (1 - \gamma) g_1^2$$

$$v_2 = \gamma v_1 + (1 - \gamma) g_2^2 = \gamma^2(1 - \gamma) g_0^2 + \gamma(1 - \gamma) g_1^2 + (1 - \gamma) g_2^2$$

Pros

RMSprop changes the learning rate slower than Adagrad, but still reaps the benefits of converging relatively fast.

Cons No momentum



Optimizers – RMSprop (Root Mean Squared Propagation)

RMSPROP

```
CLASS torch.optim.RMSprop(params, lr=0.01, alpha=0.99, eps=1e-08, weight_decay=0,  
    momentum=0, centered=False, foreach=None, maximize=False,  
    differentiable=False) [SOURCE]
```

Implements RMSprop algorithm.

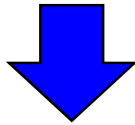
input : α (alpha), γ (lr), θ_0 (params), $f(\theta)$ (objective)
 λ (weight decay), μ (momentum), *centered*
initialize : $v_0 \leftarrow 0$ (square average), $\mathbf{b}_0 \leftarrow 0$ (buffer), $g_0^{ave} \leftarrow 0$

for $t = 1$ **to** $\dots$ **do**
 $g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$
 if $\lambda \neq 0$
 $g_t \leftarrow g_t + \lambda \theta_{t-1}$
 $v_t \leftarrow \alpha v_{t-1} + (1 - \alpha) g_t^2$
 $\tilde{v}_t \leftarrow v_t$
 if *centered*
 $g_t^{ave} \leftarrow g_{t-1}^{ave} \alpha + (1 - \alpha) g_t$
 $\tilde{v}_t \leftarrow \tilde{v}_t - (g_t^{ave})^2$
 if $\mu > 0$
 $\mathbf{b}_t \leftarrow \mu \mathbf{b}_{t-1} + g_t / (\sqrt{\tilde{v}_t} + \epsilon)$
 $\theta_t \leftarrow \theta_{t-1} - \gamma \mathbf{b}_t$
 else
 $\theta_t \leftarrow \theta_{t-1} - \gamma g_t / (\sqrt{\tilde{v}_t} + \epsilon)$

return θ_t

Optimizers – Adam (Adaptive Moment Estimation)

$$\hat{w}_j(t+1) := \hat{w}_j(t) - \alpha \cdot \frac{\partial \text{Cost}(\hat{w}_j(t))}{\partial \hat{w}_j(t)}$$



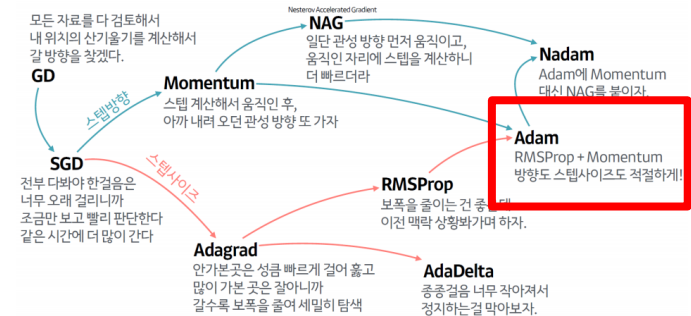
$$\hat{w}_j(t+1) := \hat{w}_j(t) - \left(\frac{\alpha}{\sqrt{v'_t} + \epsilon} \right) \cdot m'_t$$

가중 평균 이동정보 $v_t = \gamma v_{t-1} + (1 - \gamma) g_t^2, g_t^2 = \left(\frac{\partial \text{Cost}(\hat{w}_j(t))}{\partial \hat{w}_j(t)} \right)^2$

$$v'_t = \frac{v_t}{1 - \gamma^t} \quad \bullet \text{Bias Correction}$$

가중 평균 모멘텀 $m_t = \beta m_{t-1} + (1 - \beta) g_t, g_t = \frac{\partial \text{Cost}(\hat{w}_j(t))}{\partial \hat{w}_j(t)}$

$$m'_t = \frac{m_t}{1 - \beta^t} \quad \bullet \text{Bias Correction}$$



Anyone getting into deep learning will probably get the best and most consistent results using Adam

Optimizers – Adam (Adaptive Moment Estimation)

ADAM

```
CLASS torch.optim.Adam(params, lr=0.001, betas=(0.9, 0.999), eps=1e-08,  
weight_decay=0, amsgrad=False, *, foreach=None, maximize=False,  
capturable=False, differentiable=False, fused=False) [SOURCE]
```

Implements Adam algorithm.

input : γ (lr), β_1, β_2 (betas), θ_0 (params), $f(\theta)$ (objective)
 λ (weight decay), *amsgrad*, *maximize*

initialize : $m_0 \leftarrow 0$ (first moment), $v_0 \leftarrow 0$ (second moment), $\widehat{v}_0^{max} \leftarrow 0$

for $t = 1$ **to** $\dots$ **do**

if *maximize* :

$g_t \leftarrow -\nabla_{\theta} f_t(\theta_{t-1})$

else

$g_t \leftarrow \nabla_{\theta} f_t(\theta_{t-1})$

if $\lambda \neq 0$

$g_t \leftarrow g_t + \lambda \theta_{t-1}$

$m_t \leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t$

$v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$

$\widehat{m}_t \leftarrow m_t / (1 - \beta_1^t)$

$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$

if *amsgrad*

$\widehat{v}_t^{max} \leftarrow \max(\widehat{v}_t^{max}, \widehat{v}_t)$

$\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t^{max}} + \epsilon)$

else

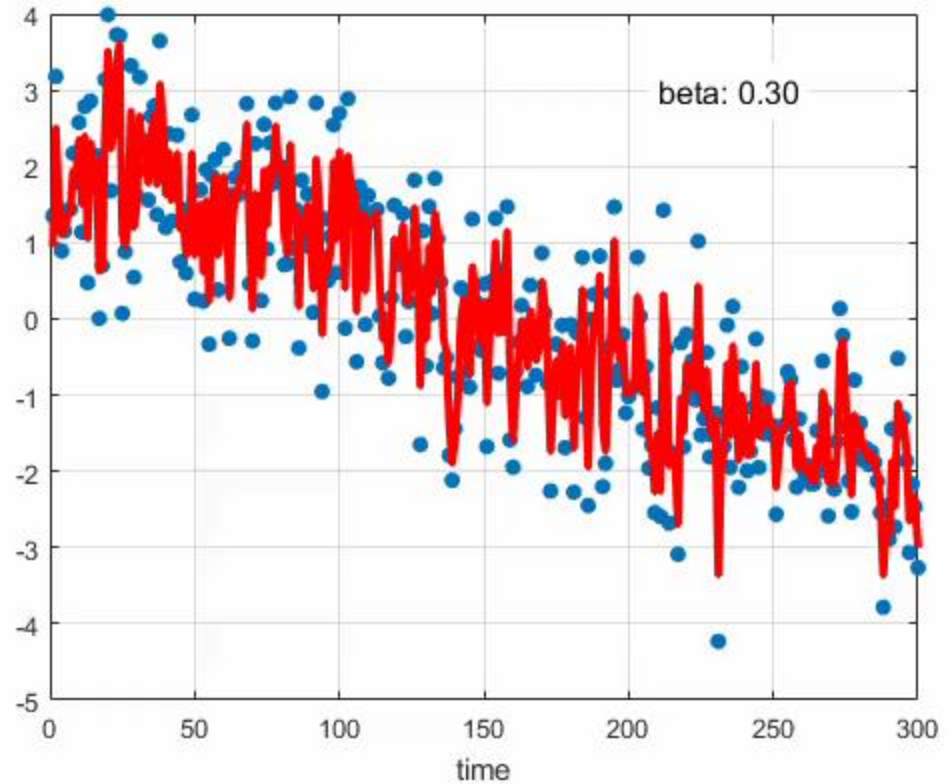
$\theta_t \leftarrow \theta_{t-1} - \gamma \widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon)$

return θ_t

Optimizers – Adam (Adaptive Moment Estimation)

가중 평균 이동정보

$$v_t = \gamma v_{t-1} + (1 - \gamma) g_t^2, g_t^2 = \left(\frac{\partial \text{Cost}(\hat{w}_j(t))}{\partial \hat{w}_j(t)} \right)^2$$



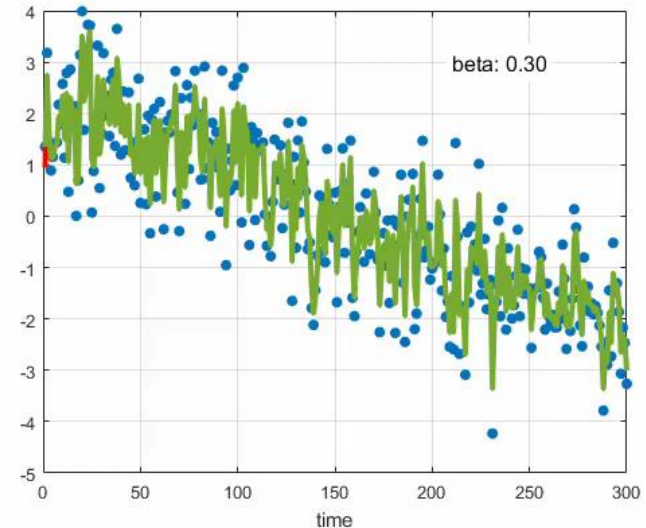
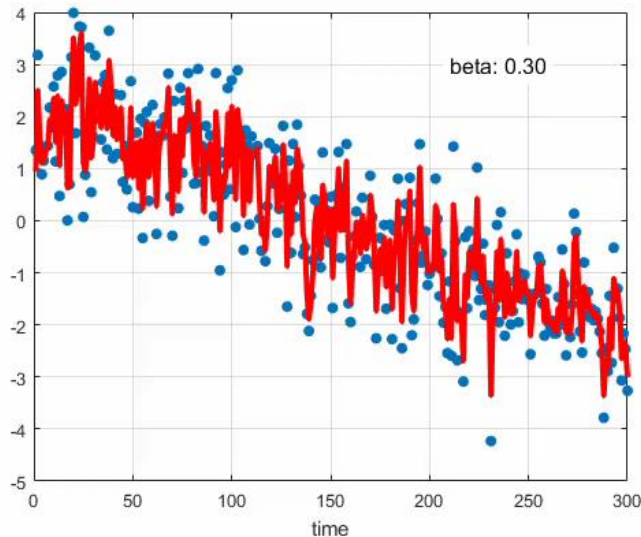
smoothing이 많이 필요한 경우 γ 값을 키워주게 되면 초기 time의 smoothing 결과가 원래의 데이터 포인트들에 비해 낮게 나온다는 것을 알 수 있다

초기 값이 영인 경우 영으로 수렴...

Optimizers – Adam (Adaptive Moment Estimation)

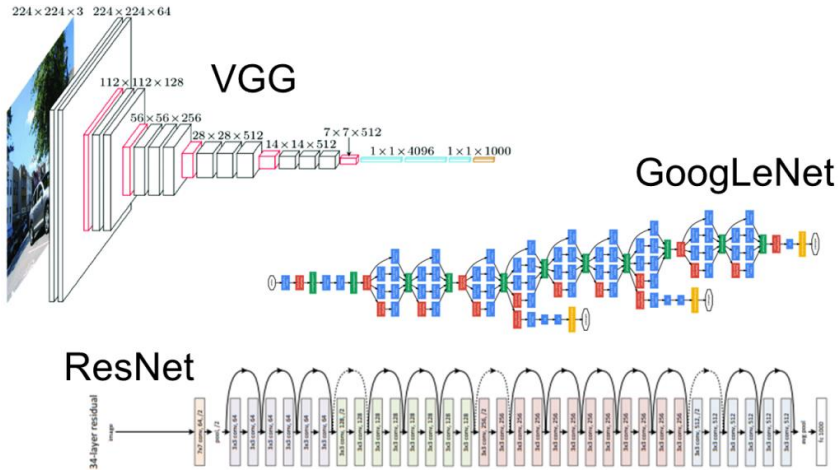
가중 평균 이동정보

$$v_t = \gamma v_{t-1} + (1 - \gamma) g_t^2, g_t^2 = \left(\frac{\partial \text{Cost}(\hat{w}_j(t))}{\partial \hat{w}_j(t)} \right)^2$$

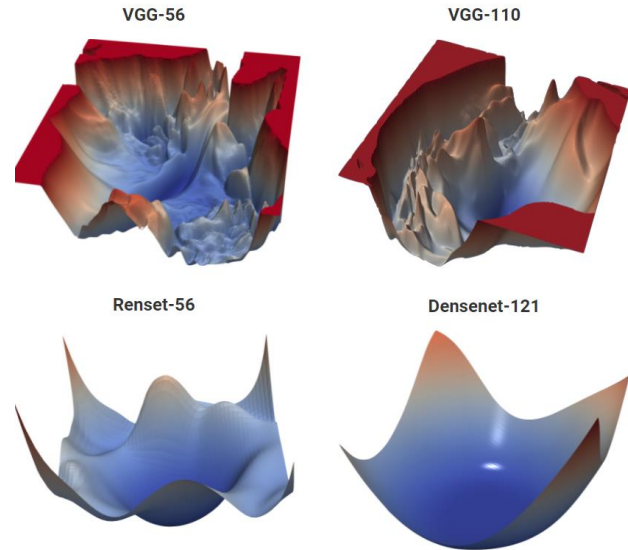


$$v'_t = \frac{v_t}{1 - \gamma^t} \quad \bullet \text{Bias Correction}$$

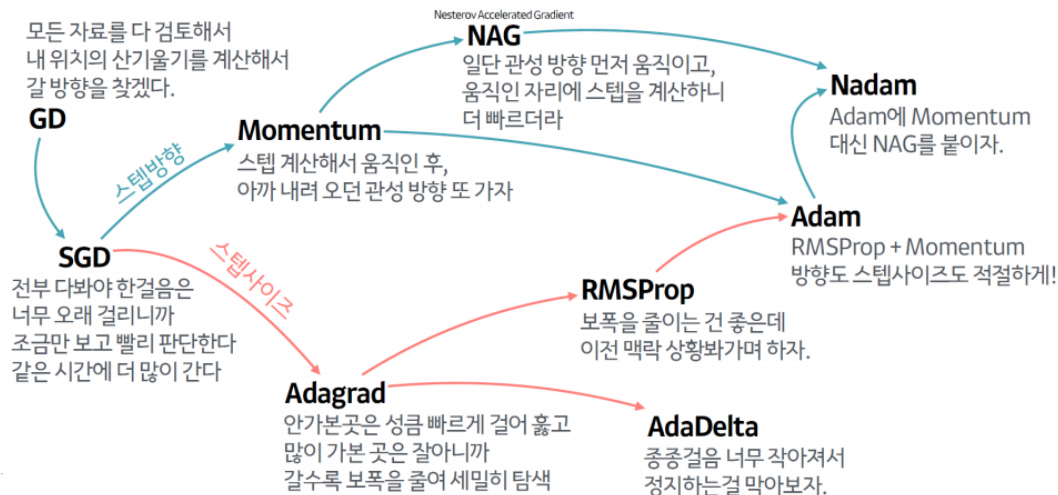
Deep Learning



Parameter Space



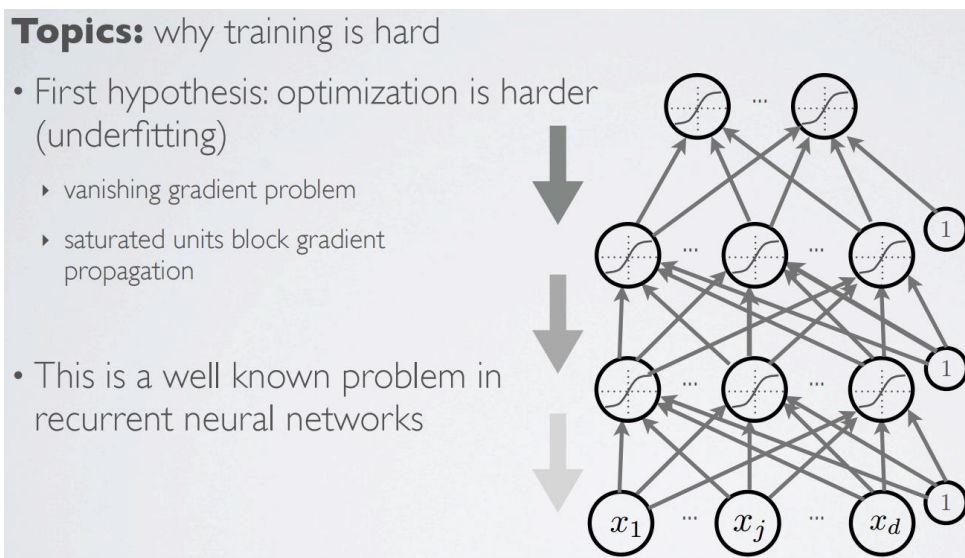
ResNet 50, 152: # of layers = 50, 152



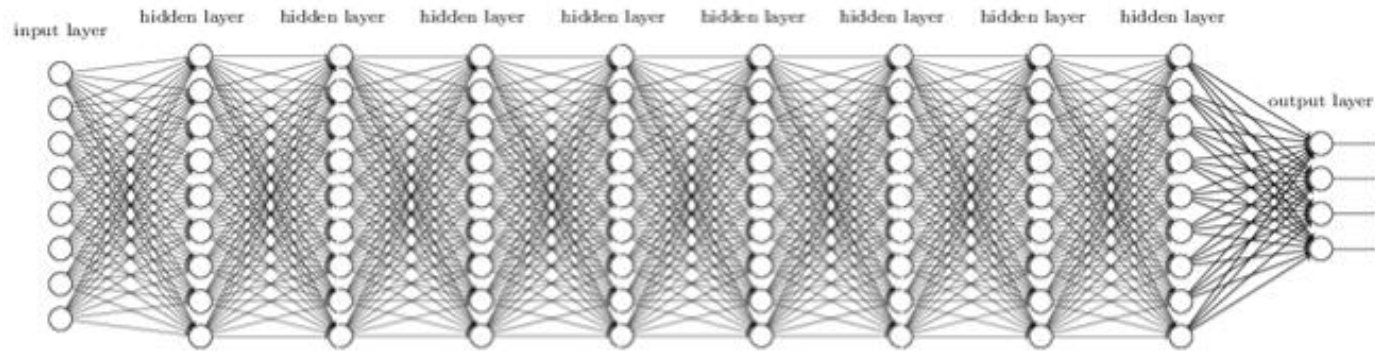
Deep Neural Network (DNN, 심층 신경망)

- 과거 문제점 1: 층(layer)을 많이 사용하면 학습이 안됨

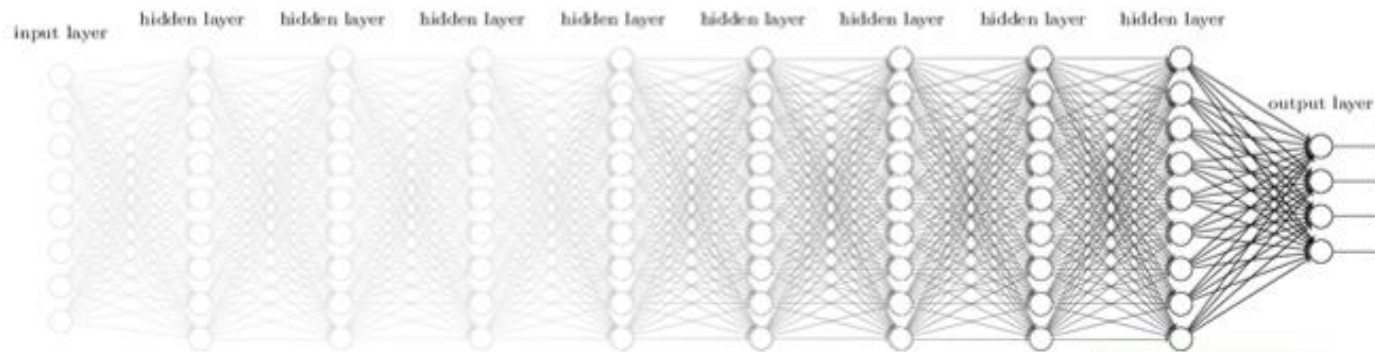
- ✓ Vanishing gradient problem: Sigmoid activation function을 사용할 경우 은닉층의 수가 증가함에 따라 하위 층으로 갈수록 Gradient가 0이 될 가능성이 높아져서 Error Backpropagation이 잘 되지 않음
- ✓ Sigmoid: 실수 범위를 가지는 Input을 0과 1사이의 실수로 매핑
- ✓ Input에서 큰 변화가 있더라도 Output에는 작은 변화



Deep Neural Network (DNN, 심층 신경망)



Deep Neural Network



Vanishing Gradient

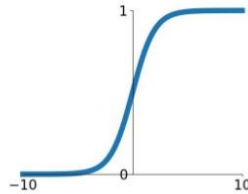
Backpropagation

Activation Function (활성화 함수)

Activation Functions

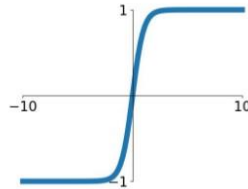
Sigmoid

$$\sigma(x) = \frac{1}{1+e^{-x}}$$



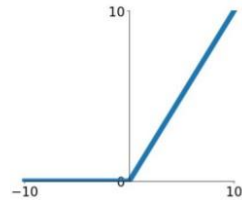
tanh

$$\tanh(x)$$



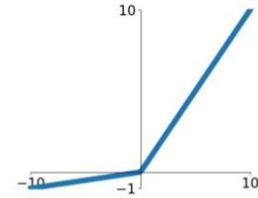
ReLU

$$\max(0, x)$$



Leaky ReLU

$$\max(0.1x, x)$$

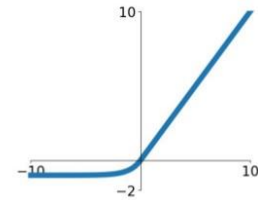


Maxout

$$\max(w_1^T x + b_1, w_2^T x + b_2)$$

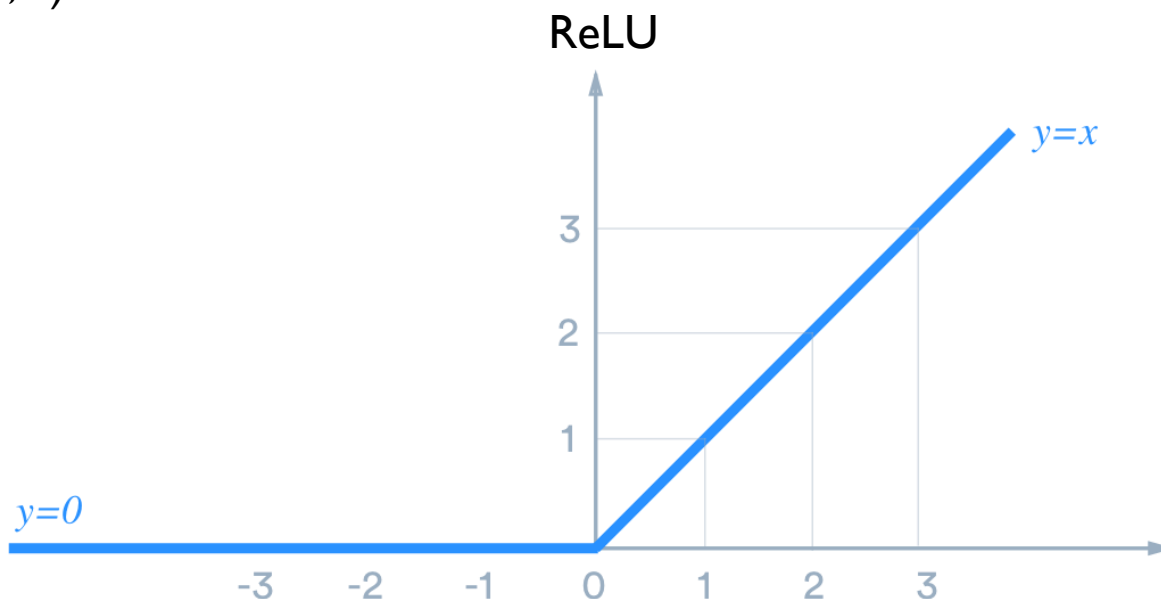
ELU

$$\begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$



Deep Neural Network (DNN, 심층 신경망)

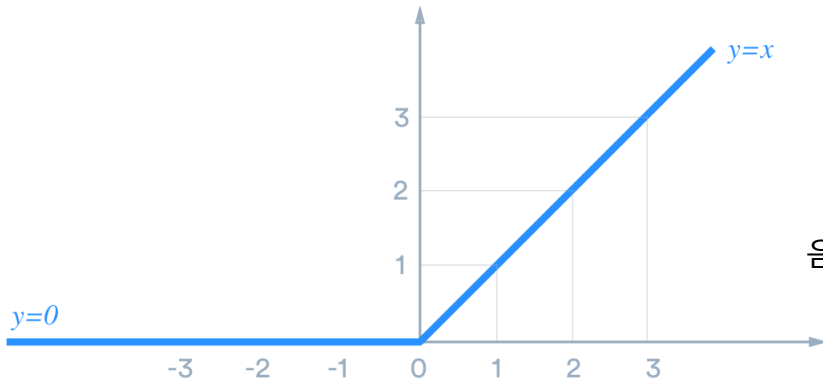
- 해결책 1: Gradient가 소실되지 않은 새로운 활성화 함수 사용!
 - ✓ "Squashing" 특성이 없는 activation function을 사용하자
 - ✓ Rectified Linear Unit (ReLU)
 - ✓ ReLU: 0보다 작을 때는 0을 사용하고, 0보다 큰 값에 대해서는 해당 값을 그대로 사용 (음수에 대해서는 값을 바꾸고, 양수는 값을 바꾸지 않음)
 - ✓ 코드: $\max(0, x)$



Deep Neural Network (DNN, 심층 신경망)

- 해결책 1: Gradient가 소실되지 않은 새로운 활성화 함수 사용!
 - ✓ ReLU 를 변형한 활성화 함수 사용

ReLU

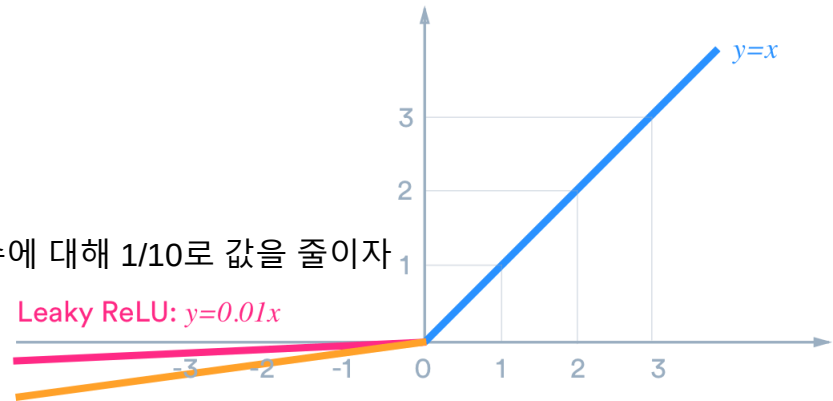


Leaky ReLU

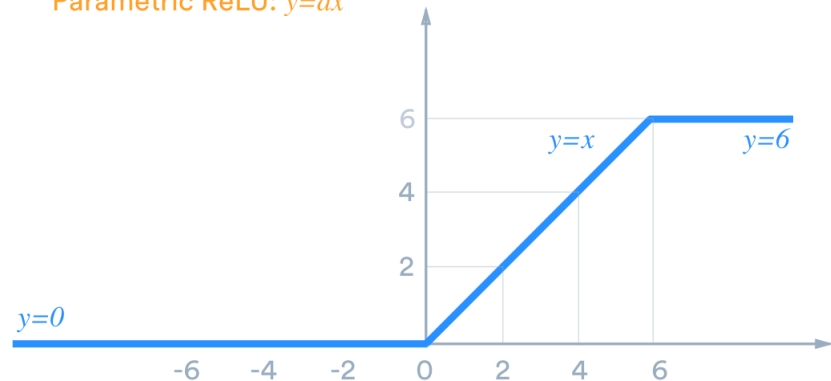
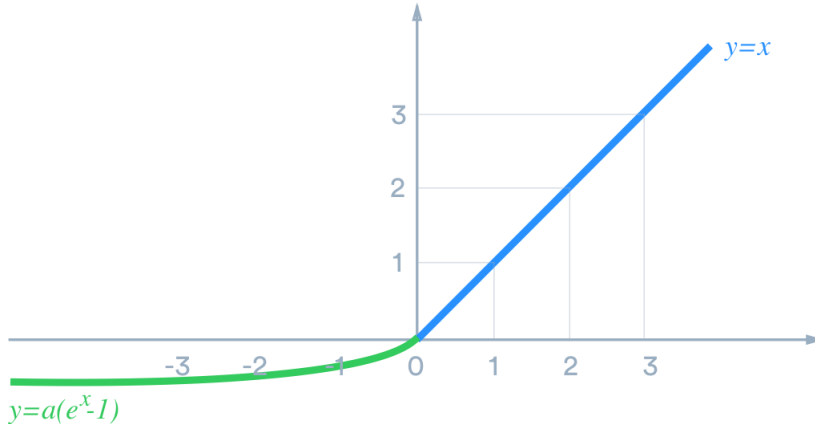
음수에 대해 1/10로 값을 줄이자

Leaky ReLU: $y=0.01x$

Parametric ReLU: $y=ax$



ELU

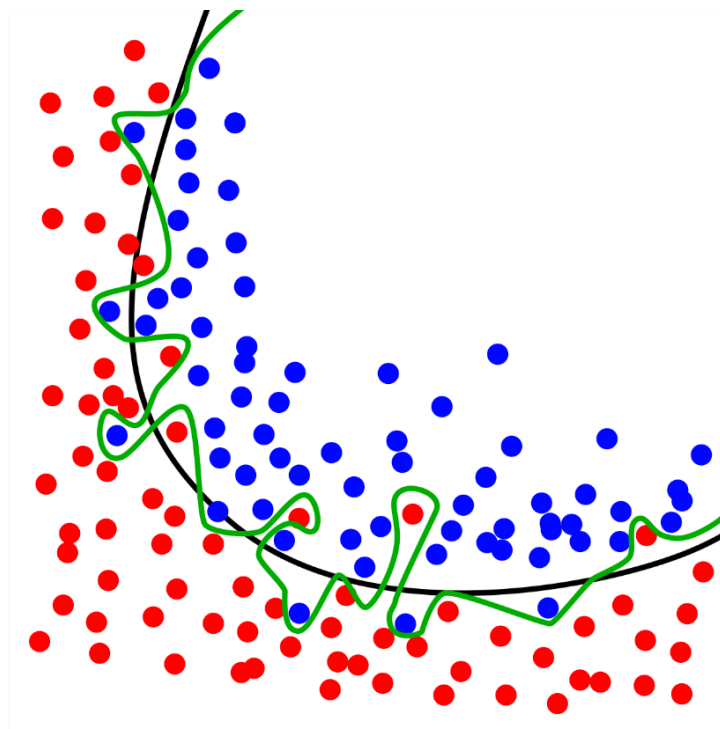
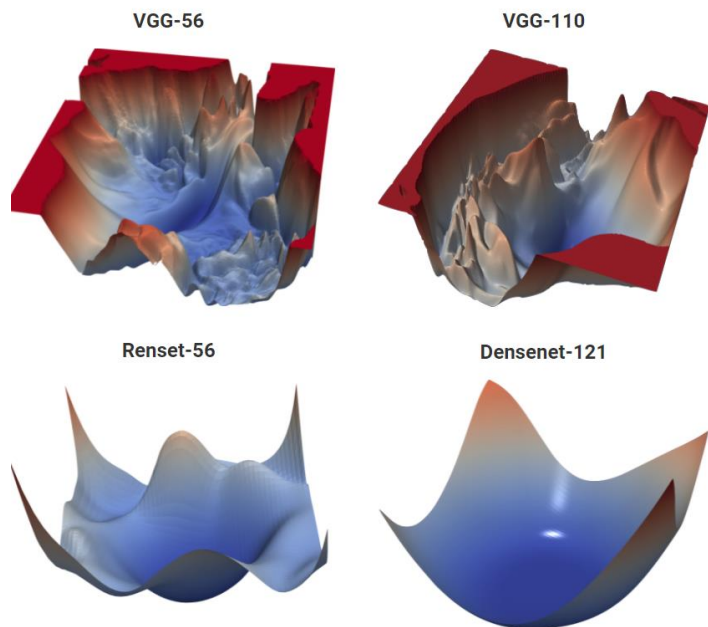


<https://www.tinymind.com/learn/terms/relu>

Deep Neural Network (DNN, 심층 신경망)

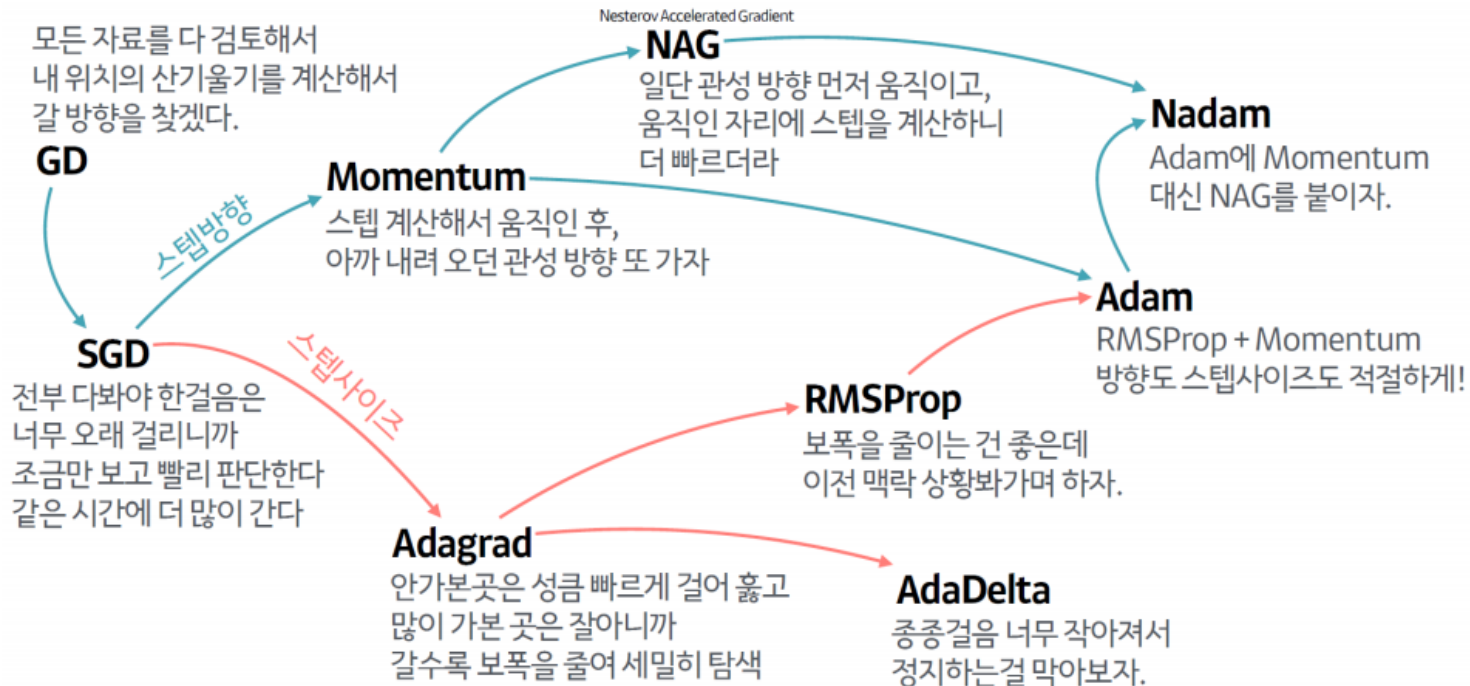
- 과거 문제점 2: 층(layer)을 많이 사용하면 과적합 발생

- ✓ 은닉층과 은닉노드의 수 증가로 어떤 형태의 함수(회귀)나 분류 경계면(분류)도 찾아낼 가능성이 있음
- ✓ 데이터의 숫자가 적을 때는 이 특징이 꼭 좋은 것만은 아님



Deep Neural Network (DNN, 심층 신경망)

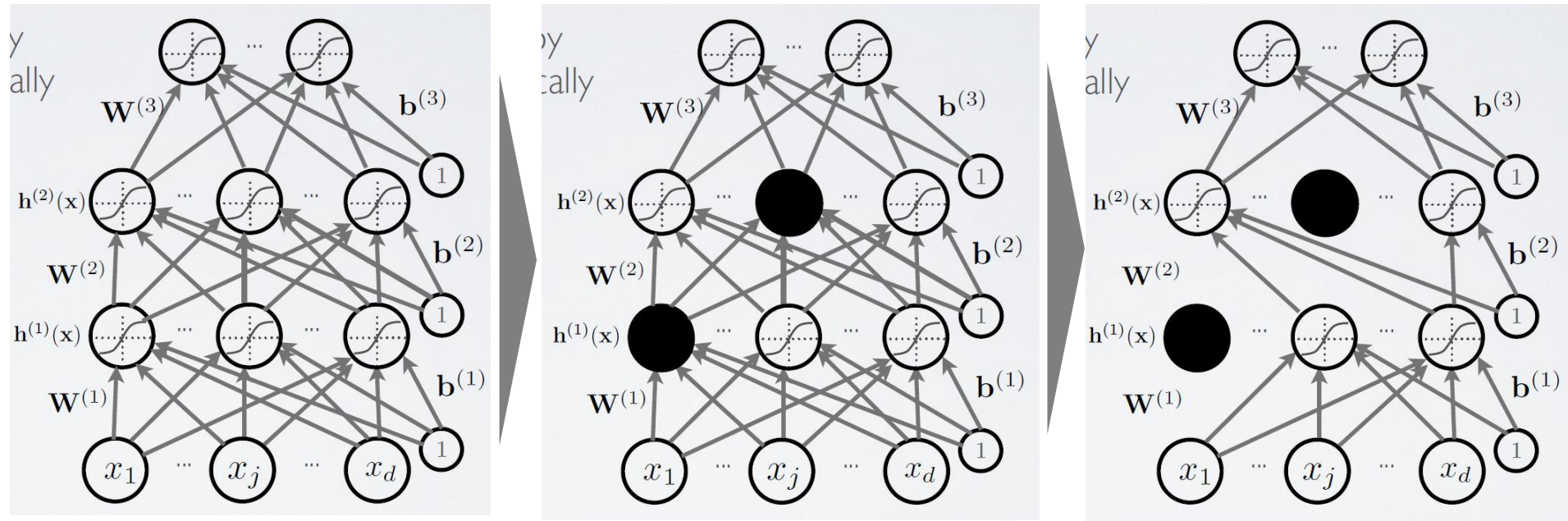
• 해결책 2-1: Optimizer



Deep Neural Network (DNN, 심층 신경망)

• 해결책 2-2: Drop-Out (모델 관점)

- ✓ 학습 과정에서 일부러 특정 노드들에 연결된 가중치들은 업데이트를 하지 말자
- ✓ 어떤 노드를 업데이트 시키지 않을 것인가는 무작위로 그때그때 바뀌가며 결정



Deep Neural Network (DNN, 심층 신경망)

- 해결책 2-3: Data Augmentation

EOD